

go语言

1.切片原理 和数组的区别

1. **数组**是固定长度，常量。**切片**长度是可以改变，所以是一个可变的数组
2. **数组**是值类型，数组传递是值。**切片**是引用类型，切片传递是指针
3. **数组**不能使用append添加元素(因为数组是常量)，**切片**通过append添加元素

改变slice底层数据：使用s[i] = xxx赋值，不能只对值拷贝进行更改；通过slice的array字段可以拿到数组的地址；要想真的改变外层 slice，只有将返回的新的 slice 赋值到原始 slice，或者向函数传递一个指向 slice 的指针。

2.切片初始化问题

```
1 var a = []string
2
3 var a = []int{}
4
5 a := make([]int, 2,10)//make初始化时必须指定len，否则len=cap=0会panic
6
7 a := []int{}//一个元素也没有，需要往里append元素，append会在栈中开辟底层数组
```

3.map什么内容不能成为key

slice, map, function，不能用==来判断。数字、string、bool、array、channel、指针可以，以及 包含前面类型的 struct

4.map和sync map（读写问题）

golang 的 map 本身不是 thread-safe 的，但是通过使用读写锁，我们可以构造出一个 thread-safe 的 syncmap。sync.map

在 syncmap 中实际存在两个 map：read map & dirty map，syncmap管理的一条条entry，每个 entry有两种状态，nil&enpunged

sync.Map

使用syncmap，放弃了类型保护机制（即不是所有key/value）类型的数据都能存放在syncmap中；失去了一致性快照保护；

map+sync.Mutex--->map+sync.RWMutex(多线程操作时性能不高)----->sync.Map

看过啥底层包（net，sync等等）

5.GC垃圾回收机制

（回答思路：先说三色标记，再说三色标记（并行标记）的问题，再引出强弱三色不变性，再说混合写屏障）

- Go v1.0 原始标记-清除：STW

STW：写屏障在执行时需要中断go主程序一段时间，成为stop the world(STW)

同时在STW中，会对整个堆区（heap）进行扫描，清除未标记对象时也会产生一定的heap碎片。

在后期发展过程中将清除工作放在暂停STW之后，与程序同步进行，减少了STW的时间，但还是浪费了大量的时间。



- Go v1.3 三色标记清除，依然STW, 但也做了一定优化，先停止STW，再执行清除操作
- Go v1.5 写屏障技术让垃圾回收与用户代码得以并行执行。从而只有在执行写屏障和很短一段时间内才需要进行 STW。
- Go v1.8 引入混合写屏障机制，优化STW的时间

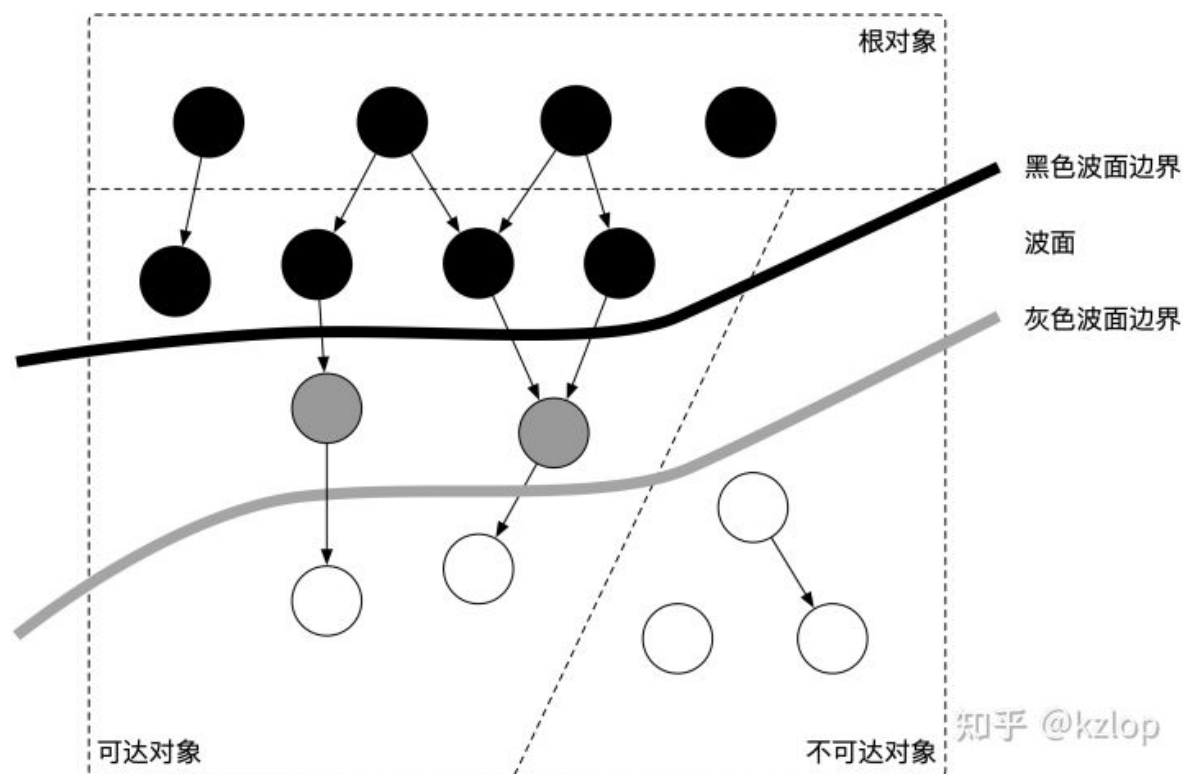
5.1 垃圾回收算法 -- 三色标记清除

(不说) 三色抽象只是一种描述追踪式回收器的方法，在实践中并没有实际含义，它的重要作用在于从逻辑上严密推导标记清理这种垃圾回收方法的正确性。也就是说，当我们谈及三色标记法时，通常指标记清扫的垃圾回收。

(说) 从垃圾回收器的视角来看，三色抽象规定了三种不同类型的对象，并用不同的颜色相称：

- 白色对象（可能死亡）：未被回收器访问到的对象。在回收开始阶段，所有对象均为白色，当回收结束后，白色对象均不可达。
- 灰色对象（波面）：已被回收器访问到的对象，但回收器需要对其中的一个或多个指针进行扫描，因为他们可能还指向白色对象。
- 黑色对象（确定存活）：已被回收器访问到的对象，其中所有字段都已被扫描，黑色对象中任何一个指针都不可能直接指向白色对象。

这样三种不变性所定义的回收过程其实是一个 **波面 (Wavefront)** 不断前进的过程，这个波面是黑色对象和白色对象的边界，灰色对象存在于这个波面。



三色标记清除的过程

1. 起初所有对象都是白色。
2. 从根出发扫描所有可达对象，标记为灰色，放入待处理队列。

3. 从队列取出灰色对象，将其引用对象标记为灰色放入队列，**自身标记为黑色**。
4. 重复 3，直到灰色对象队列为空。此时白色对象即为垃圾，进行回收。

5.2 屏障技术-目前正在使用的

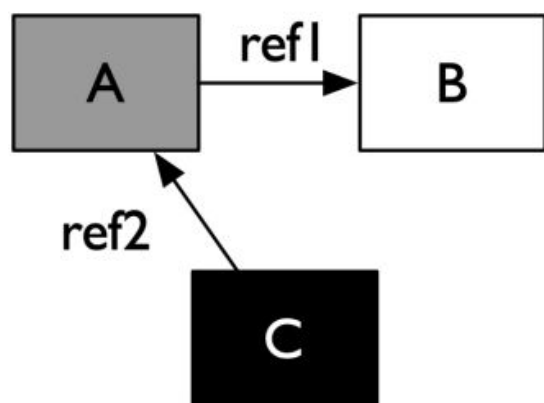
并行标记清理产生的问题

没有STW的垃圾清除中，用户态代码在回收过程中会并发的更新对象图，从而赋值器和回收器可能对对象图的结构产生不同的认知，这时以一个固定的三色波面作为回收过程前进的边界则不再合理。

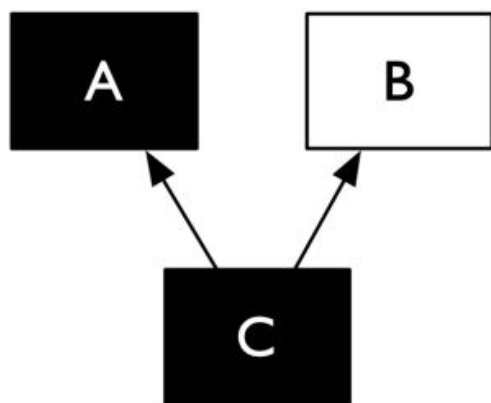
垃圾回收器的正确性体现在：不应出现对象的丢失，也不应错误的回收还不需要回收的对象。

作为内存屏障的一种，写屏障（Write Barrier）是一个在并发垃圾回收器中才会出现的概念。

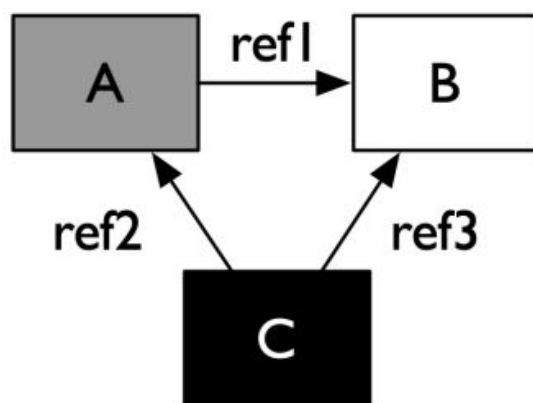
垃圾回收器



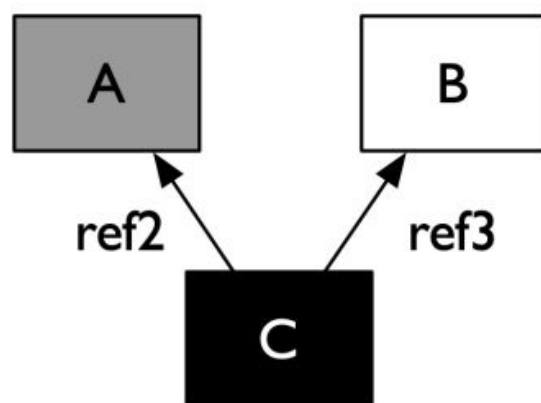
初始状态



赋值器



$C.ref3 = C.ref2.ref1$



$A.ref1 = nil$

scan(A)



知乎 @kzlop

(说) 可以证明, 当以下两个条件同时满足时会破坏垃圾回收器的正确性:

- 条件 1: 赋值器修改对象图, 导致某一黑色对象引用白色对象;
- 条件 2: 从灰色对象出发, 到达白色对象的、未经访问过的路径被赋值器破坏。

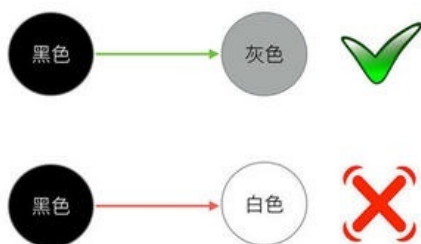
只要能够避免其中任何一个条件, 则不会出现对象丢失的情况, 因为:

- 如果条件 1 被避免, 则所有白色对象均被灰色对象引用, 没有白色对象会被遗漏;
- 如果条件 2 被避免, 即便白色对象的指针被写入到黑色对象中, 但从灰色对象出发, 总存在一条没有访问过的路径, 从而找到到达白色对象的路径, 白色对象最终不会被遗漏。

强-弱三色不变性

- 强三色不变式: 不存在黑色对象引用到白色对象的指针

强三色不变式

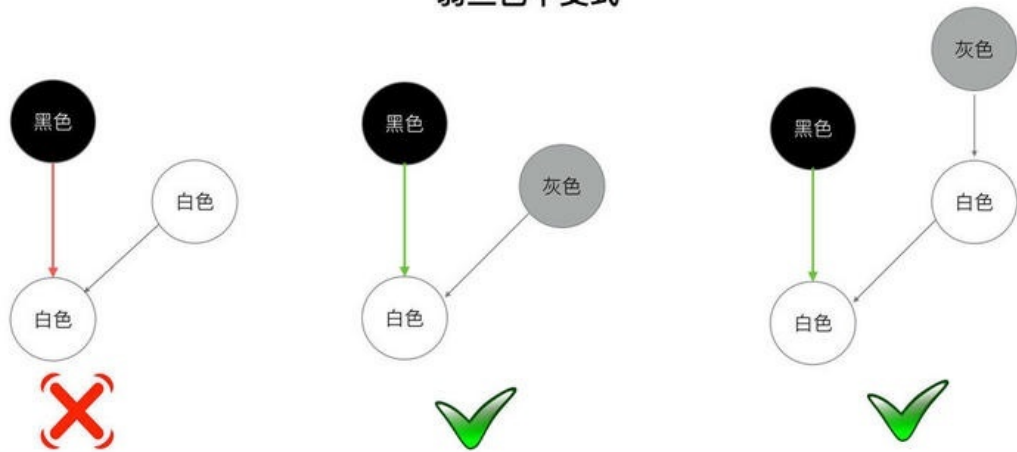


强制性的不允许黑色对象引用白色对象

知乎 @kzlop

- 弱三色不变式: 所有被黑色对象引用的白色对象都处于灰色保护状态

弱三色不变式



黑色对象可以引用白色对象，
白色对象存在其他灰色对象对它的引用，或者可达它的链路上游存在灰色对象

知乎 @kzlop

插入屏障 (insertion barrier) 技术，又称为**增量更新屏障 (incremental update)** 其核心思想是把赋值器对已存活的对象集合的插入行为通知给回收器，进而产生可能需要额外（重新）扫描的对象。如果某一对象的引用被插入到已经被标记为黑色的对象中，这类屏障会**保守地**将其作为非白色存活对象，**以满足强三色不变性**。（条件1）

Dijkstra 插入屏障 [Dijkstra et al. 1978] 作为诸多插入屏障中的一种，对于插入到黑色对象中的白色指针，无论其在未来是否会被赋值器删除，该屏障都会将其标记为可达（着色）。在这种思想下，避免满足条件 1 的出现

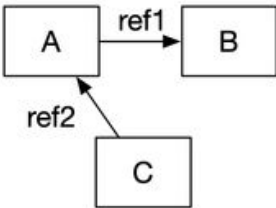
垃圾回收期间，将新建引用的对象标记为灰色。

避免，白对象建立黑对象引用后，白对象仍被误删。

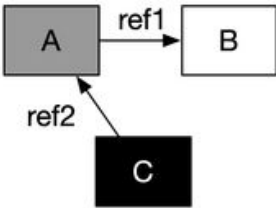
因为栈和寄存器不能hook，所以栈没有插入屏障机制，那么栈的垃圾回收仍需要STW

Dijkstra 写屏障：在写入前，对指针所要指向的对象进行着色

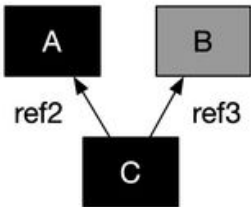
垃圾回收器



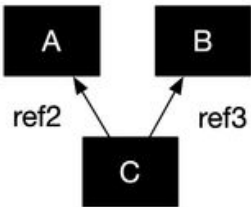
初始状态



Scan(C)

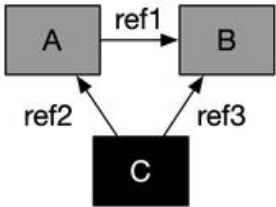


scan(A)

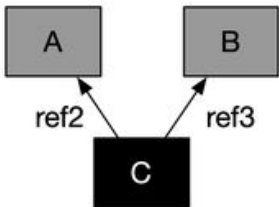


scan(B)

赋值器



DijkstraWritePointer(C.ref3, C.ref2.ref1)

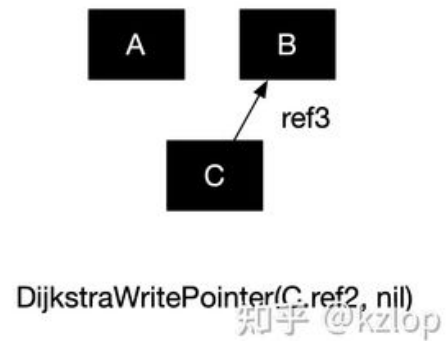


DijkstraWritePointer(A.ref1, nil)



scan(p)

此时 A 虽然着为黑色，但确为标记阶段产生的垃圾
在下一个 GC 阶段中由于赋值器已经不包含该对象的引用，
会始终保持白色，从而一定会被回收



删除屏障 (deletion barrier) 技术，又称为**写屏障**。其思想是当赋值器从灰色或白色对象中删除白色指针时，通过写屏障将这一行为通知给并发执行的回收器。这一过程很像是在操纵对象图之前对图进行了一次快照。

如果一个指针位于波面之前，则删除屏障会保守地将**目标对象标记为非白色存活对象**，进而避免条件 2 来**满足弱三色不变性**。（条件2）具体来说，Yuasa 删除屏障对于在回收过程中，对于被赋值器删除最后一个指向这个对象导致该对象不可达的情况，仍将其对象进行着色

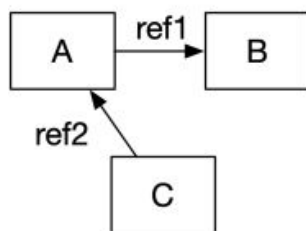
Yuasa 删除屏障的优势则在于不需要标记结束阶段的重新扫描，结束时候能够准确的回收所有需要回收的白色对象。缺陷是 Yuasa 删除屏障会拦截写操作，进而导致波面的退后，产生冗余的扫描

垃圾回收期间，引用被删除的对象标记为灰色。

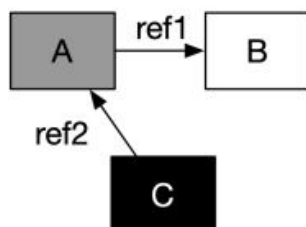
避免，白对象和灰对象引用被删除，但立刻新建立了和黑对象的引用关系后，白色对象仍被误删。

Yuasa 写屏障：在写入前，对指针所在对象进行着色

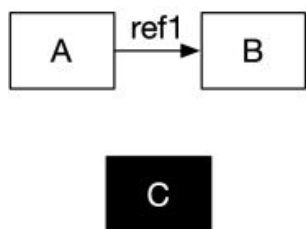
垃圾回收器



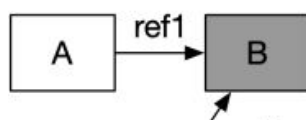
初始状态



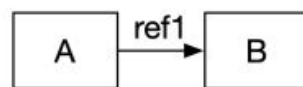
scan(C)



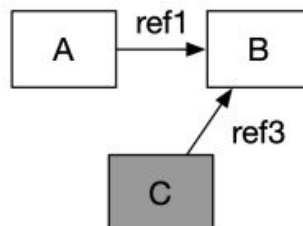
scan(C)



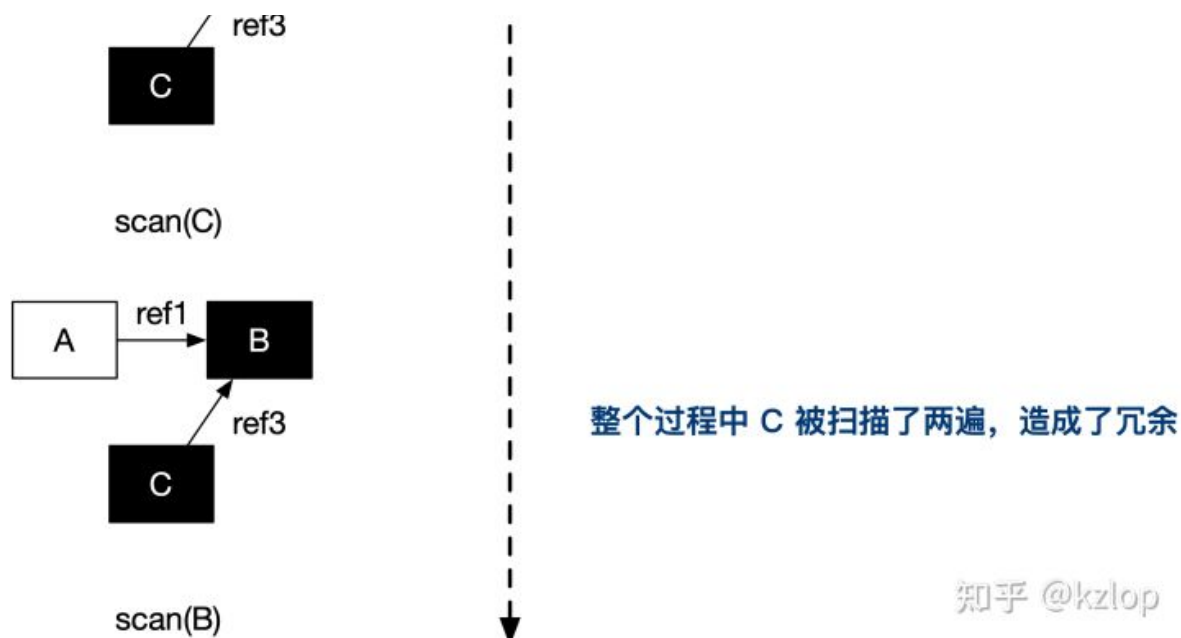
赋值器



YuasaWritePointer(C.ref2, nil)



YuasaWritePointer(A.ref3, B)



(说) 混合写屏障

在诸多屏障技术中，Go 使用了 Dijkstra 与 Yuasa 屏障的结合，即**混合写屏障 (Hybrid write barrier) 技术** [Clements and Hudson, 2016]。Go 在 1.8 的时候为了简化 GC 的流程，同时减少标记终止阶段的重扫成本，将 Dijkstra 插入屏障和 Yuasa 删除屏障进行混合，形成混合写屏障，沿用至今。

该屏障提出时的**基本思想是：对正在被覆盖的对象进行着色，且如果当前栈未扫描完成，则同样对指针进行着色**。混合写屏障为了消除栈的重扫过程，因为一旦栈被扫描变为黑色，则它会继续保持黑色，并要求将对象分配为黑色。

插入和删除屏障的结合，同时赋值器也变成了黑色赋值器，见下文。无需STW，但也存在精度问题，它的流程如下：

1. GC开始将栈上的对象全部扫描并标记为黑色。
2. GC期间，任何在栈上创建的新对象，均为黑色。
3. 被删除的对象标记为灰色。
4. 被添加的对象标记为灰色。

阶段	说明	赋值器状态
清扫终止	为下一个阶段的并发标记做准备工作，启动写屏障	STW
标记	与赋值器并发执行，写屏障处于开启状态	并发
标记终止	保证一个周期内标记任务完成，停止写屏障	STW
内存清扫	将需要回收的内存归还到堆中，写屏障处于关闭状态	并发
内存归还	将过多的内存归还给操作系统，写屏障处于关闭状态	并发

2.4 内存分配

听起来Golang的GC机制好像并不高级，既没有分代回收的概念，也没有压缩整理内存空间的能力。带着这样的疑问，我不得不再看看golang的内存分配机制，来尝试寻找答案。

主要结构

Go 的内存分配器主要包含以下几个核心组件：

- heapArena: 保留整个虚拟地址空间
- mheap: 分配的堆，在页大小为 8KB 的粒度上进行管理
- mspan: 是 mheap 上管理的一连串的页
- mcentral: 收集了给定大小等级的所有 span
- mcache: 为 per-P 的缓存。

传统意义上的栈被 Go 的运行时霸占，不开放给用户态代码；而传统意义上的堆内存，又被 Go 运行时划分为了两个部分，一个是 Go 运行时自身所需的堆内存，即堆外内存；另一部分则用于 Go 用户态代码所使用的堆内存，也叫做 Go 堆。Go 堆负责了用户态对象的存放以及 goroutine 的执行栈。

小对象分配

当对一个对象（<32KB）分配内存时，会将该对象所需的内存大小调整到某个能够容纳该对象的大小等级（size class），并查看 mcache 中对应等级的 mspan，通过扫描 mspan 的 freeindex 来确定是否能够进行分配。

当没有可分配的 mspan 时，会从 mcentral 中获取一个所需大小空间的新的 mspan，从 mcentral 中分配会对其进行加锁，但一次性获取整个 span 的过程均摊了对 mcentral 加锁的成本。

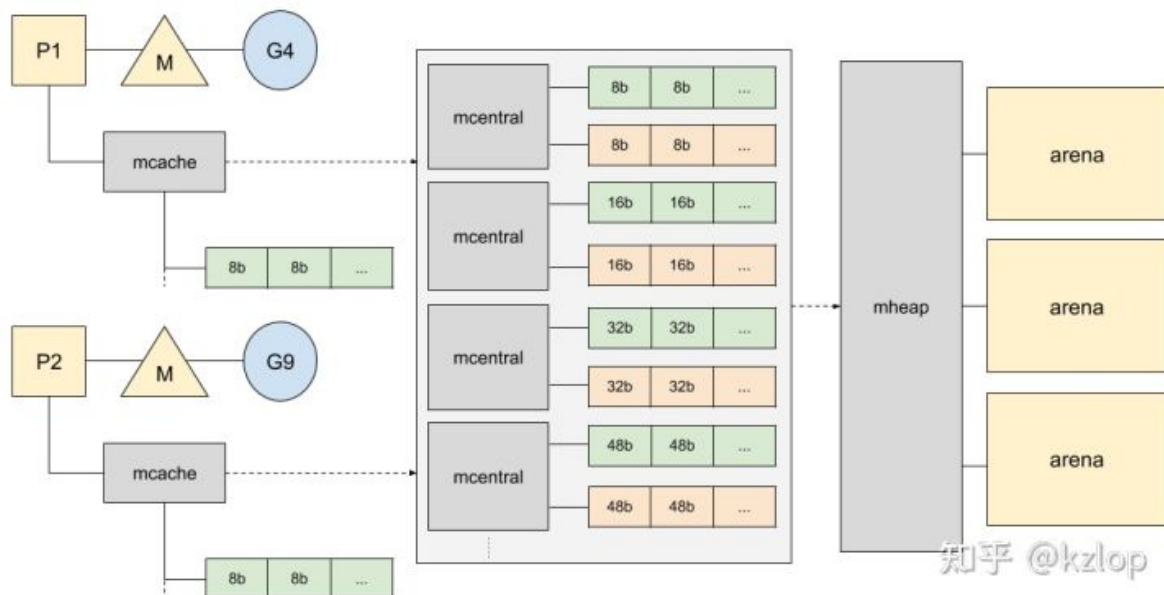
如果 mcentral 的 mspan 也为空时，则它也会发生增长，从而从 mheap 中获取一连串的页，作为一个新的 mspan 进行提供。而如果 mheap 仍然为空，或者没有足够大的对象来进行分配时，则会从操作系统中分配一组新的页（至少 1MB），从而均摊与操作系统沟通的成本。

微对象分配

对于过小的微对象（<16B），它们的分配过程与小对象的分配过程基本类似，但是是直接存储在 mcache 上，并由其以 16B 的块大小直接进行管理和释放。

大对象分配

大对象分配非常粗暴，不与 mcache 和 mcentral 沟通，直接绕过并通过 mheap 进行分配。



6. 知道那一些锁

需要导入sync包

1. sync.Mutex: 多个进程时，同一时间只有一个进程处理加锁的代码，连接释放后别的线程才能处理这段代码
2. sync.RWMutex(): 多个线程时，只能有一个线程处理写操作，但是同时可以有多个线程处理读操作，处理速度比简单互斥锁快很多

同步原语: sync.Mutex, sync.RWMutex, sync.WaitGroup, sync.Once, sync.Cond.; 但是在更多情况下使用更为高级的channel进行同步;

sync包属于悲观锁

atomic 包，可以在不形成临界区和创建互斥量的情况下完成并发安全的值替换操作，这个包应用的便是乐观锁的原理。

悲观锁

悲观锁(Pessimistic Lock), 顾名思义, 就是很悲观, 每次去拿数据的时候都认为别人会修改, 所以每次在拿数据的时候都会上锁, 这样别人想拿这个数据就会block直到它拿到锁。传统的关系型数据库里边就用到了很多这种锁机制, 比如行锁, 表锁等, 读锁, 写锁等, 都是在做操作之前先上锁。

乐观锁

乐观锁(Optimistic Lock), 顾名思义, 就是很乐观, 每次去拿数据的时候都认为别人不会修改, 所以不会上锁, 但是在更新的时候会判断一下在此期间别人有没有去更新这个数据, 可以使用版本号等机制。乐观锁适用于多读的应用类型, 这样可以提高吞吐量, 像数据库如果提供类似于write_condition机制的其实都是提供的乐观锁。

互斥锁的底层实现

state是当前互斥锁处于的状态, sema是用于控制锁的信号量; 当持有锁的goroutine释放锁后, 会释放sema信号量, 会唤醒之前抢锁阻塞的goroutine获取锁

设计上存在两种模式: 正常模式和饥饿模式

正常模式下, 所有阻塞在等待队列中的goroutine会按顺序进行锁获取, 当唤醒一个等待队列中的goroutine时, 此goroutine并不会直接获取到锁, 而是会和新请求锁的goroutine竞争。通常新请求锁的goroutine更容易获取锁, 这是因为新请求锁的goroutine正在占用cpu片执行, 大概率可以直接执行到获取到锁的逻辑。

饥饿模式下, 新请求锁的goroutine不会进行锁获取, 而是加入到队列尾部阻塞等待获取锁。

饥饿模式的触发条件:

- 当一个goroutine等待锁的时间超过1ms时, 互斥锁会切换到饥饿模式

panic: lock后重复加锁会导致; 在lock前unlock;

7.go map底层实现

总体来说golang的map是hashmap, 是使用数组+链表的形式实现的, 使用拉链法消除hash冲突。

golang的map由两种重要的结构, hmap和bmap(下文中都有解释), 主要就是hmap中包含一个指向bmap数组的指针, key经过hash函数之后得到一个数, 这个数低位用于选择bmap(当作bmap数组指针的下表), 高位用于放在bmap的[8]uint8数组中, 用于快速试错。然后一个bmap可以指向下一个bmap(拉链)。

hashmap的数据结构: 会把每一个key都存到一个不同的bucket中, 当然会存在hash碰撞: 即不同的key被存放到相同的bucket中; 这时采用拉链法消除hash冲突, 即将bucket存储一个链表, 每一个存入bucket中的key都会被添加到这个链表中; hash会被归为一类, 这一类的hash对应的key会被存放到一个bucket中, 并且一个单独的bucket最多存放8个hash, 如果再有更多的key, 则再创建一个bucket, 并通过overflow字段连接起来;

```
1  ageMp := make(map[string]int)
2
3  ageMp := make(map[string]int, 20) //分配了底层的数组, 并指定了map长度
4
5  var ageMp map[string]int //nil, 不能向其直接添加元素, 会panic
```

makemap函数会初始化hmap的各种字段, 完成对map的初始化内存分配

slice在make后会形成一个新的slice, 在函数内部对参数进行操作的时候, 只会影响拷贝的副本而不会影响原本的参数值, 但是map则会被影响, 因为makemap函数返回的是一个指针, 值传递后也是指向同一个map;

哈希表的特点是会有一个哈希函数，对你传来的key进行哈希运算，得到唯一的值，一般情况下都是一个数值。Golang的 `map` 中也有这么一个哈希函数，也会算出唯一的值，对于这个值的使用，Golang也是很有意思。

golang的map遍历是无序遍历的

golang rehash

hash就避免不了hash冲突问题

两个或以上的键被hash到同一个bucket时，就发生了冲突。Go是用链地址法来解决键冲突。但是由于同一个bucket只能存放8个键值对，所有超过8个键值对就会再创建一个bucket，将之后的键值对放入新建的bucket中，并用类似链表的方式将后面的bucket连接起来。

负载因子（装载因子）是用来衡量一个hash表冲突的情况：负载因子=键数量/bucket数量

负载因子过小，说明空间利用率低；

负载因子过大，说明冲突严重，存取效率低。

hash表中的负载因子过大时，需要申请更多的bucket，并需要对所有的键值对重新组织，使其能够均匀分布到bucket中，这个过程称之为rehash。

map扩容

增量扩容

原有的key不会一次性搬迁完毕，而是每次最多搬迁两个bucket；核心代码中的`hashGrow()`并没有真正进行搬迁，它只是分配好了新的buckets并把老的buckets挂到了`oldbuckets`字段上。只有在插入，修改，删除key的时候才会进行真正的搬迁。

为了降低负载因子，需要扩容；当负载因子> 6.5或overflow的数量>2¹⁵，则会发生扩容

第一种负载因子>6.5

当所有的bucket都装满时，装载因子=8

对于这种情况，元素太多而bucket太少，扩容时将B加1，直接将bucket扩容两倍，老的buckets挂在oldbucket上

第二种数量>2¹⁵

这意味着虽然每个bucket中的数据并不多，但是存在着大量的overflow bucket，这很有可能是先插入删除了大量的数据，插入过程并没有触发条件1，注意overflow bucket中的所有元素删除了之后也不会释放overflow bucket，这导致的问题是虽然数据量不大，但是查找时需要遍历的bucket(包括overflow bucket)多，也会降低效率

这种情况其实元素没那么多，但是overflow bucket很多，说明很多的bucket都没有装满，因此扩容时会开辟一个新的bucket空间，将老的bucket中的元素移动到新bucket是的同一个bucket中的key排列的更紧密。这就把原来在overflow bucket中的元素移动到了bucket中

8.go mod ; go sum

go mod管理项目，不需要指定GOPATH，go mod init 项目名称 就可以创建一个go.mod文件，此时就可以使用这个项目了

包含go.mod文件的目录也被称为模块根，也就是说，go.mod 文件的出现定义了它所在的目录为一个模块。

go.sum 是记录所依赖的项目的版本的锁定。（版本号）

9.一致性哈希了解过吗？简单介绍下原理？一致性哈希的应用场景有哪些？为什么比普通哈希好

一致性哈希（consistent hashing）是另一个种在工程中使用较为广泛的数据分布方式。一致性哈希最初在P2P网络中作为分布式哈希表（DHT）的常用数据分布算法。一致性哈希的基本方式是使用一个哈希函数计算数据或数据特征的哈希值，令该哈希函数的输出值域为一个封闭的环，即哈希函数输出的最大值是最小值的前序。将节点随机分布到这个环上，每个节点负责处理从自己开始顺时针至下一个节点的全部哈希值域上的数据。

3. **一致性哈希将整个哈希输出空间设置为一个环形区域，将整个哈希值空间组织成一个虚拟的圆环。**
而且这个哈希值空间的大小为 $[0, 2^{32}-1]$ 。
4. 服务器分布在环形的固定点上。
 2. key经过计算，也映射到环形上。
 3. 顺时针寻找下一个节点。

2.2 为什么要取模 2^{32}

一致性hash算法是允许对任意一个数字取模的。但是，如果数字太小，扩容时服务器数量大于该数字，就会存在环空间节点重复。所以一般选择一个固定大的数来取模运算（ $2^{32}=4294967296$ （最大的非符号整形数，也是ip地址的数值空间））。

2.3 为什么需要想象成环形

只有环形可以覆盖所有hash值，不然就必须在首或者尾存在节点。

- 从 $0-2^{32}$ ，不用环形，如果 $2^{32}-1$ 上没有节点，那 $2^{32}-1$ 这个数据就没地方存放。
- 从 $2^{32}-0$ ，不用环形，如果0上没有节点，那0这个数据就没地方存放。
- 用环形，首尾没有节点，也可以找到下一个。

2.4 虚拟节点

上面的简单的一致性hash的方案在某些情况下但依旧存在问题：一个节点宕机之后，数据需要落到距离他最近的节点上，会导致下个节点的压力突然增大，可能导致雪崩，整个服务挂掉。

可以用虚拟节点解决，用实际节点虚拟复制而来的节点被称为“虚拟节点”，将虚拟节点分散在环上，并映射到实际节点上。

这个就解决之前的问题，某个节点宕机之后，存储及流量压力并没有全部转移到某台机器上，而是分散到了多台节点上。解决了节点宕机可能存在的雪崩问题。当物理节点多的时候，虚拟节点多，这个的雪崩可能就越小。

10.defer的底层实现

关键字 defer 用于注册延迟调用，每次 defer 都会把一个函数压入栈中，函数返回前再把延迟的函数取出并执行即 defer 的调用直到return前才会被执行。

1.3.2 例1：主函数拥有匿名返回值的的影响(不影响返回值)

一个主函数拥有一个匿名的返回值，返回时使用字面值，比如返回“1”、“2”、“Hello”这样的值，这种情况下defer语句是无法操作返回值的，如下所示：

```
1 func test() int {
2     var i int
3     defer func() {
4         i++
5     }()
6
7     return 1
8 }
```

上面的return语句，直接把1写入栈中作为返回值，延迟函数无法操作该返回值，所以就无法影响返回值。

1.3.3 例2：主函数拥有匿名返回值的影响(不影响)

一个主函数拥有一个匿名的返回值，返回使用本地或全局变量，这种情况下defer语句可以引用到返回值，但不会改变返回值

```
1 func test() int {  
2     var i int  
3  
4     defer func() {  
5         i++  
6     }()  
7  
8     return i  
9  
10 }
```

函数是匿名返回值的情况，它的执行过程过程如下（例1、例2均如此）：

将 i 赋值给返回值（可以理解成Go自动创建了一个返回值returnValue，相当于执行returnValue = i）

然后检查是否有defer，如果有则执行

返回刚才创建的返回值（returnValue）

根据例1、例2可得结论：

defer中的修改是对 i 执行的，而不是 returnValue，所以defer返回的依然是 returnValue

1.3.4 例3：主函数拥有具名返回值的影响(影响返回值)

主函数声明语句中带名字的返回值，会被初始化成一个局部变量，函数内部可以像使用局部变量一样使用该返回值。如果defer语句操作该返回值，可能会改变返回结果。

```
1 func test() (i int) {  
2     defer func() {  
3         i++  
4     }()  
5  
6     return 0  
7  
8 }
```

以上代码执行过程：

```
1 i = 0  
2 i++  
3 return
```

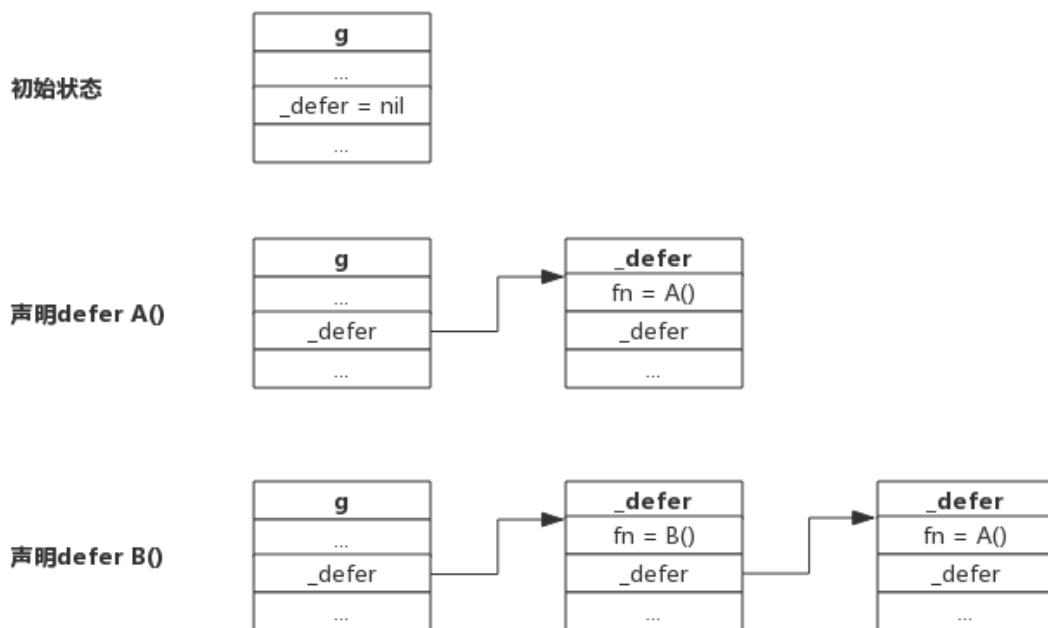
函数真正返回前，在defer中对返回值做了+1操作，所以函数最终返回1。

根据例3可得结论：

由于返回值在方法定义时已经被定义，所以没有创建returnValue 的过程，i 就是 returnValue，defer 对于 i 的修改也会被直接返回

defer 的数据结构跟一般函数类似，也有栈地址、程序计数器、函数地址等等。

它与函数不同的一点是：它含有一个指针，可用于指向另一个 defer，每个goroutine 数据结构中实际上也有一个defer指针，该指针指向一个defer的单链表，每次声明一个 defer 时就将 defer 插入到单链表表头，每次执行 defer 时就从单链表表头取出一个 defer 执行



11.go语言基础问题

for range问题

`for range`在遍历的时候不会改变地址，而是将地址赋值给创建的一个临时变量，这样会导致最后赋值给变量的是最后一个元素的地址；因此在输出元素地址的时候，输出的是同一个值

```

1 package main
2
3 import "fmt"
4
5 type girl struct {
6     Name string
7     Age  int
8 }
9
10 func main() {
11     gl := make(map[string]*girl)
12     studs := []girl{
13         {Name: "Lili", Age: 23},
14         {Name: "Lucy", Age: 24},
15         {Name: "Han Mei", Age: 21},
16     }
17
18     for _, v := range studs {
19         gl[v.Name] = &v
20     }
21
22     for mk, mv := range gl {
23         fmt.Println(mk, "=>", mv.Age)
24     }
25
26 }

```

输出结果是

```
1 | Lili => 21
2 | Lucy => 21
3 | Han Mei => 21
```

defer

defer lifo栈，在执行defer之前，先执行defer中的函数，获取当前的变量值，defer中的函数执行顺序是正序

```
1 func calc(index string, a, b int) int {
2     ret := a + b
3     fmt.Println(index, a, b, ret)
4     return ret
5 }
6
7
8 func main() {
9
10     a := 1
11     b := 2
12     defer calc("1", a, calc("10", a, b))
13     a = 0
14     defer calc("2", a, calc("20", a, b))
15     b = 1
16
17 }
```

输出结果

```
10 1 2 3
20 0 2 2
2 0 2 2
1 1 3 4
```

defer还可以返回一个有名返回值，并且在defer中完成对变量的操作

闭包

闭包=函数+环境

定义在一个函数中的函数（内部定义）

```

1 //函数片段
2 func add(base int) func(int) int {
3     fmt.Printf("%p\n", &base) //打印变量地址，可以看出来 内部函数时对外部传入参数的引用
4
5     f := func(i int) int {
6         fmt.Printf("%p\n", &base)
7         base += i
8         return base
9     }
10
11     return f
12 }
13

```

```

1 //由 main 函数作为程序入口点启动
2 func main() {
3     t1 := add(10)
4     fmt.Println(t1(1), t1(2))
5     t2 := add(100)
6     fmt.Println(t2(1), t2(2))
7 }

```

```

go build main.go
<4 go setup calls>
0xc04204a080
0xc04204a080
0xc04204a080
11 13
0xc04204a0b8
0xc04204a0b8
0xc04204a0b8
101 103

```

重点：延迟调用有些知识点有异曲同工的地方，函数体内某个变量作为defer匿名函数的参数，则在定义defer时已获得值拷贝，否则引用某个变量的地址(引用拷贝)。代码片段如下所示：

```

1 //由 main 函数作为程序入口点启动
2 func main() {
3     x, y := 1,2
4
5     defer func(a int){
6         fmt.Println("defer x, y = ", a, y) //y为闭包引用
7     }(x) //x值拷贝 调用时传入参数
8
9     x += 100
10    y += 200
11
12    fmt.Println(x, y)
13
14 }
15
16

```

```
go build main.go
<4 go setup calls>
101 202
defer x, y = 1 202

Process finished with exit code 0
```

deepequal

在源码里，有对 DeepEqual 函数的非常清楚地注释，列举了不同类型，DeepEqual 的比较情形，这里做一个总结：

类型	深度相等情形
Array	相同索引处的元素“深度”相等
Struct	相应字段，包含导出和不导出，“深度”相等
Func	只有两者都是 nil 时
Interface	两者存储的具体值“深度”相等
Map	1、都为 nil；2、非空、长度相等，指向同一个 map 实体对象，或者相应的 key 指向的 value “深度”相等
Pointer	1、使用 == 比较的结果相等；2、指向的实体“深度”相等
Slice	1、都为 nil；2、非空、长度相等，首元素指向同一个底层数组的相同元素，即 &x[0] == &y[0] 或者 相同索引处的元素“深度”相等
numbers, bools, strings, and channels	使用 == 比较的结果为真

go 中用 for 遍历多次执行 goroutine 会存在什么问题？怎么改进？

<https://blog.csdn.net/luolianxi/article/details/105648309>

不能简单将闭包绑定在一个变量上，要将变量传入 goroutine 闭包，否则会导致并发抢占内存资源，或者导致多个 goroutine 只执行最后一次遍历结果

要将每次遍历的元素都进栈

引用传递和值传递，必须使用值传递

12. 什么是内存逃逸，在什么情况下发生，原理是什么

<https://blog.csdn.net/reigns/article/details/102928763>

使用栈分配：函数的内部中不对外开放的局部变量，只作用于函数中。

使用堆分配：1. 函数的内部中对外开放的局部变量（变量在函数外还被引用）。2. 变量所需内存超出栈所提供最大容量。

指针逃逸

在一个方法内创建了一个局部指针，并将这个指针作为变量返回，则发生内存逃逸

如果一个变量被取地址，那么它就有可能被分配到堆上。然而，还要对这些变量做逃逸分析，如果函数 return 之后，变量不再被引用，则将其分配到栈上。

栈空间不足

是否逃逸取决于栈空间是否足够大，或者栈是否可以判断当前存放的类型的长度

动态类型

被调用的函数的参数是interface或者是不定参数，也会发生逃逸

闭包调用

返回函数

2) 向 channel 发送指针数据。

3) 在闭包中引用包外的值。

4) 在 slice 或 map 中存储指针。

一个经验是：指针指向的数据都是在堆上分配的

好处

- 1、逃逸分析的好处是为了减少gc的压力，不逃逸的对象分配在栈上，当函数返回时就回收了资源，不需要gc标记清除。
- 2、逃逸分析完后可以确定哪些变量可以分配在栈上，栈的分配比堆快，性能好(逃逸的局部变量会在堆上分配,而没有发生逃逸的则有编译器在栈上分配)。
- 3、同步消除，如果你定义的对象的方法上有同步锁，但在运行时，却只有一个线程在访问，此时逃逸分析后的机器码，会去掉同步锁运行。

总结

- 1、逃逸分析在编译阶段完成的。
- 2、编译器会根据变量是否被外部引用来决定是否逃逸，逃逸会分配到堆上。
- 3、编译器会对变量的长度确定性进行判断，如果是不确定的，会分配到堆上；如果是确定的，但是过长也会分配到堆上。
- 4、通过go build -gcflags '-m'命令来观察变量逃逸情况。
- 5、不要盲目使用变量的指针作为函数参数，仅对复杂变量进行引用传递才有意义。
- 6、堆上动态分配内存：无需提前知道变量需要的内存大小，并可进行动态扩容，内存回收由go的GC来处理，性能相比栈要低的多。
- 7、栈上静态分配内存：代表要提前知变量需要内存的大小，栈上内存回收由内核自动处理。

13.堆栈go内存分配

go语言编译器将会分配函数的**局部变量到函数的栈帧**上，由操作系统自动分配释放，存放函数的参数值，局部变量的值等。其操作方式类似于数据结构中的栈。栈**内存**分配计算内置于cpu的指令集，效率很高，但是分配内存量有限。

如果**编译器不能证实局部变量在函数返回后，不会再被引用，编译器将必须分配该变量到垃圾回收堆**上。就是通过new、[malloc](#)、realloc分配的内存块，编译器不会负责它们的释放工作，需要用程序区释放。

在Go语言中，如果一个局部变量在函数返回后仍然被使用，这个变量会从heap，而不是stack中分配内存，就是从栈内存到堆内存，成为内存逃逸

14.函数传指针和传值有什么区别

当我们传一个参数值到被调用函数里面时，实际上是传了这个值的一份copy，当在被调用函数中修改参数值的时候，调用函数中相应实参不会发生任何变化，因为数值变化只作用在copy上。

我们知道，变量在内存中是存放于一定地址上的，修改变量实际是修改变量地址处的内存。只有add1函数知道x变量所在的地址，才能修改x变量的值。所以我们需要将x所在地址&x传入函数，并将函数的参数的类型由int改为*int，即改为指针类型，才能在函数中修改x变量的值。此时参数仍然是按copy传递的，只是copy的是一个指针。

指针类型：*+类型名

指针：&+变量名

Go语言中string，slice，map这三种类型的实现机制类似指针，所以可以直接传递，而不用取地址后传递指针。（注：若函数需改变slice的长度，则仍需要取地址传递指针）

```
1 func test3(){
2     var p *int //默认初始化nil, p是一个指针变量
3     var a int
4     p = &a
5     *p = 200//获取指针变量p指向的值
6
7     fmt.Println(a)
8 }
9
```

15.channel了解吗，channel的工作原理是什么(重要)

channel在goroutine之间传递信息，goroutine之间的通信方式。

创建channel后阻塞接收写入channel的数据

无缓冲的channel是同步的，读写操作必须一起进行。只读或只写则会阻塞死锁

有缓冲的channel是异步的，可以先写，等缓冲区buf满了才会阻塞

GO的channel具有两个特点

- 发送方。对于同一个通道，发送操作（协程或者函数中的），在接收者准备好之前是阻塞的；如果ch中的数据无人接收，就无法再给通道传入其他数据；新的输入无法在通道非空的情况下传入。所以发送操作会等待 channel 再次变为可用状态：就是通道值被接收时（可以传入变量）。
- 接收方。对于同一个通道，接收操作是阻塞的（协程或函数中的），直到发送者可用：如果通道中没有数据，接收者就阻塞了。

```
1 # src/main/blocking3.go
2 package main
3
4 import (
5     "fmt"
6 )
7
8 func f1(in chan int) {
9     number := <- in
10    fmt.Println(number)
11 }
12
13 func main() {
14     out := make(chan int)
15     out <- 2
16     go f1(out)
17     fmt.Println("end")
18 }
```

```
1 1) 第一种情况
2 会造成死锁
3     out <- 2 # 接受者阻塞，因为没有发送者（此时f1还没启动）
4     go f1(out) 在f1中，通道 in 没有传送，所以一直等待；
```

```
1 # src/main/blocking3.go
2 package main
3
4 import (
5     "fmt"
6 )
7
8 func f1(in chan int) {
9     number := <- in
10    fmt.Println(number)
11 }
12
13 func main() {
14     out := make(chan int)
15     go f1(out)
16     out <- 2
17     fmt.Println("end")
18 }
```

```
1 2
2 end
3 2) 不会造成死锁
4     go f1(out)
5     out <- 2
6     f1在等待传送， main相当于一个协程（因为利用了out），等待out传送才关闭；
7 也因为如此f1的打印才得以输出
```

channel在已关闭的情况下

向channel中发送数据会引发:"panic: send on closed channel"

如果channel通道中还有数据、则可以读取其中的数据

从已经关闭的channel读取数据

从一个有缓冲的 channel 里读数据，当 channel 被关闭，依然能读出有效值。只有当返回的 ok 为 false 时，读出的数据才是无效的。

（非缓冲型的关闭chan和缓冲型关闭并且buf中没有元素的chan，从这里面接收元素则会返回一个元素类型的0值，但并不会panic）

16.channel底层原理

底层存在一个循环数组，buf指向底层循环数组（只存在于有缓冲通道中），sendx，recvx指向底层数组，用来索引已发送和已接收的元素；同时也存在一些阻塞的goroutine，他们正在尝试读取channel或者向channel发送数据；lock作为一个互斥锁，用来确保channel的操作都是原子性的

channel 内部维护了两个 goroutine 队列，一个是待发送数据的 goroutine 队列sendq，另一个是待读取数据的 goroutine 队列recvq。

每当对 channel 的读写操作超过了可缓冲的 goroutine 数量，那么当前的 goroutine 就会被挂到对应的队列上，直到有其他 goroutine 执行了与之相反的读写操作，将它重新唤起。

17.channel阻塞接收数据和非阻塞接收数据

非阻塞接收数据

对于空chan：直接返回false

对于接收失败：

不用获取锁，直接return

阻塞接收数据

空chan：调用gopark函数挂起goroutine

接收失败：channel当前没有关闭的话，直接返回false；非缓冲chan的sendq中没有goroutine在等待，缓冲chan中buf里没有元素

即使是关闭状态，但在缓冲型的 channel， buf 里有元素的情况下还能接收到元素；清理掉循环数组中相应位置的值，buf中元素-1，更新对应位置的元素值为ep

但是其他情况例如非缓冲型的关闭chan和缓冲型关闭并且buf中没有元素的chan，从这里面接收元素则会返回一个元素类型的0值，并且清空ep指向的地址；此时并不会panic

但是如果有等待发送的队列sendq，说明channel已经满了：要么是无缓冲的chan，要么是缓冲型的chan但是buf满了；这两种情况下都可以正常接收数据；

对于无缓冲的chan：直接从发送者的栈拷贝到接受者的栈

有缓冲的chan：

阻塞的情况：构造一个sudog，并保存下待接收数据的地址，进入recvq等待队列中，然后调用gopark函数将当前的goroutine挂起；知道被唤醒后继续进行一些扫尾工作

阻塞代码

1.无缓冲信道导致死锁

```
1 package main
2
3 func main(){
4     // 例如本案例:fatal error: all goroutines are asleep - deadlock!
5     // sch := make(chan string)
6     // sch <- "string"
7     // fmt.Println(<-sch)
8
9     // 处理方法,使用双协程,并且接收者在发送者就绪之前,-> 运行正常
10    // sch := make(chan string)
11    //
12    // go func() {
13    //     fmt.Println(<-sch) // 接收者就绪
14    // }()
15    //
16    // time.Sleep(time.Second)
17    // sch <- "string"
18 }
```

2.案例2,改用有缓冲的信道, -> 运行正常

```
1 package main
```

```

2
3 func main() {
4 // 案例2,改用有缓冲的信道, -> 运行正常
5 // sch := make(chan string, 1)
6 // sch <- "hello"
7 // fmt.Println(<-sch)
8
9 // 有缓冲信道导致的死锁,因信道容量满了,再向信道添加数据会产生阻塞,后续无法消费,从而导致死
  锁 - >运行死锁
10 // sch := make(chan string, 1)
11 // sch <- "hello"
12 // sch <- "world" // 因信道满了,无法接收数据从而产生阻塞
13 // fmt.Println(<-sch)
14
15 // 解决方法,生产者跟消费者区分开来,使用不同的协程
16 sch := make(chan int, 1)
17 go func() {
18     count := 0
19     for {
20         sch <- count
21         count++
22     }
23     close(sch) // 生产者不再生产消息,关闭掉通道即可
24 }()
25 go func() {
26     for {
27         fmt.Println(<-sch)
28     }
29 }()
30 select {}
31 }

```

18.向chan发送数据

chansend函数

nil chan: 挂起当前goroutine

不阻塞的chan: 快速检测失败场景; 对chan加锁, 如果chan关闭了, 则panic; 如果接受队列里面有goroutine, 直接将要发送的数据拷贝到接收 goroutine;

对于缓冲的chan, 如果还有缓冲空间, 则将qp指向sendx, 发送游标值+1, 如果超过当前缓冲流量, 则游标值归0并将qp放在缓冲区中;

channel满了的话会构造一个sudog, 将当前goroutine进入发送等待队列sendq, 并挂起当前goroutine; 唤醒后, channel有机会被发送, 然后去掉msg上的channel

19.chan是否会引起资源泄露

会, 当goroutine操作chan, 此时处于发送或者接收阻塞状态, 而chan一直处于满或者空的状态, 因此goroutine会一直处于等待队列中无法被释放, 垃圾回收器也不会回收此类资源

channel的任务定时功能

与timer结合, 实现超时控制或者实现定期执行某个任务

在select选择块中选择不同的情况执行

20.chan关闭过程

会执行closechan函数

对于nil chan: panic: close of nil channel

然后会加锁

对于closed chan: 解锁后panic: close of closed channel

其他正常chan:

修改关闭状态

将recvq的sudog释放, 如果当前接收chan没有忽略接收数据, 则赋予一个响应的0值;

去除sudog中的goroutine, 并相连形成链表;

将sendq中的sudog释放, 如果sudog不为nil, 则取出sudog中的goroutine并panic掉, 并形成链表 (此链表与recvq的链表相同)

解锁

然后遍历链表, 对非空的goroutine进行唤醒, 并移动指针到下一个需要唤醒的goroutine处

关闭操作:

不要从一个 receiver 侧关闭 channel, 也不要存在有多个 sender 时, 关闭 channel。

根据 sender 和 receiver 的个数, 分下面几种情况:

1. 一个 sender, 一个 receiver
2. 一个 sender, M 个 receiver
3. N 个 sender, 一个 receiver
4. N 个 sender, M 个 receiver

对于 1, 2, 只有一个 sender 的情况就不用说了, 直接从 sender 端关闭就好了, 没有问题。重点关注第 3, 4 种情况。

第 3 种情形下, 优雅关闭 channel 的方法是: the only receiver says “please stop sending more” by closing an additional signal channel。解决方案就是增加一个传递关闭信号的 channel, receiver 通过信号 channel 下达关闭数据 channel 指令。senders 监听到关闭信号后, 停止接收数据。

第四种情况, 如果直接还是采取第 3 种解决方案, 由 receiver 直接关闭 stopCh 的话, 就会重复关闭一个 channel, 导致 panic。因此需要增加一个中间人, M 个 receiver 都向它发送关闭 dataCh 的“请求”, 中间人收到第一个请求后, 就会直接下达关闭 dataCh 的指令; 另外, 这里的 N 个 sender 也可以向中间人发送关闭 dataCh 的请求。

21.了解socket编程吗, 其中accept方法是什么

accept函数主要用于服务器端, 一般位于listen函数之后, 默认会阻塞进程, 直到有一个客户请求连接, 建立好连接后, 它返回的一个新的套接字 sockfd_new, 此后, 服务器端即可使用这个新的套接字sockfd_new与该客户端进行通信, 而sockfd 则继续用于监听其他客户端的连接请求。

22.new和make有什么区别

- new(T) 为每个新的类型T分配一片内存, 初始化为 0 并且返回类型为*T的内存地址: 这种方法 **返回一个指向类型为 T, 值为 0 的地址的指针**, 它适用于值类型如数组和结构体; 它相当于 &T{ }。
- make(T) **返回一个类型为 T 的初始值**, 它只适用于3种内建的引用类型: 切片、map 和 channel (
- 换言之, **new 函数分配内存, make 函数初始化**;

24.GMP调度 (必考)

https://blog.csdn.net/weixin_51991615/article/details/124225235

随着服务器硬件迭代升级，配置也越来越高。为充分利用服务器资源，并发编程也变的越来越重要。在开始之前，需要了解一下并发(concurrency)和并行(parallesim)的区别。

并发: 逻辑上具有处理多个同时性任务的能力。able to

并行: 物理上同一时刻执行多个并发任务。现实情况

1) 组成

G: goroutine; G中存放并发执行的代码入口地址、上下文、运行环境（关联的P和M）、运行栈等执行相关的信息。

G的新建、休眠、恢复、停止都受到runtime的管理。

主要保存 goroutine 的一些状态信息以及 CPU 的一些寄存器的值，例如 IP 寄存器，以便在轮到本 goroutine 执行时，CPU 知道要从哪一条指令处开始执行。

M: 线程 (machine)

M代表**OS内核线程、工作线程、系统线程**，是操作系统层面调度和执行的实体。M仅负责执行，M不停地被唤醒或创建，然后执行。

M启动时进入的是运行时的管理代码，由这段代码获取G和P资源，然后执行调度。另外，Go语言运行时单独创建一个监控线程，负责对程序的内存、调度等信息进行监控和控制。

M里面存了两个比较重要的东西，一个是g0，一个是curg。

- g0: 会深度参与运行时的调度过程，比如goroutine的创建、内存分配等
- curg: 代表当前正在线程上执行的goroutine。

所有M是有线程栈的。**最多会有GOMAXPROCS个活跃线程能够正常运行**

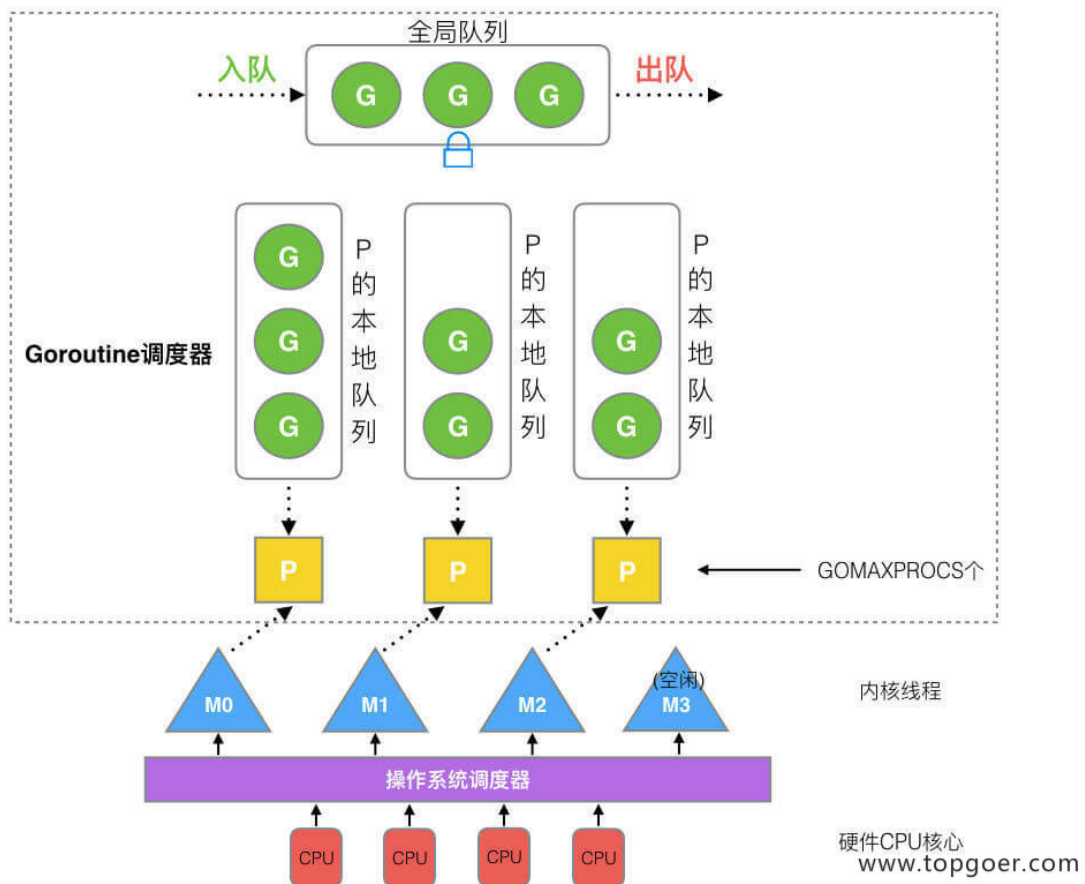
P: P(Processor)是一个抽象的概念，并不是真正的物理CPU。所以当P有任务时需要创建或者唤醒一个系统线程来执行它队列里的任务。所以P/M需要进行绑定，构成一个执行单元。（具有两种队列：本地队列和全局队列）

P决定了同时可以并发任务的数量，可通过GOMAXPROCS限制同时执行用户级任务的操作系统线程。可以通过**runtime.GOMAXPROCS**进行指定。在Go1.5之后GOMAXPROCS被默认设置可用的核数，而之前则默认为1。

调度过程

首先创建一个G对象，G对象保存到P本地队列或者是全局队列。P此时去唤醒一个M。P继续执行它的执行序。M寻找是否有空闲的P，如果有则将该G对象移动到它本身。接下来M执行一个调度循环(调用G对象->执行->清理线程->继续找新的Goroutine执行)。

M执行过程中，随时会发生上下文切换。当发生上下文切换时，需要对执行现场进行保护，以便下次被调度执行时进行现场恢复。Go调度器M的栈保存在G对象上，只需要将M所需要的寄存器(SP、PC等)保存到G对象上就可以实现现场保护。当这些寄存器数据被保护起来，就随时可以做上下文切换了，在中断之前把现场保存起来。如果此时G任务还没有执行完，M可以将任务重新丢到P的任务队列，等待下一次被调度执行。当再次被调度执行时，M通过访问G的vdsoSP、vdsoPC寄存器进行现场恢复(从上次中断位置继续执行)。



GMP不能无限扩增；M的数量最大为10000，但实际程序中数量对应CPU的核数；P是最大活跃线程数量，可以使用GOMAXPROCS定义，默认为核数

调用过程：

GMP的调度流程从go func()开始创建一个goroutine，新建的goroutine优先保存在P的本地队列中，如果P的本地队列已经满了，则会保存到全局队列中。

之后本地队列会从全局队列中获取；M会从P的队列中取一个可执行状态的G来执行，如果P的本地队列为空，则从其他的队列中偷取一半的G，并选择一个可执行的G来执行

当M执行某一个G时候发生系统调用或者阻塞，M阻塞，如果这个时候G在执行，runtime会把这个线程M从P中摘除，然后创建一个新的操作系统线程来服务于这个P，当M系统调用结束时，这个G会尝试获取一个空闲的P来执行，并放入到这个P的本地队列，如果这个线程M变成休眠状态，加入到空闲线程中，然后整个G就会被放入到全局队列中。

GMP是抢占式调度，线程抢占P

25.go func()gmp调度流程

- 1、通过go func()来创建一个goroutine；
- 2、有两个存储G的队列，一个是局部调度器P的本地队列、一个是全局G队列。新创建的G会先保存在P的本地队列中，如果P的本地队列已经满了就会保存在全局的队列中；
- 3、G只能运行在M中，一个M必须持有一个P，M与P是1：1的关系。M会从P的本地队列弹出一个可执行状态的G来执行，如果P的本地队列为空，就会向其他的MP组合偷取一个可执行的G来执行；
- 4、一个M调度G执行的过程是一个循环机制；
- 5、当M执行某一个G时候如果发生了syscall或则其余阻塞操作，M会阻塞，如果当前有一些G在执行，runtime会把这个线程M从P中摘除(detach)，然后再创建一个新的操作系统的线程(如果有空闲的线程可用就复用空闲线程)来服务于这个P；

6、当M系统调用结束时候，这个G会尝试获取一个空闲的P执行，并放入到这个P的本地队列。如果获取不到P，那么这个线程M变成休眠状态，加入到空闲线程中，然后这个G会被放入全局队列中。

调度场景

Channel阻塞（G阻塞）：当goroutine读写channel发生阻塞时候，会调用gopark函数，该G会脱离当前的M与P，调度器会执行schedule函数调度新的G到当前M。可参考上一篇文章[channel探秘](#)。

系统调用：当某个G由于系统调用陷入内核态时，该P就会脱离当前的M，此时P会更新自己的状态为Psyscall，M与G互相绑定，进行系统调用。结束以后若该P状态还是Psyscall，则直接关联该M和G，否则使用闲置的处理器处理该G。

系统监控：当某个G在P上运行的时间超过10ms时候，或者P处于Psyscall状态过长等情况就会调用retake函数，触发新的调度。

主动让出：由于是协作式调度，该G会主动让出当前的P，更新状态为Grunnable，该P会调度队列中的G运行。

goroutine可以无限制的创建吗

如果迅速的开启 goroutine (**不控制并发的 goroutine 数量**)的话，会在短时间内占据操作系统的资源(CPU、内存、文件描述符等)。

- CPU 使用率浮动上涨
- Memory 占用不断上涨
- 主进程崩溃（被杀掉了）

这些资源实际上是所有用户态程序共享的资源，所以大批的 goroutine 开启最终引发的问题不仅仅是自身，还会影响到其他运行的程序。

数据结构

树及其分类

树是由节点和边组成的没有环的数据结构,可能是线性也可能是非线性;

二叉树

每个结点至多拥有两棵子树(即二叉树中不存在度大于2的结点)，并且，二叉树的子树有左右之分，其次序不能任意颠倒。

二叉树有三种的特殊结构，分别为完美二叉树，完全二叉树，完满二叉树。

二叉查找树

二叉查找树有以下四种性质：

1. 任意左子树不为空的节点的左子树的所有节点都小于该节点的值；
2. 任意右子树不为空的节点的右子树的所有节点都大于该节点的值；
3. 任意节点的左右子树都为二叉查找树；
4. 任意两个节点的值都不相等。

二叉查找树的查找、插入的时间复杂度为 $O(\log n)$ 。

但是二叉查找树有一个弊端，可能会退化成有一个有n个节点的线性链。

平衡二叉树（AVL树）出现了。

平衡二叉树是指当且仅当任何节点的两棵子树的高度差不大于1的二叉树。不管执行什么操作，只要不满足这个条件，就要通过旋转保持树的平衡。在代码实现过程中，添加了高度这一变量，用于平衡因子的计算。

红黑树

虽然AVL树可以解决出现退化为线性链的情况，但是每次的插入删除时候的旋转操作很浪费时间，适用于查找比较多的情况，红黑树的出现可以解决需按照浪费时间的情况。

性质：

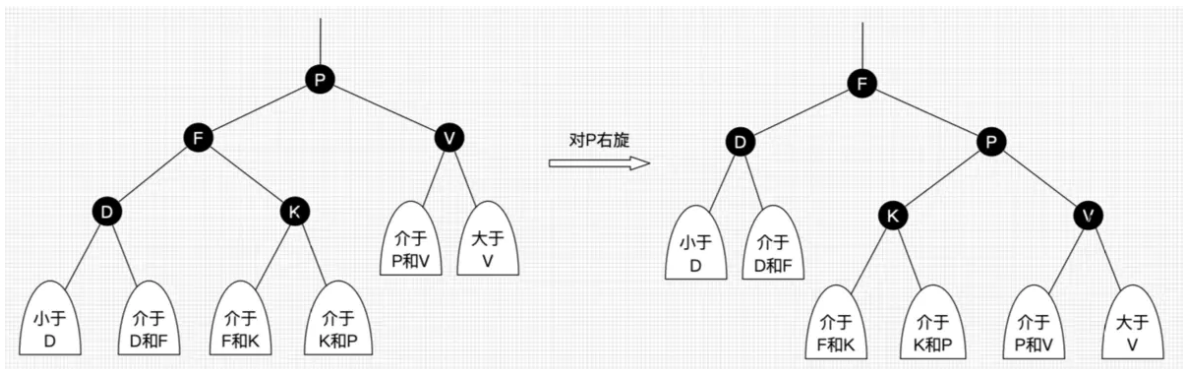
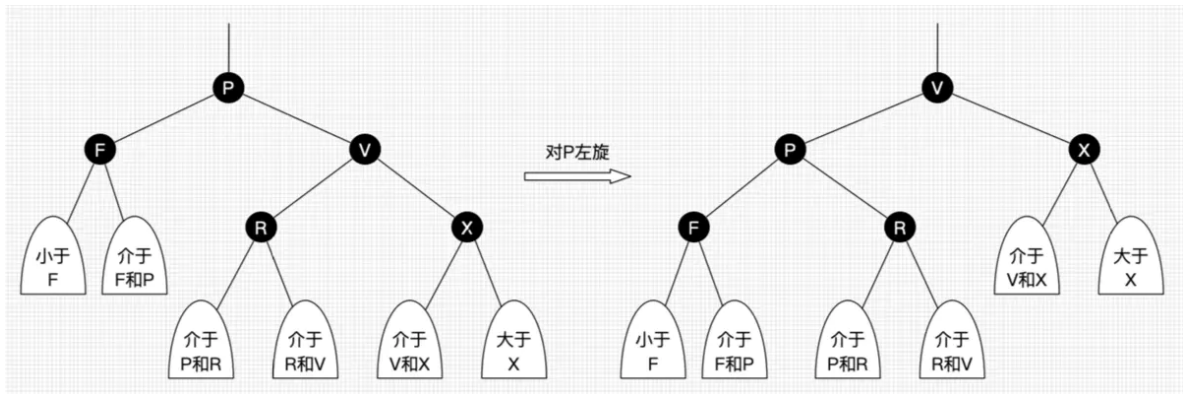
- 1.节点是红色或黑色。
- 2.根节点是黑色。
- 3.每个叶子节点都是黑色的空节点（NIL节点）。
- 4 每个红色节点的两个子节点都是黑色。(从每个叶子到根的所有路径上不能有两个连续的红色节点)
- 5.从任一节点到其每个叶子的所有路径都包含相同数目的黑色节点。

红黑树并不是一个完美平衡二叉查找树，从图1可以看到，根结点P的左子树显然比右子树高，但左子树和右子树的黑结点的层数是相等的，也即任意一个结点到到每个叶子结点的路径都包含数量相同的黑结点(性质5)。所以我们叫红黑树这种平衡为**黑色完美平衡**。

红黑树的自平衡操作

前面讲到红黑树能自平衡，它靠的是是什么？三种操作：左旋、右旋和变色。

- **左旋**：以某个结点作为支点(旋转结点)，其右子结点变为旋转结点的父结点，右子结点的左子结点变为旋转结点的右子结点，左子结点保持不变。如图3。
- **右旋**：以某个结点作为支点(旋转结点)，其左子结点变为旋转结点的父结点，左子结点的右子结点变为旋转结点的左子结点，右子结点保持不变。如图4。
- **变色**：结点的颜色由红变黑或由黑变红。



红黑树插入过程

P: 父节点

S: 叔叔节点

PP：祖父节点



插入的节点一定是红色————为了保证黑色平衡

B- B+树区别

B树	B+树
具有n个关键字的结点含有n+1棵子树	具有n个关键字的结点只含有n棵子树, 即每个关键字对应一棵子树
非叶子结点不仅起索引作用, 而且还带信息, 所有的叶子结点不带信息 (可以视为外部结点或类似看成折半查找判定树的查找失败结点)	叶结点包含信息, 所有非叶结点仅起索引作用, 非叶结点中的每个索引项只含有对应子树的最大关键字和指向该子树的指针, 不含有该关键字对应记录的存储地址
叶结点包含的关键字和其他结点包含的关键字是不重复的	叶结点包含了全部关键字, 即在非叶结点中出现的关键字也会出现在叶结点中
非叶结点上的关键字值等于给定值时立即终止	非叶结点上的关键字值等于给定值时并不终止, 而是继续向下查找, 直到叶结点上的该关键字为止, 即无论查找成功与否, 每次查找都是一条从根结点到叶结点的路径

算法

排序算法

名称	数据对象	稳定性	时间复杂度		额外空间复杂度	描述
			平均	最坏		
冒泡排序	数组	✓	$O(n^2)$		$O(1)$	(无序区, 有序区)。 从无序区透过交换找出最大元素放到有序区前端。
选择排序	数组	✗	$O(n^2)$		$O(1)$	(有序区, 无序区)。 在无序区里找一个最小的元素跟在有序区的后面。对数组: 比较得多, 换得少。
	链表	✓				
插入排序	数组、链表	✓	$O(n^2)$		$O(1)$	(有序区, 无序区)。 把无序区的第一个元素插入到有序区的合适的位置。对数组: 比较得少, 换得多。
堆排序	数组	✗	$O(n \log n)$		$O(1)$	(最大堆, 有序区)。 从堆顶把根卸出来放在有序区之前, 再恢复堆。
归并排序	数组	✓	$O(n \log^2 n)$		$O(1)$	把数据分为两段, 从两段中逐个选最小的元素移入新数据段的末尾。 可从上到下或从下到上进行。
			$O(n \log n)$		$O(n) + O(\log n)$	
	链表				如果不是从下到上 $O(1)$	
快速排序	数组	✗	$O(n \log n)$	$O(n^2)$	$O(\log n)$	(小数, 基准元素, 大数)。 在区间中随机挑选一个元素作基准, 将小于基准的元素放在基准之前, 大于基准的元素放在基准之后, 再分别对小区间与大数区进行排序。
希尔排序	数组	✗	$O(n \log^2 n)$	$O(n^2)$	$O(1)$	每一轮按照事先决定的间隔进行插入排序, 间隔会依次缩小, 最后一次一定要是1。
计数排序	数组、链表	✓	$O(n + m)$		$O(n + m)$	统计小于等于该元素值的元素的个数 <i>i</i> , 于是该元素就放在目标数组的索引 <i>i</i> 位 ($i \geq 0$)。
桶排序	数组、链表	✓	$O(n)$		$O(m)$	将值为 <i>i</i> 的元素放入 <i>i</i> 号桶, 最后依次把桶里的元素倒出来。
基数排序	数组、链表	✓	$O(k \times n)$	$O(n^2)$		一种多关键字的排序算法, 可用桶排序实现。

<https://www.runoob.com/w3cnote/ten-sorting-algorithm.html>

数据库

数据库表设计范式

第一范式：如果数据库表中的所有字段值都是不可分解的原子值，就说明该数据库表满足了第一范式。

第二范式：第二范式需要确保数据库表中的每一列都和主键相关，而不能只与主键的某一部分相关（主要针对联合主键而言）。也就是说在一个数据库表中，一个表中只能保存一种数据，不可以把多种数据保存在同一张数据库表中。

第三范式：第三范式需要确保数据表中的每一列数据都和主键直接相关，而不能间接相关。比如在设计一个订单数据表的时候，可以将客户编号作为一个外键和订单表建立相应的关系。而不可以再在订单表中添加关于客户其它信息（比如姓名、所属公司等）的字段。如下面这两个表所示的设计就是一个满足第三范式的数据库表。

建表除了要满足三个范式以外，最好还要满足

1.数据插入时间

2.数据更新时间

3.数据是否逻辑删除，而不是实际删除。实现方式：新建一个字段is_deleted，删除的时候更新这个字段值，展示的时候根据字段值变化情况决定是否展示。（避免真正删除数据库原有数据）

关系型和非关系型数据库的区别？

关系型数据库的优点

容易理解，因为它采用了关系模型来组织数据。

可以保持数据的一致性。

数据更新的开销比较小。

支持复杂查询（带 where 子句的查询）

非关系型数据库（NOSQL）的优点

无需经过 SQL 层的解析，读写效率高。

基于键值对，读写性能很高，易于扩展

可以支持多种类型数据的存储，如图片，文档等等。

扩展（可分为内存性数据库以及文档型数据库，比如 Redis，MongoDB，HBase 等，适合场景：数据量大高可用的日志系统/地理位置存储系统）。

详细说下一条 MySQL 语句执行的步骤

Server 层按顺序执行 SQL 的步骤为：

客户端请求 -> 连接器（验证用户身份，给予权限）

查询缓存（存在缓存则直接返回，不存在则执行后续操作）

分析器（对 SQL 进行词法分析和语法分析操作）

优化器（主要对执行的 SQL 优化选择最优的执行方案方法）

执行器（执行时会先看用户是否有执行权限，有才去使用这个引擎提供的接口） -> 去引擎层获取数据返回（如果开启查询缓存则会缓存查询结果）

1.mysql默认存储引擎？myisam和innodb 区别是什么

MyISAM: 它不支持事务，也不支持外键，尤其是访问速度快，对事务完整性没有要求或者以SELECT、INSERT为主的应用基本都可以使用这个引擎来创建表。数据文件和索引文件可以放置在不同的目录，平均分配IO，获取更快的速度。

InnoDB：InnoDB就是作为默认的存储引擎。InnoDB还引入了行级锁定和外键约束，一般来说，如果需要事务支持，并且有较高的并发读取频率，使用InnoDB

- 1.更新密集的表。InnoDB存储引擎特别适合处理多重并发的更新请求。
- 2.事务。InnoDB存储引擎是支持事务的标准MySQL存储引擎。

- 3.自动灾难恢复。与其它存储引擎不同，InnoDB表能够自动从灾难中恢复。
- 4.外键约束。MySQL支持外键的存储引擎只有InnoDB。
- 5.支持自动增加列AUTO_INCREMENT属性。

Memory：使用MySQL Memory存储引擎的出发点是速度。为得到最快的响应时间，采用的逻辑存储介质是系统内存。存取速度最快

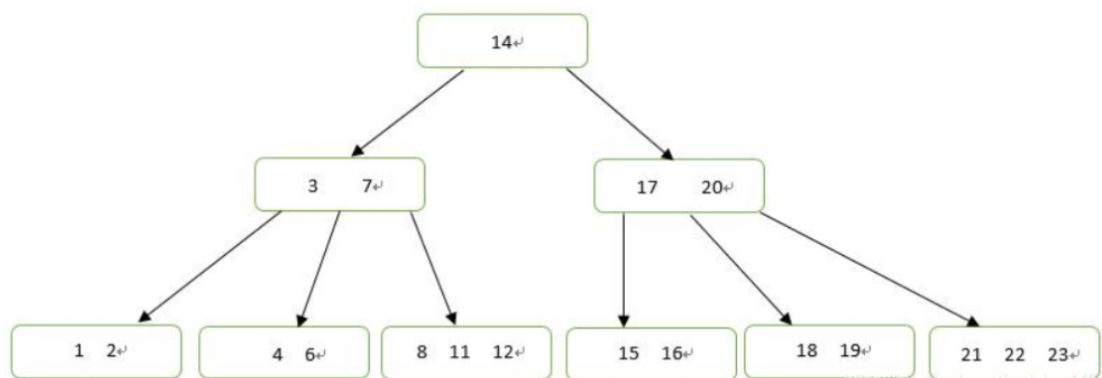
2.sql优化策略

数据库设计符合第三范式，为了查询方便可以有一定的数据冗余。选择数据类型优先级 int > date, time > enum, char > varchar > blob，选择数据类型时，可以考虑替换，如ip地址可以用ip2long()函数转换为unsigned int型来进行存储。对于char(n)类型，在数据完整的情况下尽量较小的n值。

3.聚集索引的底层，mysql索引的底层数据结构，B+树与B树的比较

1.1 B (B-) 树

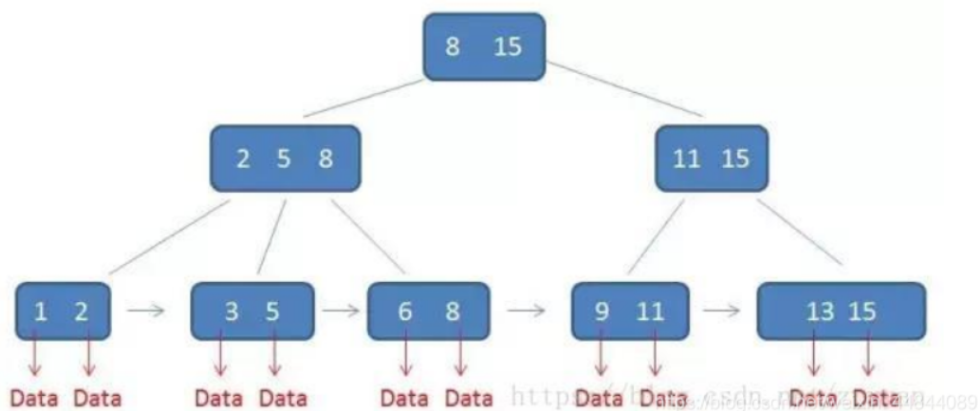
- 首先明确一个概念：B树就是B-树
- 如果想要详细学习B树或者B-树，那么就看这篇博客：[B树详解](#)
- 这里仅仅对B树做一个简单的介绍
 - 1) B树是平衡树，类似于二叉排序树，所以左子节点的数据一定小于父节点，右子节点的数据一定大于父节点
 - 2) 但是B树每一个节点的结构是一个线性表，分别存储关键字（key：也就是数据）和指向子节点的指针。
 - 3) 举个例子：例如如图的【3,7】节点，实际上里面存储的是【point1, 3, point2, 7, point3】
 - 4) 所以每次查询的时候从根节点开始比较关键字，如果找不到就去子节点找
 - 5) 至于B树的插入删除这里就不介绍了，自己看上面的博客连接



1.2 B+树简介

- B+树是B树的变式，两者各有千秋
- 如果想要学习B+树的详细知识可以看这篇博客：[B+树详解](#)
- B+树最大的区别就是所有的数据都是存储在叶子节点上的，而非叶子节点中存储的都是数据索引。并且所有的叶子节点再连成一个链表！
- B+树的三个优点：
 - 1) 层级更低，IO次数更少，因为非叶子节点中不存储数据，只存储索引，这就导致里面可以存储更多的索引！所以IO的次数更少，整个树变矮变胖起来
 - 2) 每次都需要查询到叶子节点，查询性能稳定，而B树的查询性能是不稳定的，有时候可能在根节点查到，有时候可能在叶子节点查到
 - 3) 叶子节点形成有序链表，当我们进行范围查询的时候，直接在这个链表中寻找范围即可，范围查找更加方便

B+树：



1) 聚集索引：

索引B+ Tree的叶子节点存储了整行数据的是主键索引，也被称之为聚簇索引。**聚簇索引就是按照每张表的主键构造一颗B+树**，同时叶子节点中存放的就是整张表的行记录数据，也将聚集索引的叶子节点称为数据页。这个特性决定了索引组织表中数据也是索引的一部分，**每张表只能拥有一个聚簇索引**。

- 如果表设置了主键，则主键就是聚簇索引
- 如果表没有主键，则会默认第一个NOT NULL，且唯一（UNIQUE）的列作为聚簇索引
- 以上都没有，则会默认创建一个隐藏的row_id作为聚簇索引

2) 非聚集索引：

以主键以外的列值作为键值构建的B+树索引，我们称之为非聚集索引；InnoDB的普通索引叶子节点存储的是 **主键（聚簇索引）的值**，而MyISAM的普通索引存储的是记录指针。查询辅助索引，在叶子节点获取索引键值+主键值后，判断是否满足查询所需列信息，不满足则需用刚获取的主键值，再查询聚集索引，获取该记录的全部信息，所以辅助索引又称为二级索引。

对于innodb引擎，只要指定了主键或者auto_increment的字段，就指定了一个聚簇索引；但是非聚簇索引并不会自动创建，而是手动声明的

```
create table user
```

```
(id int(10) auto_increment, name varchar(30), age tinyint(4), primary key (id), index idx_age (age))
```

```
engine=innodb charset=utf8mb4;
```

这里面，聚簇索引是id，非聚簇索引是age

聚集索引（聚簇索引）的查询速度较快：因为主键索引树的叶子节点直接就是我们要查询的整行数据了。而非主键索引的叶子节点是主键的值，查到主键的值以后，还需要再通过主键的值再进行一次查询；(非聚簇索引的查询方式称为回表查询)

但是非聚集索引也不一定是需要查询多次的，例如覆盖索引中，一个查询语句的执行只用从索引中就能够取得，不必从数据表中读取。也可以称之为实现了索引覆盖。索引覆盖就是不需要进行回表查询的操作

当一条查询语句符合覆盖索引条件时，MySQL只需要通过索引就可以返回查询所需要的数据，这样避免了查到索引后再返回表操作，减少I/O提高效率。

当给表加上主键时，相当于给表加了聚集索引，一个表只能有一个聚集索引

聚集索引树中的非叶子节点只存放键值和指向子节点（数据页——叶子节点）的偏移量，叶子节点其实就是存放数据页中完整的行记录。

一个表只能有一个聚簇索引，因为只能有一种排序存储的方式，但可以有多个非聚簇索引，即多个索引目录提供数据检索

聚集索引的存储并不是物理上的连续，因为这样的维护成本会很高，而是逻辑上的连续！其中有两点：

1) 所有的叶子节点都是通过双向链表连接的 2) 每一个叶子节点中的行记录也是通过双向链表连接的

每次写入操作都需要进行平衡二叉树的判断，修改树的结构，这样肯定是浪费了时间的

B+树的索引过程：**非叶子结点会存放数据和指向下一级节点的指针，根据数据量判断目标数据在哪个指针所指向的范围内（二分查找）**

mysql索引底层是B+树，oracle底层是B-树

对查询出现概率较高的在where中出现的字段会添加索引

4.mysql索引类型

- 普通索引：仅加速查询
- 唯一索引：加速查询 + 列值唯一（可以有null）；如果是组合索引，则列值的组合必须唯一
- 主键索引：加速查询 + 列值唯一（不可以有null）+ 表中只有一个
- 组合索引：多列值组成一个索引，专门用于组合搜索，其效率大于索引合并（多个字段综合成一个索引，考虑最左匹配原则）
- 全文索引：对文本的内容进行分词，进行搜索
- 覆盖索引：不是索引，是一种数据查询方式；在索引值中查询到需要查询的字段值，并不需要再通过主键值回表查询，成为覆盖索引

组合索引与最左前缀匹配原则

我们在创建组合索引时应该将**最常用作限制条件的列放在最左边**，依次递减。

mysql在建立联合索引时，最左列的字段一定有序，先保证最左端的字段有序后再保证右边接下来的其他字段有序；（a,b）索引，当a相等的时候再对b进行排序筛选，所以a全局有序，b局部有序

mysql建立多列索引（联合索引）有最左前缀的原则，即最左优先，如：

如果有一个2列的索引(col1,col2),则已经对(col1)、(col1,col2)上建立了索引； 如果有一个3列索引(col1,col2,col3)，则已经对(col1)、(col1,col2)、(col1,col2,col3)上建立了索引；

总结：1、b+树的数据项是复合的数据结构，比如(name,age,sex)的时候，b+树是按照从左到右的顺序来建立搜索树的，比如当(张三,20,F)这样的数据来检索的时候，b+树会优先比较name来确定下一步的所搜方向，如果name相同再依次比较age和sex（先比较靠左的字段，当靠左的字段值相等的时候，再比较右边的字段），最后得到检索的数据；但当(20,F)这样的没有name的数据来的时候，b+树就不知道第一步该查哪个节点，因为建立搜索树的时候name就是第一个比较因子，必须先根据name来搜索才能知道下一步去哪里查询。

2、比如当(张三,F)这样的数据来检索时，b+树可以用name来指定搜索方向，但下一个字段age的缺失，所以只能把名字等于张三的数据都找到，然后再匹配性别是F的数据了，这个是非常重要的性质，即索引的最左匹配特性。（这种情况无法用到联合索引）

(a,b,c)建立索引, a, ab, abc都会走索引, b, bc不走索引, ac会部分走索引

5.InnoDB索引

MySQL 的 InnoDB 存储引擎在设计时是将根节点常驻内存的, 因此力求达到树的深度不超过 3, 也就是说 **I/O 不需要超过 3 次**。

由于内存中的关键字是一个有序表结构, 可以利用二分法查找提高效率。

而 3 次磁盘 I/O 操作是影响整个 B-Tree 查找效率的决定因素。

B-Tree 相对于 AVLTree 缩减了节点个数, 使每次磁盘 I/O 取到内存的数据都发挥了作用, 从而提高了查询效率。

B+Tree 是在 B-Tree 基础上的一种优化, 使其更适合实现外存储索引结构, InnoDB 存储引擎就是用 B+Tree 实现其索引结构。

在 B-Tree 中, 每个节点中有 key, 也有 data, 而每一个页的存储空间是有限的, 如果 data 数据较大时将会导致每个节点 (即一个页) 能存储的 key 的数量很小。

B+Tree 在 B-Tree 的基础上有两点变化:

1. 数据是存在叶子节点中的;
2. 数据节点之间是有指针指向的。

通常在 B+Tree 上有两个头指针, 一个指向根节点, 另一个指向关键字最小的叶子节点, 而且所有叶子节点 (即数据节点) 之间是一种链式环结构。

因此可以对 B+Tree 进行两种查找运算: 一种是对主键的范围查找和分页查找, 另一种是从根节点开始, 进行随机查找。

6.B+树和hash索引的区别

- 哈希索引适合等值查询, 但是无法进行范围查询
- 哈希索引没办法利用索引完成排序
- 哈希索引不支持多列联合索引的最左匹配规则
- 如果有大量重复键值的情况下, 哈希索引的效率会很低, 因为存在哈希碰撞问题

7.为什么不用红黑树或者二叉排序树?

答: 树的查询时间跟树的高度有关, B+树是一棵多路搜索树可以降低树的高度, 提高查找效率。B树的查找可以优化, 可以根据B类树的特点, 构造一个多阶的B类树, 然后在尽量多的在结点上存储相关的信息, 保证层数(树的高度)尽量的少, 以便后面我们可以更快的找到信息, 磁盘的I/O操作也少一些, 而且B类树是平衡树, 每个结点到叶子结点的高度都是相同, 这也保证了每个查询是稳定的。

8.查询优化器是什么

mysql具有内置的查询优化器

逻辑优化

查询优化器在逻辑优化阶段主要解决的问题是: **如何找出SQL语句等价的变换形式, 使得SQL执行更高效**。一条SQL查询语句结构复杂, 包含多种类型的字句, 优化操作依赖于表的一些属性(如索引和约束等)。可用于优化的思路包括:

- 字句局部优化: 每种类型字句都可能存在优化方式, 如等价谓词重写、where和having条件化简中的大部分情况, 都属于这种字句范围内的优化。
- 字句间关联优化: 字句与字句之间关联的语义存在优化的可能, 如外连接消除、连接消除、子查询优化、视图重写等都属于字句间的关联优化, 因为他们的优化都需要借助其他字句、表定义或列属

性等信息进行。

- 局部与整体的优化: 需要协同考虑局部表达式和整体的关系, 如OR重写并集规则需要考虑UNION操作(UNION是变换后的整体的形式)的花费和OR操作(OR是局部表达式)的花费。
- 形式变化优化: 多个子句存在嵌套, 可以通过形式的变化完成优化, 如嵌套连接消除。
- 语义优化: 根据完整性约束、SQL表达的含义等信息对语句进行语义优化。

物理优化

主要解决的问题

- 从可选的单表扫描方式中, 挑选什么样的单表扫描方式是最优的?
- 对于两个表连接时, 如何选择是最优的?
- 对多个表连接, 连接顺序有多种组合, 是否要对每种组合都探索? 如果不全部探索, 怎么找到最优的一种组合?

9.主键索引和唯一索引的区别

主键必唯一, 但是唯一索引不一定是主键;

一张表上只能有一个主键, 但是可以有一个或多个唯一索引。

10.为什么B+树比B树更适合实现数据库索引?

B+ 树叶子结点之间用链表有序连接, 所以扫描全部数据只需扫描一遍叶子结点, 利于扫描和范围查询; B 树由于非叶子结点也存数据, 所以只能通过中序遍历按序来扫。也就是说, 对于范围查询和有序遍历而言, B+ 树的效率更高。

B+ 树更相比 B 树减少了 I/O 读写的次数。由于索引文件很大因此索引文件存储在磁盘上, B+ 树的非叶子结点只存关键字不存数据, 因而单个页可以存储更多的关键字, 即一次性读入内存的需要查找的关键字也就越多, 磁盘的随机 I/O 读取次数相对就减少了。

B+树的查询效率更加稳定, 任何关键字的查找必须走一条从根结点到叶子结点的路。所有关键字查询的路径长度相同, 导致每一个数据的查询效率相当。

11.怎么建索引, 什么时候需要建, 建索引的坏处和好处

选择哪些列建索引

在经常需要搜索的列上, 可以加快搜索的速度;

在作为主键的列上, 强制该列的唯一性和组织表中数据的排列结构;

在经常用在连接的列上, 这些列主要是一些外键, 可以加快连接的速度;

在经常需要根据范围进行搜索的列上创建索引, 因为索引已经排序, 其指定的范围是连续的;

在经常需要排序的列上创建索引, 因为索引已经排序, 这样查询可以利用索引的排序, 加快排序查询时间;

在经常使用在WHERE子句中的列上面创建索引, 加快条件的判断速度。

哪些列不能建索引

1) 对于那些在查询中很少使用或者参考的列不应该创建索引。这是因为, 既然这些列很少使用到, 因此有索引或者无索引, 并不能提高查询速度。相反, 由于增加了索引, 反而降低了系统的维护速度和增大了空间需求。

2) 对于那些只有很少数据值的列也不应该增加索引。这是因为, 由于这些列的取值很少, 例如人事表的性别列, 在查询的结果中, 结果集的数据行占了表中数据行的很大比例, 即需要在表中搜索的数据行的比例很大。增加索引, 并不能明显加快检索速度。

3) 对于那些定义为text, image和bit数据类型的列不应该增加索引。这是因为, 这些列的数据量要么相当大, 要么取值很少。

4) 当修改性能远远大于检索性能时, 不应该创建索引。这是因为, 修改性能和检索性能是互相矛盾的。当增加索引时, 会提高检索性能, 但是会降低修改性能。当减少索引时, 会提高修改性能, 降低检索性能。因此, 当修改性能远远大于检索性能时, 不应该创建索引

5) 对经常修改的字段也不能加索引, 改动成本很高

索引失效的几种情况

- 1.有or必全有索引, sql中不能出现or;
- 2.复合索引未用左列字段;

复合索引绑定的多个列是有顺序的,某一个列没有出现在查询条件中,存储引擎不能使用索引中该列及其后的所有列。(a, b, c)索引, where ac的情况

3.like以%开头;

a.like %keyword **索引失效**, 使用全表扫描。但可以通过翻转函数+like前模糊查询+建立翻转函数索引=走翻转函数索引, 不走全表扫描。如where reverse(code) like reverse('%Code2')

b.like keyword% 索引有效。

c.like %keyword% 索引失效, 也无法使用反向索引。

4.需要类型转换;

查询条件中出现某个列是范围查询的, 存储引擎不能使用复合索引中该列其后的所有列。

5.where中索引列有运算;

6.where中索引列使用了函数;

7.如果mysql觉得全表扫描更快时(数据少);

索引的优缺点

优点

- 提高数据检索的效率, 降低数据库IO成本。
- 通过索引对数据进行排序, 降低数据的排序成本, 降低CPU的消耗。

缺点

- 建立索引需要**占用物理空间**
- 会降低表的增删改的效率, 因为每次对表记录进行增删改, 需要进行**动态维护索引**, 导致增删改时间变长

索引大集合

<https://blog.csdn.net/yudiandemingzi/article/details/122329773>

12.mysql 中的外键作用

- 1.外键的作用: 使两张表形成关联, 外键只能引用外表中的指定列的值!
- 2.建立外键的前提: 本表的列必须与外键类型相同(外键必须是外表的主键)。
- 3.事件触发限制: on delete (删除) 和on update (更新), cascade可设参数(跟随外键改动), restrict(限制外表中的外键改动, 也就是说禁止主表变更), set Null(设空值, 子表相应字段设置为空), set Default (设默认值), no action[默认]

SQL语句分类

DQL:数据查询语言, 用于对数据进行查询, 如select

DML:数据操作语言, 对数据进行增加、修改、删除, 如insert、update、delete

TPL:事务处理语言, 对事务进行处理, 包括begin transaction、commit、rollback

DDL:数据定义语言, 进行数据库、表的管理等, 如create、drop

WAL技术

WAL, 中文全称是 Write-Ahead Logging, 它的关键点就是日志先写内存, 再写磁盘。MySQL 执行更新操作后, 在真正把数据写入到磁盘前, 先记录日志。

好处是不用每一次操作都实时把数据写盘, 就算 crash 后也可以通过redo log 恢复, 所以能够实现快速响应 SQL 语句。

13..mysql的表锁有哪些?

不同的存储引擎支持不同的锁机制。MySQL的表锁有两种模式: 表共享读锁 (Table Read Lock) 和表独占写锁 (Table Write Lock)

14.事务特性ACID

- 原子性 (Atomicity) : 事务是一个原子操作单元, 其对数据的修改, 要么全都执行, 要么全都不执行。
- 一致性 (Consistent) : 在事务开始和完成时, 数据都必须保持一致状态。这意味着所有相关的数据规则都必须应用于事务的修改, 以保持完整性; 事务结束时, 所有的内部数据结构 (如B树索引或双向链表) 也都必须是正确的。
- 隔离性 (Isolation) : 数据库系统提供一定的隔离机制, 保证事务在不受外部并发操作影响的“独立”环境执行。这意味着事务处理过程中的中间状态对外部是不可见的, 反之亦然。
- 持久性 (Durable) : 事务完成之后, 它对于数据的修改是永久性的, 即使出现系统故障也能够保持。

15.并发事务带来的问题

- **更新丢失** (Lost Update) : 当两个或多个事务选择同一行, 然后基于最初选定的值更新该行时, 由于每个事务都不知道其他事务的存在, 就会发生丢失更新问题——最后的更新覆盖其他事务所做的更新。例如, 两个编辑人员制作了同一文档的电子副本。每个编辑人员独立地更改其副本, 然后保存更改后的副本, 这样就覆盖了原始文档。最后保存其更改保存其更改副本的编辑人员覆盖另一个编辑人员所做的修改。如果在一个编辑人员完成并提交事务之前, 另一个编辑人员不能访问同一文件, 则可避免此问题
- **脏读** (Dirty Reads) : A事务读取B事务尚未提交的更改数据, 并在这个数据的基础上进行操作, 这时候如果事务B回滚, 那么A事务读到的数据是不被承认的。。这种现象被形象地叫做“脏读”。
- **不可重复读** (Non-Repeatable Reads) : 事务A首先读取了一条数据, 然后执行逻辑的时候, 事务B将这条数据改变了, 然后事务A再次读取的时候, 发现数据不匹配了, 就是所谓的不可重复读了。
- 也就是说, 当前事务先进行了一次数据读取, 然后再次读取到的数据是别的事务修改成功的数据, 导致两次读取到的数据不匹配, 也就照应了不可重复读的语义。
- **幻读**
(Phantom Reads) : 事务A首先根据条件索引得到N条数据, 然后事务B改变了这N条数据之外的M条或者增添了M条符合事务A搜索条件的数据, 导致事务A再次搜索发现有N+M条数据了, 就产生了幻读。

也就是说, 当前事务读第一次取到的数据比后来读取到数据条目少。

16.事务隔离级别

read-uncommitted：读取未提交。最低的级别，允许读取尚未提交的数据变更，可能导致脏读幻读不可重复读

read-committed：读取已提交。允许读取并发事务已经提交的数据。可以阻止脏读，但是幻读和不可重复读还存在

repeatable-read：可重复读。对同一字段读取结果一致，除非被当前事务自己修改。可以阻止脏读和不可重复读，幻读会存在

serializable：最高级别，ACID完全满足，三种错误都可以阻止

mysql默认为可重复读，redis默认满足ACID，不一定满足，oracle默认为读取已提交

redis事务没有隔离级别，也不满足原子性和隔离性

读取已提交跟可重复读的区别

可重复读：两个事务进行数据操作他们是互不干扰的，事务先A进行数据查询，事务B进行一次事务修改并进行数据提交，事务A再进行一次查询，数据是不改变的。

读已提交：两个事务进行数据操作，事务先A进行数据查询，事务B进行一次事务修改并进行数据提交，事务A再进行一次查询，数据是B修改后的数据。

可重复读是沿袭了读已提交并让事务隔离级别更高：

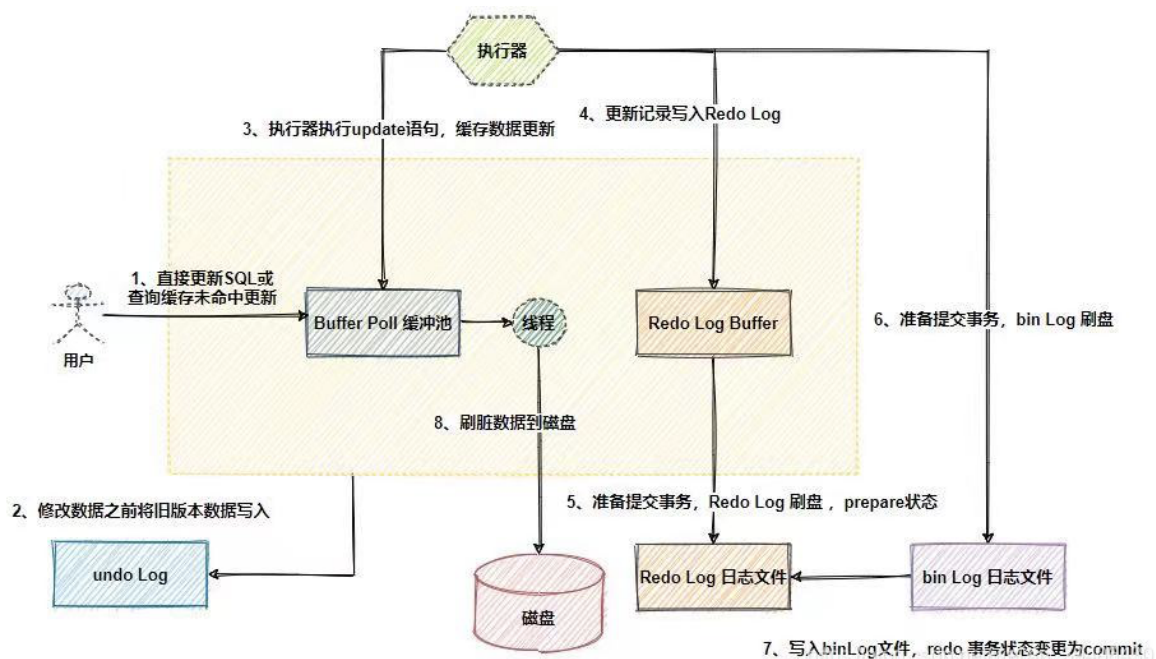
读已提交是A事务本身不用提交，就能读取到B事务修改并提交事务的数据，当然，A提交事务自然也是能读取B修改后的数据的；

可重复读对此基础上，要求A也必须提交事务，才能读取到B修改并提交事务的数据，若A在B修改并提交事务后没有提交事务就去读取，每次的结果都是一样的，但实际B事务已对数据做出成功修改（硬盘中的文件数据以改变），A事务读到的都是备份数据，总感觉出现了幻觉，所以叫幻读！

17.mysql的日志种类

https://blog.csdn.net/qq_35239471/article/details/120783558

update更新操作流程



1.redo log

确保事务的持久性。redo日志记录**事务执行后的状态**，用来恢复未写入data file的已成功事务更新的数据。

它记录的是数据库中每个页的修改，而不是某一行或某几行修改成怎么样，可以用来恢复提交后的数据页，并且只能恢复至最后一次提交的位置。

在客户端尝试修改数据时，InnoDB会把记录下写在redo log中，再修改缓存池中数据，当事务提交时，调用fsync把redo log刷入磁盘。至于缓存池的数据何时刷入磁盘由后台线程异步处理。

事务开始后就产生redo log；redo log并不是随着事务的提交才写入的，而是在事务执行过程中就开始写入redo log文件中

redo log更新：redo log采用**固定大小，循环写入**的方式。当redo log写满后从头开始写，形成一个环状。因为redo log记录的是数据页上的修改，如果缓存池的数据页已经刷盘了，那redo log中记录的信息就失去了价值，那么新日志就可将这些失效记录进行擦除、覆盖。

2.undo log

保证数据的原子性，**保存了事务发生之前的数据的一个版本，可以用于回滚**，同时可以提供多版本并发控制下的读（MVCC），也即非锁定读

记录的是数据修改前的状态，即在数据修改的流程中，记录一条与当前操作相反的回滚日志。假如一个事务中一条记录被多次修改，但是undo log只会记录原始版本的一条数据，每当对数据进行修改时，都会写入redo log。

逻辑格式的日志，在执行undo的时候，仅仅是将数据从逻辑上恢复至事务之前的状态，而不是从物理页面上操作实现的

事务开始之前，将当前是版本生成undo log，undo 也会产生 redo 来保证undo log的可靠性

undo log负责完成回滚，redo log负责完成前滚

- 回滚：未提交的事务，即事务未执行commit。但是该事务内修改的脏页中，可能有一部分脏块已经刷盘。如果此时数据库宕机，则需要用回滚来将已经刷盘的脏块从磁盘上撤销
- 前滚：未完全提交的事务，即事务已经执行commit,但是缓存吃中的脏页还未未来的急刷盘，此时数据库宕机就需要使用redo log进行前滚刷盘恢复数据

数据库重启时，先做前滚，后做回滚。可再看一下本文第一张图，可以看到redo log、undo log、bin log都是在刷脏页之前就已经刷到了磁盘上，这俄中互相协作才能保证用户提交的数据不会丢失。

3.bin log

记录了所有DDL（数据定义语言，主要用于创建数据库对象；CREATE、ALTER、DROP、TRUNCATE、COMMENT、GRANT、REVOKE）和DML操作（数据操纵语言，主要用于操纵数据库对象。SELECT、INSERT、UPDATE、DELETE、CALL、EXPLAIN PLAN、LOCK TABLE）（不包括select和show）。默认情况下是关闭的。bin log不会向redo log那样会擦出无用日志，而是永久存储(也可以设置过期时间，默认是永久保存)，默认文件1G,超过此阈值会新建一个文件进行记录。用于复制，在主从复制中，从库利用主库上的binlog进行重播，实现主从同步。用于数据库的基于时间点的还原。

逻辑格式的日志，可以简单认为就是执行过的事务中的sql语句。

但又不完全是sql语句这么简单，而是包括了执行的sql语句（增删改）反向的信息，也就意味着delete对应着delete本身和其反向的insert；update对应着update执行前后的版本的信息；insert对应着delete和insert本身的信息。

事务提交的时候，一次性将事务中的sql语句（一个事物可能对应多个sql语句）按照一定的格式记录到binlog中

4.relay log中继日志

relay log和bin log具有相同的格式，上面主从复制流程页提到了relay log,其主要作用起到一个中转作用。

5.slow query log慢查询日志

用于记录在mysql执行过程超过指定时间的查询语句，处于性能考虑，一般只有在排查慢查询、调试参数时候才会开启，默认情况下是关闭的，默认时间是10s

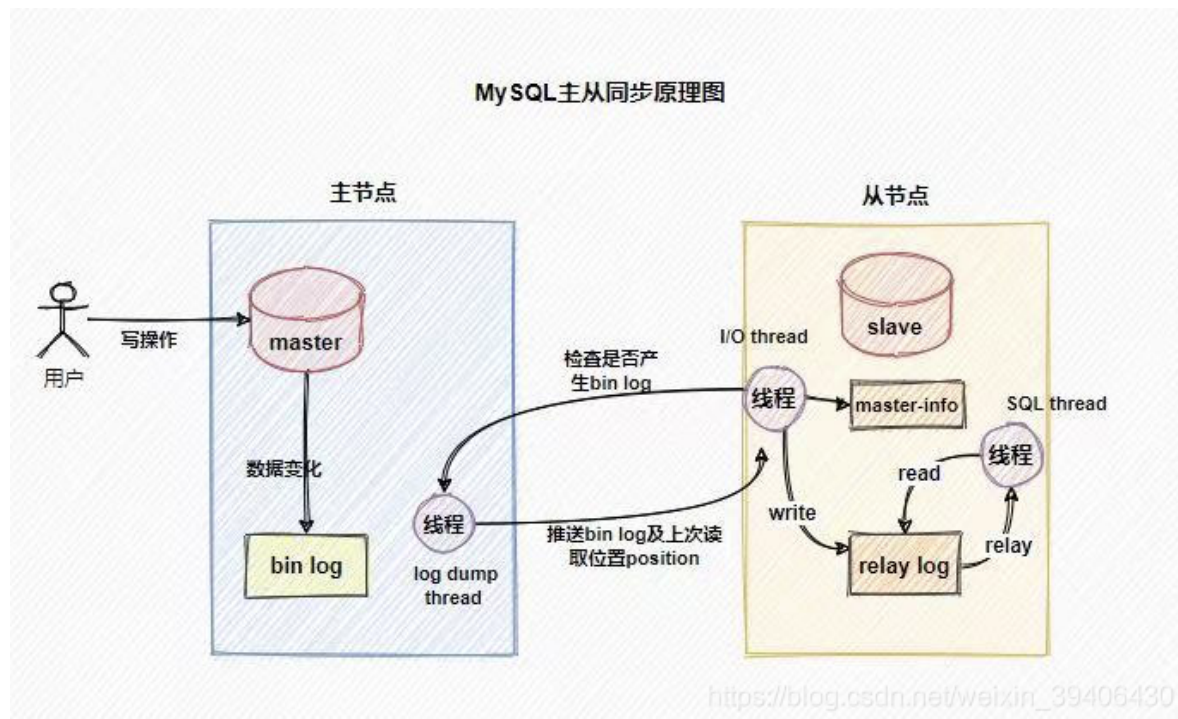
6.general query log一般查询日志

一般查询日志用来记录用户的所有操作，包括客户端何时连接的服务器、客户端发送的所有sql以及其他事情。处于性能考虑，该日志功能默认也是关闭的。

7.error log

错误日志记录mysql服务每次启动和停止的时间以及诊断和出错信息，默认开启，可通过 show variables like 'log_error' 查找存放路径。

18.binlog主从复制



主从复制是MySQL自带的一种横向扩展方案，通过“数据变更在主库执行，查询请求在从库执行”这样的形式可以在一定程度上实现读写分离。

有了主从复制后，还可以实现服务滚动升级（先升级从库进行测试，然后手动切换主从后升级其他从库）。主从结构通常可以分为一主一从、一主多从、[级联复制](#)等

主节点通过Binlog记录了数据库所有DDL与DML操作（SELECT、SHOW等语句不对数据产生变化，不会有记录，另外临时表也不会有记录），从库将binlog日志复制到自己的中继日志Relaylog中，然后根据Relaylog做相同的操作，实现了数据的同步。

binlog格式介绍

statement（标准模式，简称SBR）：保存的是每一条修改数据的SQL语句，然后在从服务器上会执行相同的语句来同步数据。优点是日志记录量比较少，缺点是数据一致性不是最高级别

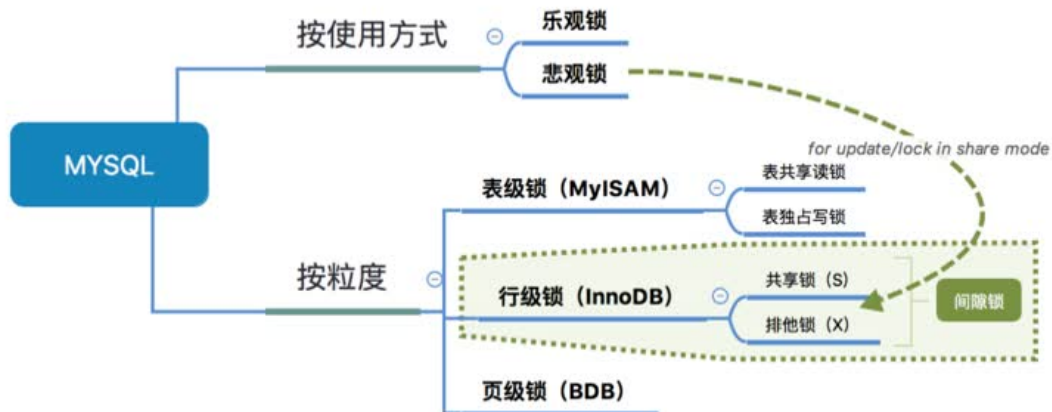
row（行模式，简称RBR）：最安全的复制模式，将数据的具体改变记录在日志中，日志量大，不会因为某些会产生随机数的SQL而让主从数据不一致（比如包含了时间函数的语句）。row格式的二进制日志无法直接阅读，需要通过mysqlbinlog转码

mixed（混合模式，简称MBR）：statement和row的结合模式，当基于语句无法精确复制时，就会采用行模式进行复制。实际上比较鸡肋，不考虑使用

复制过程

- 1.主库执行DDL和DML操作，按照修改顺序以此写入bin log.
- 2.从库的IO线程连接上主库并请求读取指定位置position的日志内容
- 3.主库收到请求后，将制定位置position之后的内容日志、主库bin log文件名称以及在日志中的位置推送给从库。
- 4.从库IO线程收到数据后，将日志内容以此写入relay log文件最末端，并将bin log文件名和位置position记录到master-info文件中，以遍下次使用。
- 5.从库的sql线程检测到relay log中内容更新后，读取日志并解析成可执行的sql语句进行主从同步。

19.mysql数据库锁



对于**UPDATE、DELETE、INSERT**语句，InnoDB会自动给涉及数据集加排他锁（X）

MyISAM在执行查询语句SELECT前，会自动给涉及的所有表加读锁，在执行增、删、改操作前，会自动给涉及的表加写锁，这个过程并不需要用户干预

从锁的粒度，我们可以分成两大类：

表锁：开销小，加锁快；不会出现死锁；锁定力度大，发生锁冲突概率高，并发度最低

行锁：开销大，加锁慢；会出现死锁；锁定粒度小，发生锁冲突的概率低，并发度高

不同的存储引擎支持的锁粒度是不一样的：InnoDB行锁和表锁都支持、MyISAM只支持表锁！InnoDB只有通过索引条件检索数据才使用行级锁==，否则，InnoDB使用表锁也就是说，InnoDB的行锁是基于索引的！

表锁下又分为两种模式：表读锁（Table Read Lock）&& 表写锁（Table Write Lock）

在表读锁和表写锁的环境下：读读不阻塞，读写阻塞，写写阻塞

InnoDB和MyISAM有两个本质的区别：**InnoDB支持行锁、InnoDB支持事务**

InnoDB实现了以下两种类型的行锁：

- 共享锁（S锁、读锁）：允许一个事务去读一行，阻止其他事务获得相同数据集的排他锁。即多个客户可以同时读取同一个资源，但不允许其他客户修改。SELECT * FROM table_name WHERE ... LOCK IN SHARE MODE
- 排他锁（X锁、写锁）：允许获得排他锁的事务更新数据，阻止其他事务取得相同数据集的读锁和写锁。写锁是排他的，写锁会阻塞其他的写锁和读锁。
SELECT * FROM table_name WHERE ... FOR UPDATE

另外，为了允许行锁和表锁共存，实现多粒度锁机制，InnoDB还有两种内部使用的意向锁（Intention Locks），这两种意向锁都是表锁：

- 意向共享锁（IS）：事务打算给数据行加行共享锁，事务在给一个数据行加共享锁前必须先取得该表的IS锁。

- 意向排他锁 (IX)：事务打算给数据行加行排他锁，事务在给一个数据行加排他锁前必须先取得该表的IX锁。
- 意向锁也是数据库隐式帮我们做了，不需要程序员关心！

MVCC(Multi-Version ConcurrencyControl)多版本并发控制

可以简单地认为：MVCC就是行级锁的一个变种(升级版)。在表锁中我们读写是阻塞的，基于提升并发性能的考虑，MVCC一般读写是不阻塞的(很多情况下避免了加锁的操作)。

可以简单的理解为：对数据库的任何修改的提交都不会直接覆盖之前的数据，而是产生一个新的版本与老版本共存，使得读取时可以完全不加锁。

多版本并发控制。MVCC是一种并发控制的方法，一般在数据库管理系统中，实现对数据库的并发访问，在编程语言中实现事务内存。

MySQL InnoDB下的当前读和快照读：

1) 当前读

像select lock in share mode(共享锁)、select xxx for update、update、insert、delete(排他锁)这些操作都是一种当前读，它读取的是记录的最新版本，**读取时还要保证其他并发事务不能修改当前记录，会对读取的记录进行加锁。**当前读实际上是一种加锁的操作，**是悲观锁的实现**

2) 快照读

像不加锁的select操作就是快照读，**快照读的前提是隔离级别不是串行级别**，串行级别下的快照读会退化当前读；

之所以出现快照读的情况，是基于提高并发性能的考虑，快照读的实现是基于多版本并发控制，即**MVCC,可以认为MVCC是行锁的一个变种**，但它在很多情况下，避免了加锁操作，降低了开销；**既然是基于多版本，即快照读可能读到的并不一定是数据的最新版本，而有可能是之前的历史版本**

普通的SELECT查询都是读取MVCC版本链中的一个版本，相当于读取一个快照，因此称为快照读。

MVCC模型在MySQL中的具体实现则是由 3个隐式字段，undo日志，Read View 等去完成的，具体可以看下面的MVCC实现原理

MVCC解决读写和写写问题。也就是为事务分配单向增长的时间戳，为每个修改保存一个版本，版本与事务时间戳关联，读操作只读该事务开始前的数据库的快照。

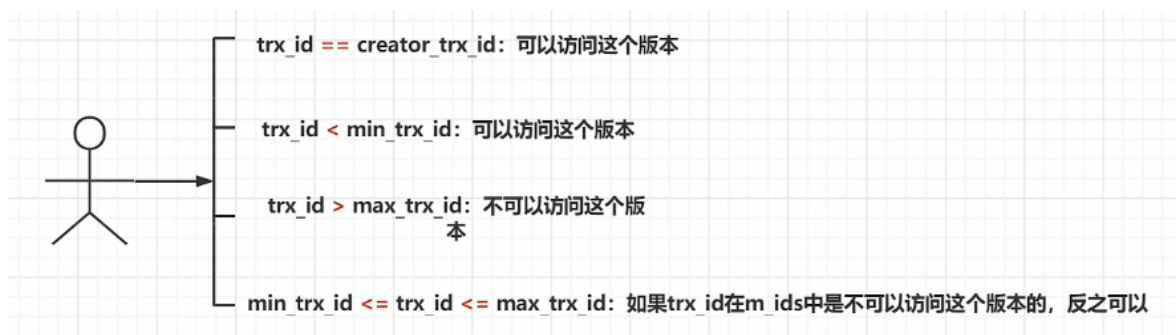
- 在并发读写数据库时，可以做到在读操作时不用阻塞写操作，写操作也不用阻塞读操作，提高了数据库并发读写的性能
- 同时还可以解决脏读，幻读，不可重复读等事务隔离问题，但不能解决更新丢失问题

隐式字段

DB_TRX_ID,DB_ROLL_PTR,DB_ROW_ID

readview：当我们用select读取数据时，这一时刻的数据会有多个版本（例如上图有四个版本），但我们并不知道读取哪个版本，这时就靠readview来对我们进行读取版本的限制，**通过readview我们才知道自己能够读取哪个版本。**

1.readview如何判断版本链中的哪个版本可用



从上到下分别为 (1) (2) (3) (4), 依次进行解释

trx_id表示要读取的事务id

(1) 如果要读取的事务id等于进行读操作的事务id, 说明是我读取我自己创建的记录, 那么为什么不可以呢。

(2) 如果要读取的事务id小于最小的活跃事务id, 说明要读取的事务已经提交, 那么可以读取。

(3) max_trx_id表示生成readview时, 分配给下一个事务的id, 如果要读取的事务id大于max_trx_id, 说明该id已经不在该readview版本链中了, 故无法访问。

(4) m_ids中存储的是活跃事务的id, 如果要读取的事务id不在活跃列表, 那么就可以读取, 反之不行。

2.mvcc如何实现RC和RR的隔离级别

(1) RC的隔离级别下, 每个快照读都会生成并获取最新的readview。

(2) RR的隔离级别下, 只有在同一个事务的第一个快照读才会创建readview, 之后的每次快照读都使用的同一个readview, 所以每次的查询结果都是一样的。

3.幻读问题

快照读: 通过mvcc, RR的隔离级别解决了幻读问题, 因为每次使用的都是同一个readview。

当前读: 通过next-key锁 (行锁+gap锁), RR隔离级别并不能解决幻读问题。

乐观锁和悲观锁

解决更新覆盖、读写冲突问题

悲观锁

`select * from xxxx for update`, 在 `select` 语句后边加了 `for update` 相当于加了排它锁(写锁), 加了写锁以后, 其他事务就不能对它修改了! 需要等待当前事务修改完之后才可以修改。

乐观锁

乐观锁不是数据库层面上的锁, 需要用户手动去加的锁。一般我们在数据库表中添加一个版本字段version来实现, 例如操作1和操作2在更新User表的时候, 执行语句如下:

```
1 update A set Name=lisi,version=version+1 where ID=#{id} and version=#{version};
```

此时即可避免更新丢失。

间隙锁GAP

当我们用范围条件检索数据而不是相等条件检索数据, 并请求共享或排他锁时, InnoDB会给符合范围条件的已有数据记录的索引项加锁; 对于键值在条件范围内但并不存在的记录, 叫做“间隙 (GAP)”。InnoDB也会对这个“间隙”加锁, 这种锁机制就是所谓的间隙锁。例子: 假如emp表中只有101条记录, 其empid的值分别是1,2,...,100,101

```
1 | select * from emp where empid > 100 for update;
```

上面是一个范围查询，InnoDB不仅会对符合条件的empid值为101的记录加锁，也会对empid大于101（这些记录并不存在）的“间隙”加锁

什么是共享锁和排他锁

SELECT 的读取锁定主要分为两种方式：共享锁和排他锁。

select * from table where id<6 lock in share mode;--共享锁

select * from table where id<6 for update;--排他锁

这两种方式主要的不同在于LOCK IN SHARE MODE多个事务同时更新同一个表单时很容易造成死锁。

申请排他锁的前提是，没有线程对该结果集的任何行数据使用排它锁或者共享锁，否则申请会受到阻塞。在进行事务操作时，MySQL会对查询结果集的每行数据添加排它锁，其他线程对这些数据的更改或删除操作会被阻塞（只能读操作），直到该语句的事务被commit语句或rollback语句结束为止。

共享锁是读操作的时候创建的锁，一个事务对数据加上共享锁之后，其他事务只能对数据再加共享锁，不能进行写操作直到释放所有共享锁。

排他锁是写操作时创建的锁，事务对数据加上排他锁之后其他任何事务都不能对数据加任何的锁(即其他事务不能再访问该数据)

SELECT... FOR UPDATE 使用注意事项：

for update 仅适用于InnoDB，且必须在事务范围内才能生效。

根据主键进行查询，查询条件为 like或者不等于，主键字段产生表锁。

根据非索引字段进行查询，name字段产生表锁。

InnoDB使用间隙锁的目的有2个：

- 为了防止幻读(上面也说了，Repeatable read隔离级别下再通过GAP锁即可避免了幻读)
- 满足恢复和复制的需要：MySQL的恢复机制要求在一个事务未提交前，其他并发事务不能插入满足其锁定条件的任何记录，也就是不允许出现幻读

20.mysql基础建表语句

1.建立用户表

```
1 | DROP `TABLE` IF EXISTS `user`;
2 | CREATE `TABLE` `user` (
3 |   `pk_id` `int` (10) `NOT` `NULL` `AUTO_INCREMENT`,
4 |   `username` `varchar` (30) `NOT` `NULL`,
5 |   `password` `char` (32) `NOT` `NULL`,
6 |   `age` `int` (3) `DEFAULT` `NULL`,
7 |   `info` `varchar` (255) `CHARACTER` `SET` `utf8mb4` `DEFAULT` `NULL`,
8 |   `createtime` `timestamp` `NULL` `DEFAULT` `NULL` `ON` `UPDATE`
9 |   `CURRENT_TIMESTAMP`,
10 |   `modifytime` `timestamp` `NULL` `DEFAULT` `NULL` `ON` `UPDATE`
11 |   `CURRENT_TIMESTAMP`,
12 |   `sex` `char` (1) `DEFAULT` `NULL`,
13 |   `PRIMARY` `KEY` (`pk_id`)
14 | ) ENGINE=InnoDB AUTO_INCREMENT=34 DEFA
```

2.建立公司网站主页

```
1 | SET `FOREIGN_KEY_CHECKS`=0;
```

```

2  -----
3  -- Table structure for home
4  -----
5  DROP `TABLE` `IF EXISTS` `home`;
6  CREATE `TABLE` ``home` (
7    ``home_id` `int` (10) `NOT` `NULL` `AUTO_INCREMENT`,
8    ``profile` longtext comment ``'企业简介'``,
9    ``scope` longtext comment ``'经营范围'``,
10   ``product` longtext comment ``'产品介绍'``,
11   ``cooperate` longtext comment ``'校企合作'``,
12   ``extension` longtext comment ``'其他'``,
13
14   ``PRIMARY` `KEY` `( ``home_id`)
15 ) ENGINE=InnoDB AUTO_INCREMENT=34 ``DEFAULT` ``CHARSET=utf8;

```

21.笛卡尔积

select a, b from A, B -> A*B 每条A中的记录和B中的记录都要匹配，也称为cross join（这是mysql的一种错误，在连表查询时，并不能使用笛卡尔积），要使用两个表的连接条件；

着重号`` 可以用，但是并不一定需要用

22.count(*), count(1), count(非空字段)在统计表中记录行数的效率

1.如果使用myisam存储，则效率一样；O(1)

因为会保存一个表中记录数的信息，直接获取

2.如果使用innodb，则效率不一样

count(*) = count(1) > count(非空字段)

count(字段) < count(主键 id) < count(1) ≈ count()，所以建议尽量使用 count()。

原因：count(*)和count(1)会自动切换去找key length较小的索引字段，但指定的非空字段不会有这个优化，所以查询的时间较慢

区别

InnoDB 引擎执行 count(*) 的时候，需要把数据一行一行地从引擎里面读出来，然后累积计数。为什么InnoDB不直接存储表的总数？这是因为InnoDB使用MVCC实现事务保证隔离级别的方式，使同一时刻不同事务执行，返回的行数也是不一样的。所以需要去判断每行数据是否可见再去计数
对于 count(1) 来说，InnoDB 引擎遍历整张表，但不取值（忽略所有行的意思）。server 层对于返回的每一行，放一个数字“1”进去，判断是不可能为空的，按行累加。

对于count(主键 id) 来说，InnoDB 引擎会遍历整张表，把每一行的 id 值都取出来，返回给 server 层。server 层拿到 id 后，判断是不可能为空的，就按行累加。
count(1) 执行得要比 count(主键 id) 快。因为从引擎返回 id 会涉及到解析数据行，以及拷贝字段值的操作。

对于 count(字段) 来说，如果这个“字段”是定义为 not null 的话，一行行地从记录里面读出这个字段，判断不能为 null，按行累加；（count(主键id)）

如果这个“字段”定义允许为 null，那么执行的时候，判断到有可能是 null，还要把值取出来再判断一下，不是 null 才累加。

count (*) 可以筛选出null行，count (其他) 会把null去除

delete pk truncate

- 1.delete 可以加where 条件, truncate不能加
- 2.truncate删除, 效率高一丢丢
- 3.假如要删除的表中有自增长列, 如果用delete删除后, 再插入数据, 自增长列的值从断点开始, 而truncate删除后, 再插入数据, 自增长列的值从1开始。表结构被删除。
- 4.truncate删除没有返回值, delete删除有返回值
- 5.truncate删除不能回滚, delete删除可以回滚。

23.having和where的区别

- 1.having与group by一起使用, 用于过滤数据

group by xxx->分组的字段

having xxx ->聚合的字段

- 2.必须使用having代替where, 而不能使用where+group by

- 3.having要放在group by 后面

- 4.如果sql中没有聚合函数, 则having = where 但更建议使用where, 因为在从连接表中筛选时, where是先筛选后连接, having是先连接后筛选

mysql数据库数据类型

- 1.数字类型

数字类型 UNSIGNED 无符号 (没有负值, 最大值会加倍) ZEROFILL 如果位数不够前面补0

类型	大小 (字节)	有符号范围	无符号范围
tinyint	1	-128到127	0到255
smallint	2	-32768到32767	0到65535
mediumint	3	-8388608到8388607	0到16777215
int	4	-2147483648到2147483647	0到4294967295
bigint	8	-9223372036854775808到 9223372036854775807	0到 18446744073709551615
float	4	-3.402823466E+38 到-1.175494351E-38, 0 和 1.175494351E-38到 3.402823466E+38	0 和1.175494351E-38到 3.402823466E+38
double	8	-1.7976931348623157E+308 到-2.2250738585072014E-308、0 和2.2250738585072014E-308到 1.7976931348623157E+308	0和2.2250738585072014E- 308到 1.7976931348623157E+308
decimal[M,D]		根据M和D的值	根据M和D的值

- 1、根据业务数据大小决定合适的字段类型
- 2、如果字段非负数建议加上UNSIGNED，可以使其范围增加一倍
- 3、金钱等需求精度要求高的数据建议使用decimal，或者将其转化成更小的单位用其他整形字段保存，比如将金钱转成分，用int或者bigint保存

2.字符串类型

类型	范围	描述
char	1 ~ 255个字符	一个定长字符串
varchar	1 ~ 255个字符	一个变长字符串
tinyText	255(2^8-1)个字符	小型文本
text	65535(2^16-1)个字符	普通文本
mediumText	16777215(2^24-1)个字符	中型文本
longText	4294967295(2^32-1)个字符	大型文本
enum		枚举类型，格式属性名 ENUM ('值1','值2','值3'...'值n')，ENUM 有NOT NULL 属性，其默认值为取值列表的第一个元素
set		属性名 SET('值1','值2','值3'...'值n')类似多选的 ENUM
json		可以存储 json 格式数据，通过函数可以进行字符串操作

- 1、char长度固定，即每条数据占用等长字节空间；适合用在身份证号码、手机号码等定。
- 2、varchar可变长度，可以设置最大长度；适合用在长度可变的属性。
- 3、text不设置长度，可以存储大文本数据，如文字图片文档等，当然只是相对来说。
- 4、json是MySQL新增的数据结构，可以像非关系数据库一样操作json格式数据，如果功能非必要进入非关系型数据库，可以使用json代替

3.日期时间类型

类型	范围	描述
date	1000-01-01到9999-12-31	MySQL 以 YYYY-MM-DD 格式来显示DATE值，但是允许你使用字符串或数字把值赋给DATE列
datetime	1000-01-01 00:00:00到9999-12-31 23:59:59	MySQL以'YYYY-MM-DD HH:MM:SS'格式来显示DATETIME值，但是允许你使用字符串或数字把值赋给DATETIME的列
timestamp	1970-01-01 00:00:01到2038-01-19 03:14:07	MySQL以YYYYMMDDHHMMSS、YYMMDDHHMMSS、YYYYMMDD或YYMMDD 格式来显示TIMESTAMP值，取决于是否M是14(或省略)、12、8或6，但是允许你使用字符串或数字把值赋给TIMESTAMP列。一个TIMESTAMP列对于记录一个INSERT或UPDATE操作的日期和时间是有用的，因为如果你不自己给它赋值，它自动地被设置为最近操作的日期和时间
time	-838:59:59到838:59:59	MySQL以'HH:MM:SS'格式来显示TIME值，但是允许你使用字符串或数字把值赋给TIME列
year	1901到2155	一个2或4位数字格式的年(缺省是4位)。允许的值是1901到2155，和0000(4位年格式)，如果你使用2位，1970-2069(70-69)。MySQL以YYYY格式来显示YEAR值，但是允许你把使用字符串或数字值赋给YEAR列

4.二进制数据

binary M字节数为M，允许长度为0~M的定长二进制字符串
varbinary M允许长度为0~M的变长二进制字符串，字节数为值的长度加1
bit 64 M位二进制数据，M最大值为64
tinyblob 可变长二进制数据，最多255个字节
blob 可变长二进制数据，最多2的16次方-1个字节
mediumblob 可变长二进制数据，最多2的24次方-1个字节
longblob 可变长二进制数据，最多2的32次方-1个字节

数据库varchar怎么实现可变长度的

2、varchar长度设置

1.结论：varchar长度需要多长就设置为多长，不必考虑2的幂。如果遇到varchar(256)的时候，那么何必不选择varchar(255)，这样还能节省1个存储长度的字节。

2.varchar(size)是保存多少个字符，而不是字节。

3.varchar最大是可以保存65535字节，但是根据不同编码，才能决定具体保存多少个字符。

4.如果是utf8格式，无论数字、字母还是汉子，都可以存放n个。varchar(20)，那么就是可以保存20个汉子。

4.1. 如果是gbk，varchar(n)中每个字符最多占2个字节，长度不能超过 $(65535 - 3 / 2)$ 。

4.2. 如果是utf-8，varchar(n)中每个字符最多占3个字节，长度不能超过 $(65535 - 3 / 3)$ 。

2.3、varchar(n)需要1-2个额外字节记录长度n的值

当 $n \leq 255$ 的时候，只需要1个字节记录（数据表示范围：0 ~ (2^8-1) ，即0~255；**当 $n > 255$ 的时候，那么需要2个字节存储n的值**（ $(2^8) \sim (2^{16})-1$ ，即256~65535）。

那么**varchar保存的实际值的长度保存在第一个或者前两个字节中。**

- a.如果varchar(20)，实际是需要1+20个字节的存储空间；
- b.如果varchar(255)，实际是需要1+255个字节的存储空间；
- c.如果varchar(256)，实际是需要2+256个字节的存储空间；
- d.如果varchar(500)，实际是需要2+500个字节的存储空间。

2.4 关于内存对齐的问题

$(2^n - 1)$ 是不能达到内存对齐的目的，因为varchar长度是以实际保存的字符串长度为准的。另外，InnoDB是以page为基本的存储单位，一页要么有一行要么有多行数据，数据的读取是基于page的，而不是按照行读取的。

数据库分区分表的策略

[关系型数据库](#)本身比较容易成为系统瓶颈，单机存储容量、连接数、处理能力都有限。当单表的数据量达到1000W或100G以后，由于查询维度较多，即使添加从库、优化索引，做很多操作时性能仍下降严重。

数据切分，及切分后对数据的整合

1.垂直切分和水平切分

垂直切分：垂直分库和垂直分表

- 垂直分库就是根据业务耦合性，将关联度低的不同表存储在不同的数据库。做法与大系统拆分为多个小系统类似，按业务分类进行独立划分。
- 垂直分表是基于数据库中的"列"进行，某个表字段较多，可以新建一张扩展表，**将不经常用或字段长度较大的字段拆分出去到扩展表中**。在字段很多的情况下(例如一个大表有100多个字段)，通过**"大表拆小表"**，更便于开发与维护，也能避免跨页问题，MySQL底层是通过数据页存储的，一条记录占用空间过大会导致跨页，造成额外的性能开销。数据库以行为单位将数据加载到内存中，这样表中字段长度较短且访问频率较高，内存能加载更多的数据，命中率更高，减少了磁盘IO，从而提升了数据库性能。

垂直切分的优点：

解决业务系统层面的耦合，业务清晰

与微服务的治理类似，也能对不同业务的数据进行分级管理、维护、监控、扩展等

高并发场景下，垂直切分一定程度的提升IO、数据库连接数、单机硬件资源的瓶颈

缺点：

部分表无法join，只能通过接口聚合方式解决，提升了开发的复杂度

分布式事务处理复杂

依然存在单表数据量过大的问题(需要水平切分)

2.水平切分：库内分表和分库分表

是根据表内数据内在的逻辑关系，将同一个表按不同的条件分散到多个数据库或多个表中，每个表中只包含一部分数据，从而使得单个表的数据量变小，达到分布式的效果。

水平切分的优点：

不存在单库数据量过大、高并发的性能瓶颈，提升系统稳定性和负载能力

应用端改造较小，不需要拆分业务模块

缺点：

跨分片的事务一致性难以保证

跨库的join关联查询性能较差

数据多次扩展难度和维护量极大

1、根据数值范围

按照时间区间或ID区间来切分。例如：按日期将不同月甚至是日的数据分散到不同的库中；将userId为1~9999的记录分到第一个库，10000~20000的分到第二个库，以此类推。

2、根据数值取模

一般采用hash取模mod的切分方式，例如：将 Customer 表根据 cusno 字段切分到4个库中，余数为0的放到第一个库，余数为1的放到第二个库，以此类推。

mysql join on where 和 join on and 的区别

join on and 是先对元组进行筛选，再进行join，得到的结果会大于join on where

join on where 是先进行join，对join后的结果进行筛选

sql中过滤条件放在on和where中的区别

inner join:结果没有区别

前者是先求笛卡尔积然后按照on后面的条件进行过滤

后者是先用on后面的条件过滤，再用where的条件过滤。

但是两者的结果是一样的

left join、right join:使用left join(right join)时on后面的条件只对右表(左表)有效。

join过程可以这样理解: where的执行顺序在join之后，on执行在join之中。

由于join要求索引列保留，所以对于left join来说on对右表生效

但是在左连接之后使用where，就能明显看到起到了过滤作用\

24.mysql查询语句很慢怎么检查，怎么优化

- (1) 查看数据库状态
- (2) 查找日志文件中是否有sql相关打印
- (3) 锁或者死锁（常见原因）
- (4) 查看mysql慢查询日志（默认情况下，mysql不开启慢查询日志）
可通过命令：show variables like '%slow_query_log%';

查看mysql是否开启了慢查询，以及慢查询日志在哪？

ON表示已开启，时间阈值是10s。

(5) **通过解释函数explain分析sql语句 (可以检查是否匹配到索引)**

```
1 | explain select id from wow_movie_info
```

只需要关注一个最重要的type信息，很明显体现出是否用到索引，type结果值从好到坏依次是：

system > const > eq_ref > ref > fulltext > ref_or_null > index_merge > unique_subquery > index_subquery > range > index > ALL

一般来说，得保证查询至少达到range级别，最好能达到ref，否则就可能会出现性能问题。

(6) 返回不必要的行或列

(7) 子查询过多，一般不建议使用子查询，可以把子查询改成join来优化

(8) 深度分页问题

MySQL会把前120条数据都查出来，抛弃前100条，返回20条

```
1 | select name,code from student limit 100,20
```

随着分页深度的增大，可能会变成 1000000,20。如此大的数据量，速度一定快不起来，最好的方式是增加一个条件：

```
1 | select name,code from student where id > 1000000 limit 20
```

主键索引会直接链接到1000000处，然后查出20条，这个方式需要把上次查询出来的最大id以参数的方式传过来

(9) 单纯的数据量过大，分表或分库+分表

mysql判断慢查询的条件

一、MySQL数据库有几个配置选项可以帮助我们及时捕获低效SQL语句

1, slow_query_log

这个参数设置为ON，可以捕获执行时间超过一定数值的SQL语句。

2, long_query_time

当SQL语句执行时间超过此数值时，就会被记录到日志中，建议设置为1或者更短。

3, slow_query_log_file

记录日志的文件名。

4, log_queries_not_using_indexes

这个参数设置为ON，可以捕获到所有未使用索引的SQL语句，尽管这个SQL语句有可能执行得挺快。

优化

首先看是否建立索引-建索引的规则

如果已经建立索引，则进一步对数据表进行分区分表、减少数据量

25.redis使用单线程还是多线程？有多少个库

Redis 是跑在单线程中的，所有的操作都是按照顺序线性执行的，但是由于读写操作等待用户输入或输出都是阻塞的，所以 I/O 操作在一般情况下往往不能直接返回，这会导致某一文件的 I/O 阻塞导致整个进程无法对其它客户提供服务，而 I/O 多路复用就是为了解决这个问题而出现的。

内部IO操作使用多路复用

redis有16个库，默认使用第0个，使用select切换不同的库

redis部署两种方式

单机部署

集群部署

26.redis基本数据类型，RDB和AOF

string,hash(多字段存储),list,set(去重),sorted set

数据类型	可以存储的值	操作	应用场景
STRING	字符串、整数或者浮点数	对整个字符串或者字符串的其中一部分执行操作 对整数和浮点数执行自增或者自减操作	做简单的键值对缓存
LIST	列表	从两端压入或者弹出元素 对单个或者多个元素进行修剪，只保留一个范围内的元素	存储一些列表型的数据结构，类似粉丝列表、文章的评论列表之类的数据
SET	无序集合	添加、获取、移除单个元素 检查一个元素是否存在于集合中 计算交集、并集、差集 从集合里面随机获取元素	交集、并集、差集的操作，比如交集，可以把两个人的粉丝列表整一个交集
HASH	包含键值对的无序散列表	添加、获取、移除单个键值对 获取所有键值对 检查某个键是否存在	结构化的数据，比如一个对象
ZSET	有序集合	添加、获取、删除元素 根据分值范围或者成员来获取元素 计算一个键的排名	去重但可以排序，如获取排名前几名的用户

27.CAP理论

C：consistency一致性，一致性指的是所有节点在同一时间的数据完全一致。就好比刚刚举得例子中，小明和小华读取的都是正确的数据，对他们用户来说，就好像是操作了同一个数据库的同一个数据一样。

A：availability可用性，可用性指服务一直可用，而且是正常响应时间。就好比刚刚的N1和N2节点，不管什么时候访问，都可以正常的获取数据值。而不会出现问题。好的可用性主要是指系统能够很好的为用户服务，不出现用户操作失败或者访问超时等用户体验不好的情况。

P：分区容错性，分区容错性指在遇到某节点或网络分区故障的时候，仍然能够对外提供满足一致性和可用性的服务。就好比是N1节点和N2节点出现故障，但是依然可以很好地对外提供服务。

redis实现的cap

Redis单机模式：实现了数据的一致性Consistency(一致性)

redis哨兵模式：实现了数据的一致性和高可用性 Consistency(一致性)、Availability(可用性)

redis 集群模式：实现了分区容忍性和高可用性 Availability(可用性)、Partition Tolerance(分区容错性)

28.redis的6种底层数据结构

string：由SDS构成

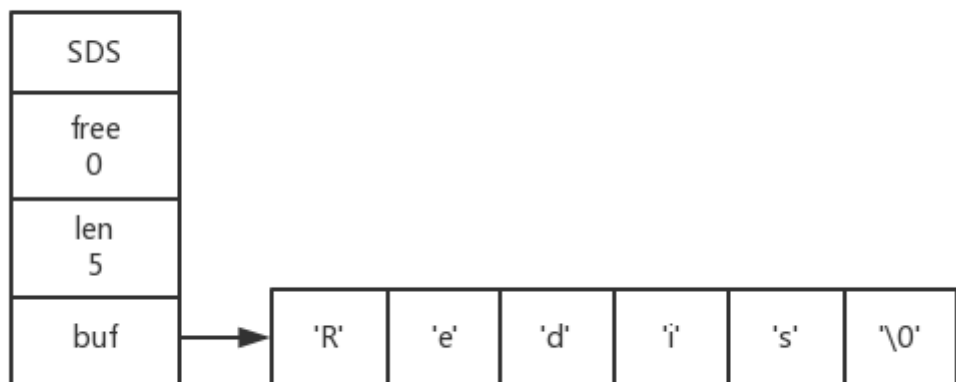
list：由压缩链表和双向链表实现

hash：由ziplist或hashtable实现

set：由intset或hashtable实现

zset：由ziplist或skiplist实现

简单动态字符串 (SDS)：redis 是使用 C 语言写的，而 C 语言里的字符串类型比较原始，比如使用 `\0` 作为字符结束符。所以 redis 实现了属于自己的字符串类型，比如字符串长度，预先分配内存，动态拓展等特点，也保证了处理安全性。



<http://blog.csdn.net/yx0628>

压缩列表(ziplist)：经过特殊编码的一块连续内存，能有效的节省内存。

压缩列表 (ziplist) 是**列表键和哈希键**的底层实现之一。当一个列表键只包含少量列表项并且每个都是小整数值或者长度比较短的字符串时，Redis 就采用压缩列表做底层实现。

适用于长度较小的值，因为他是由连续的空间实现的。

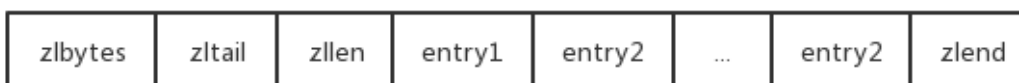
存取的效率高，内存占用小，但由于内存是连续的，在修改的时候要重新分配内存

在数据量比较小的时候使用的是ziplist

当list对象同时满足以下两个条件是，使用的ziplist编码

- 1.list对象保存的所有字符串元素长度都小于64字节
- 2.list对象保存的元素数量小于512个

压缩列表是 Redis 为了节约内存而实现的，是一系列特殊编码的连续内存块组成的顺序型数据结构。



<https://blog.csdn.net/yx0628>

zlbytes：4 字节。记录整个压缩列表占用的内存字节数，在内存重分配或者计算 zlend 的位置时使用。

zltail：4 字节。记录压缩列表表尾节点记录压缩列表的起始地址有多少个字节，可以通过该属性直接确定表尾节点的地址，无需遍历。

zllen：2 字节。记录了压缩列表包含的节点数量，由于只有 2 字节大小，那么小于 65535 时，表示节点数量。等于 65535 时，需要遍历得到总数。

entry：列表节点，长度不定，由内容决定。

zlend：1 字节，特殊值 0xFF，用于标记压缩列表的结束。

压缩列表节点保存一个字节数组或者一个整数值。

字节数组可以是下列值：

- 长度小于等于 2^6-1 字节的字节数组
 - 长度小于等于 $2^{14}-1$ 字节的字节数组
 - 长度小于等于 $2^{32}-1$ 字节的字节数组
- 整数可以是六种长度：
- 4 位长，介于 0 到 12 之间的无符号整数
 - 1 字节长的有符号整数
 - 3 字节长的有符号整数
 - int16_t 类型整数
 - int32_t 类型整数
 - int64_t 类型整数

每个压缩列表节点的结构如图：



previous_entry_length 属性以字节为单位，记录了压缩列表中前一个节点的长度。该属性的长度可以是 1 字节或者 5 字节。如果前一个节点的长度小于 254 字节，那么该属性长度为 1 字节，保存小于 254 的值。如果前一节点的长度大于等于 254 字节，那么长度需要为 5 字节，属性的第一字节会被设置为 0xFE (254) 之后的 4 个字节保存其长度。

压缩列表的从表尾到表头遍历：

1. 首先，有指向压缩列表表尾节点起始地址的指针 p1（指向表尾节点的指针可以通过指向压缩列表起始地址的指针加上 zltail 属性的值得出）；
 2. 通过用 p1 减去节点的 previous_entry_length 属性，得到前一个节点的起始地址的指针。
 3. 如此循环，最终从表尾遍历到表头节点。
- encoding 属性记录了节点的 content 属性所保存的数据的类型和长度：

- 一字节、两字节或五字节长，值的最高位为 00、01 或者 10 的是字节数组编码，字节数组的长度由编码除去最高两位之后的其他位记录；
- 一字节长，值的最高位以 11 开头的是整数编码，这种编码表示保存是整数值，整数值的类型和长度由其他位记录。

出现新增或删除节点导致 previous_entry_length 1 字节或者 5 字节的长度变化，是连锁更新的问题，但出现几率比较小，而且数量不多的情况下不会对性能造成影响。

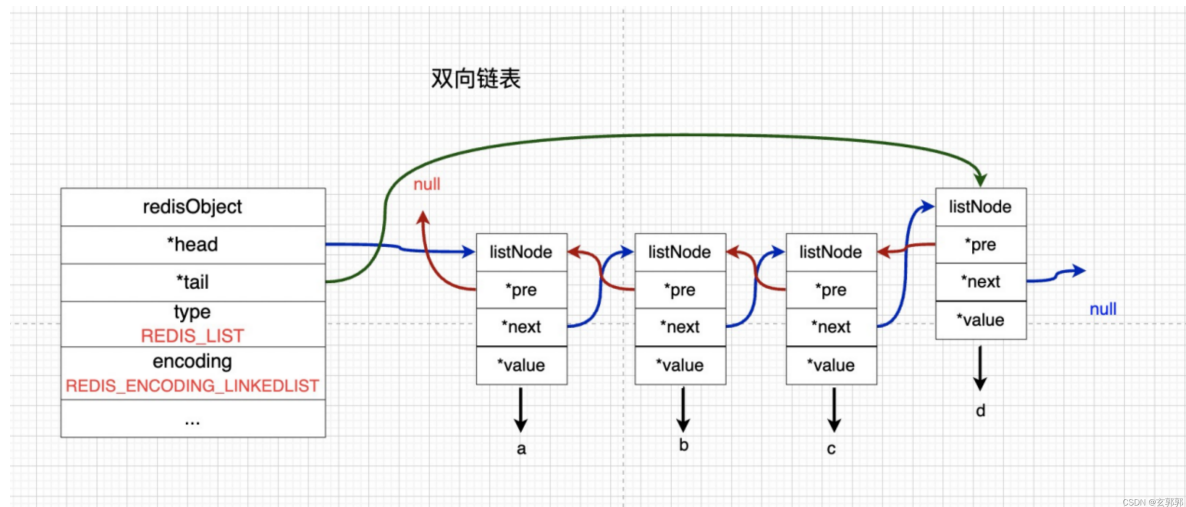
快速列表：将 ziplist 组织为了一个双向链表，由于 ziplist 的内部连续性，能降低链表的内存碎片问题，提高内存利用率。ziplist+linkedlist

双向链表：

在使用redis的list数据结构时，存储数据较大时，list对象已经不能满足上面描述的ziplist条件，则会使用linkedlist

优点：修改效率高

缺点：保存前后指针，会占内存。



hashtable哈希表：

通过哈希算法来实现 key-value 的映射操作，采用 链地址法 解决了 hash 冲突，一般时间复杂度能达到 $O(1)$ 。

一个哈希表里面可以有多个哈希节点，而每个哈希节点就保存了字典中的一个键值对

参数解析：

dict中

type标识dictType类型，privatedata是指向特定数据类型的数组；

ht包含了两个哈希表的数组，其中一个ht[0]保存哈希表，ht[1]只会在对ht[0]进行rehash时使用

dict中

table是一个数组，其中一个数组元素都是一个指向哈希表节点的指针，每一个哈希表节点都保存了一对键值对。

size记录了table的大小，也即是哈希表的大小

used记录了当前已使用节点的数量

sizemask=size-1, 决定了一个键应该放table数组的哪个索引上

dicEntry中

key保存键值对中的键，

v保存这键值对中的值

next为指向另一个哈希表节点的指针，当有hash冲突的时候

哈希表的扩展和收缩

当以下条件中的任意一个被满足时，程序会自动开始对哈希表执行扩展操作

1：服务器目前没有在执行bgsave命令或者bgrewriteof命令、并且哈希表的负载因子大于等于1

2：服务器目前正在执行bgsave命令或者bgrewriteof命令并且哈希表的负载因子大于等于5rehash

rehash

跟java中的hashmap扩容机制差不多

随着操作的不断执行，哈希表保存的键值对会逐渐地增多或减少，为了让哈希表的负载因子维持在一个合理的

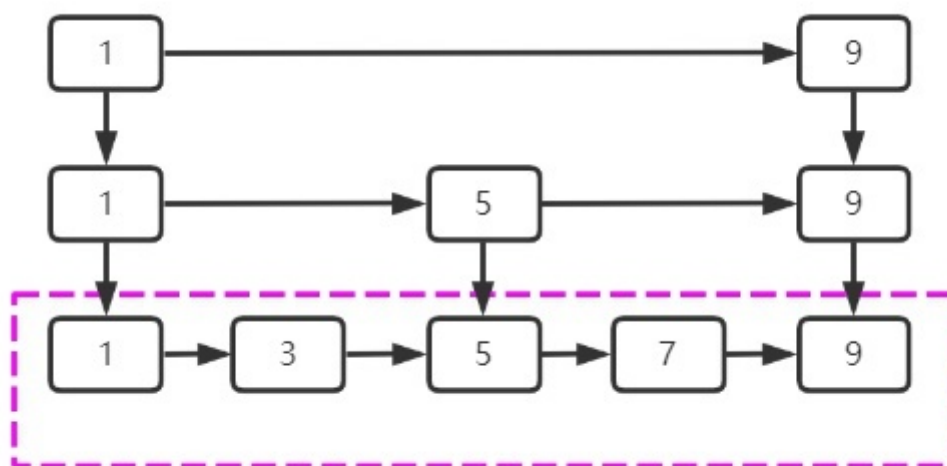
范围内当哈希表保存的键值对数量太多或者太少时程序需要对哈希表的大小进行相应的扩展或者收缩

渐进式hash

如果哈希表里保存的键值对数量是四百万、四千万甚至四亿个键值对，那么要一次性将这些键值对全部rehash到ht[1]的话，庞大的计算量可能会导致服务器在一段时间内停止服务。因此，为了避免rehash对服务器性能造成影响，服务器不是一次性将 ht[0]里面的所有键值对全部rehash到ht[1]，而是分多次、渐进式地将ht[0]里面的键值对慢慢地rehash到ht[1]。

好处：在于它采取分而治之的方式，将rehash键值对所需的计算工作均摊到对字典的每个添加、删除、查找和更新操作上，从而避免了集中式rehash 而带来的庞大计算量。

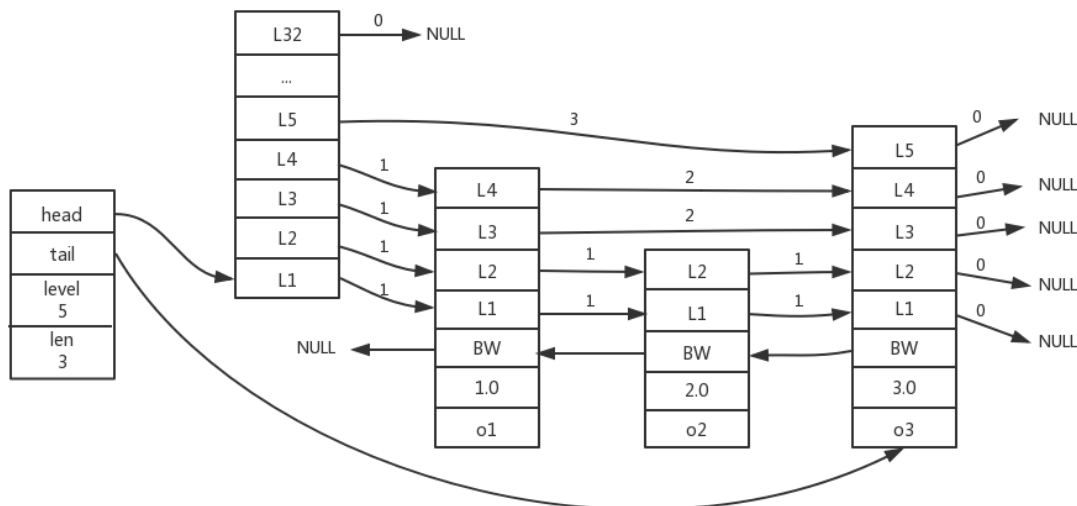
跳跃表：一个多层有序链表，每一层都是对下面一层的有序提取，能降低搜索次数，有点像有序二叉树的搜索一样。



https://blog.csdn.net/lincoln_1

跳跃表是一种有序数据结构，通过在每个节点中维持多个指向其他节点的指针，从而达到快速访问节点的目的。Redis 使用跳跃表作为有序集合键的底层实现之一。

跳跃表在 Redis 中，只有两个地方用到：一个是实现有序集合对象，另一个是在集群节点中用作内部数据结构。



跳跃表中：

head：指向跳跃表的表头节点。

tail：指向跳跃表的表尾节点。

level：记录当前跳跃表中，层数最高的节点的层数（表头节点的层数不计算）。

length：记录跳跃表的长度，即包含节点的数量。

level：每一层都有前进指针和跨度，从头到尾遍历时，访问会沿着层的前进指针进行。

BW：后退指针，指向前一个节点，从尾到头遍历时使用。

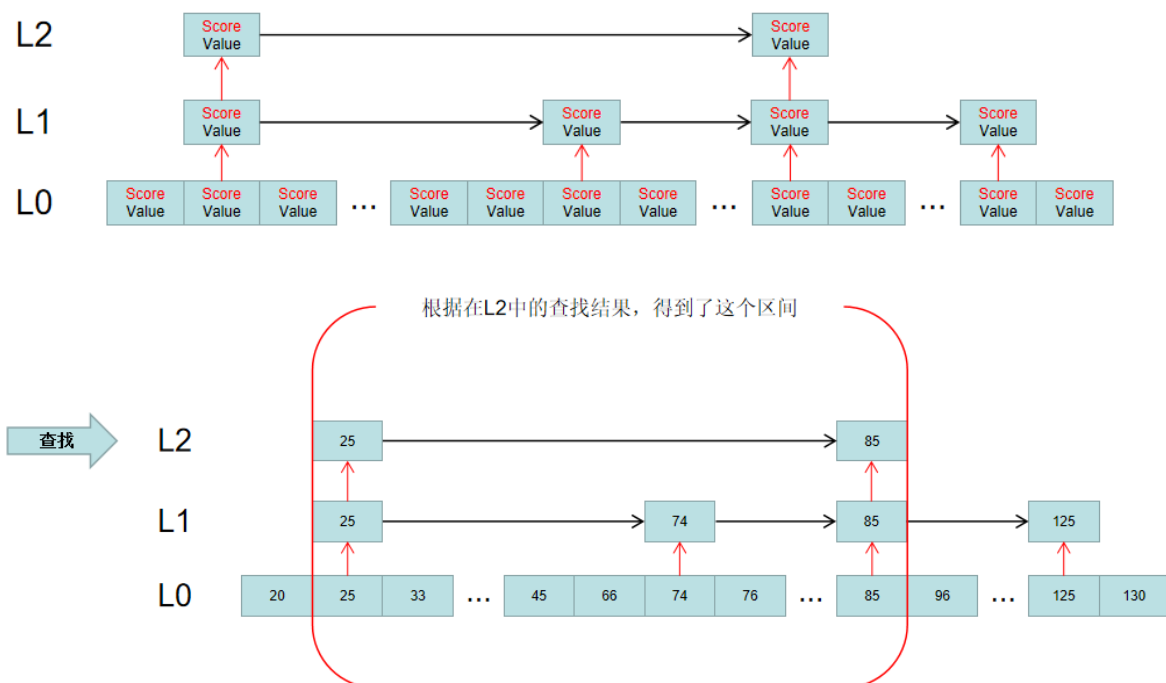
score：分值，跳跃表中的分值按从小到大排列。

obj：成员对象，各个节点保存有各个成员对象。

zset不是一个有序链表（因为对链表的查询时间是 $O(n)$ 很浪费时间），实现存储的数据结构类似于kafka的稀疏索引

kafka在进行持久化的时候，生成了两个文件：一个是log一个是index；log文件以链表的形式保存着消息的详细信息，但index中保存着这些消息的索引，而且并不是所有的索引都存储在index中，而是部分存储在index中，在查找的时候使用一个区间进行查找

跳跃表的实现是多层的稀疏索引



整数集合：

一个有序的整数集合，不会有重复元素。底层由数组实现，整数集合的每个元素都是数组的一个数组项，各个项在数组中按从小到大排列。length 属性记录了包含的元素数量，即数组的长度。

有序不重复的连续空间，特性如下：

- 1、节约内存，但由于是连续空间，修改效率不高
- 2、集合中的数都是整数时，且数据量不超过512个时，使用intset
- 3、整数集合是集合键的底层实现之一
- 4、整数集合的底层实现是数组，这个数组以有序、无重复的方式保存集合元素，在有需要的时候，程序会根据新添加元素的类型，改变这个数组的类型。
- 5、升级操作为整数集合带来了操作上的灵活性，并且尽可能的节约了内存。
- 6、整数集合只支持升级操作，不支持降级操作

29.redis两种数据持久化方式

rdb快照（每次将未过期的数据快照上传到服务器中）：RDB持久化主要是通过SAVE和BGSAVE两个命令对Redis数据库中当前的数据做snapshot并生成rdb文件来实现的。其中SAVE是阻塞的，BGSAVE是非阻塞的，通过fork了一个子进程来完成的。在Redis启动的时候会检测rdb文件，然后载入rdb文件中未过期的数据到服务器中

服务器通过维护 dirty 计数器和 lastsave 属性分别记录距离上次成功执行SAVE或者BGSAVE命令之后，服务器对数据库状态进行修改的次数和最后一次成功SAVE或者BGSAVE的UNIX时间戳。由Redis周期性操作函数 serverCron 默认每隔 100ms 来检测是否满足配置信息中的要求，然后再决定是否执行SAVE或者BGSAVE命令来对数据库进行备份。

aof增量更新：通过将存储每次执行的客户端命令，然后由一个伪客户端来执行这些命令将数据写入到服务器中的方式实现的。一共分为命令追加 (append)、文件写入、文件同步 (sync) 三个步骤完成的

- RDB可以理解为是一种全量数据更新机制，AOF可以理解为是一种增量的更新机制，AOF重写可以理解为是一种全量+增量的更新机制（第一次是全量，后面都是增量）
- RDB适合服务器数据库数据量小，写命令频繁的场景
- AOF适合数据量大，写命令少的场景
- AOF重写适合在AOF运行了很久的写命令之后执行

30.redis的优点

- 1、完全基于内存，绝大部分请求是纯粹的内存操作，非常快速。数据存在内存中，类似于 HashMap，HashMap 的优势就是查找和操作的时间复杂度都是O(1)；
- 2、数据结构简单，对数据操作也简单，Redis 中的数据结构是专门进行设计的；
- 3、采用单线程，避免了不必要的上下文切换和竞争条件，也不存在多进程或者多线程导致的切换而消耗 CPU，不用去考虑各种锁的问题，不存在加锁释放锁操作，没有因为可能出现死锁而导致的性能消耗；
- 4、使用多路 I/O 复用模型，非阻塞 IO；其实就是一种同步IO模型，它实现了一个线程可以监视多个文件句柄；一旦某个文件句柄就绪，就能够通知应用程序进行相应的读写操作；而没有文件句柄就绪时，就会阻塞应用程序，交出cpu。
- 5、使用底层模型不同，它们之间底层实现方式以及与客户端之间通信的应用协议不一样，Redis 直接自己构建了 VM 机制，因为一般的系统调用系统函数的话，会浪费一定的时间去移动和请求；**虚拟内存机制**就是暂时把不经常访问的数据(冷数据)从内存交换到磁盘中，从而腾出宝贵的内存空间用于其它需要访问的数据(热数据)。通过VM功能可以实现冷热数据分离，使热数据仍在内存中、冷数据保存到磁盘。这样就可以避免因为内存不足而造成访问速度下降的问题。

31.redis的事务和db的事务有什么区别

Redis 的事务保证了 ACID 中的一致性 (C) 和隔离性 (I)，但并不保证原子性 (A) 和持久性 (D)。

对于原子性而言，要么都成功，要么都不成功，而 redis 的事务中途某个语句出错了，比如 key 类型 出错了，还会继续执行其他语句；

对于持久性而言，redis 即使开启了最严格的数据落地，由于保存是由后台线程进行的，主线程不会阻塞直到保存成功，所以从命令执行成功到数据保存到硬盘之间，还是有一段非常小的间隔，所以这种模式下的事务也是不持久的。

Redis事务是指将多条命令加入队列，一次批量执行多条命令，每条命令会按顺序执行，事务执行过程中不会受客户端传入的命令请求影响。

Redis事务的相关命令如下：

- MULTI：标识一个事务的开启，即开启事务；组队命令
- EXEC：执行事务中的所有命令，即提交；执行被组队的命令
- DISCARD：放弃事务，放弃组队，之前组队的命令都放弃执行；和回滚不一样，Redis事务不支持回滚。
- WATCH：监视Key改变，用于实现乐观锁。如果监视的Key的值改变，事务最终会执行失败。
- UNWATCH：放弃监视。

redis事务执行出错的情况：

1.组队时命令出错

exec时会报错，组队中的命令都不会成功执行

2.执行中报错

执行阶段如果哪个命令执行失败，则会跳过这个命令继续执行其他命令，不会回滚

redis事务的锁机制

多个请求同时出现会导致事务冲突——解决事务冲突，引入锁机制

1.悲观锁

每次在操作数据之前先对数据加锁，等操作完成释放锁后其他请求才能继续操作。一般在关系型数据库中使用，效率低

2.乐观锁

在操作之前，先判断数据是否被其他请求更新，常用方式为加版本号。适用于多读的类型，能够提高吞吐量；**redis使用的就是这种类型的锁机制**

redis三种集群模式

1.主从模式

master节点和slave节点，通过同步和命令传播两种方式进行数据复制

- 同步（SYNC）用来将从服务器的状态 更新到 和主服务器 一致。白话文解释就是从服务器主动获取主服务器的数据。保持数据一致。具体实现是，主服务器收到SYNC命令后，生成RDB快照文件，然后发送给从服务器。
- 命令传播（command propagate）用于在主服务器数据被修改后，主从不一致，为了让从服务器保持和主服务器状态一致，而做的命令传播。白话文解释就是主服务器收到客户端修改数据命令后，数据库数据发生变化，同时将命令缓存起来，然后将缓存命令发送到从服务器，从服务器通过载入缓存命令来达到主从数据一致。这就是所谓的命令传播。

2.哨兵模式

为了弥补主从复制集群中主机宕机后，主备切换的复杂性而演变出来的。哨兵顾名思义，就是用来监控的，主要作用就是监控主从集群，自动切换主备，完成集群故障转移。

3.cluster

哨兵模式基本已经实现了高可用，但是每台机器都存储相同内容，很浪费内存，所以Redis Cluster实现了分布式存储。每台机器节点上存储不同的内容。

redis过期策略

在set key的时候，会设计一个过期时间

定时过期

每个设置过期时间的key都需要创建一个定时器，到过期时间就会立即对key进行清除。该策略可以立即清除过期的数据，对内存很友好；但是会占用大量的CPU资源去处理过期的数据，从而影响缓存的响应时间和吞吐量。

惰性过期

只有当访问一个key时，才会判断该key是否已过期，过期则清除。该策略可以最大化地节省CPU资源，却对内存非常不友好。极端情况可能出现大量的过期key没有再次被访问，从而不会被清除，占用大量内存。

定期过期

每隔一定的时间，会扫描一定数量的数据库的expires字典中一定数量的key，并清除其中已过期的key。该策略是前两者的一个折中方案。通过调整定时扫描的时间间隔和每次扫描的限定耗时，可以在不同情况下使得CPU和内存资源达到最优的平衡效果。

expires字典会保存所有设置了过期时间的key的过期时间数据，其中，key是指向键空间中的某个键的指针，value是该键的毫秒精度的UNIX时间戳表示的过期时间。键空间是指该Redis集群中保存的所有键。

Redis中同时使用了惰性过期和定期过期两种过期策略。

假设Redis当前存放30万个key，并且都设置了过期时间，如果你每隔100ms就去检查这全部的key，CPU负载会特别高，最后可能会挂掉。

因此，redis采取的是定期过期，每隔100ms就随机抽取一定数量的key来检查和删除的。

但是呢，最后可能会有很多已经过期的key没被删除。这时候，redis采用惰性删除。在你获取某个key的时候，redis会检查一下，这个key如果设置了过期时间并且已经过期了，此时就会删除。

redis内存淘汰策略

volatile-lru：当内存不足以容纳新写入数据时，从设置了过期时间的key中使用LRU（最近最少使用）算法进行淘汰；

allkeys-lru：当内存不足以容纳新写入数据时，从所有key中使用LRU（最近最少使用）算法进行淘汰。

volatile-lfu：4.0版本新增，当内存不足以容纳新写入数据时，在过期的key中，使用LFU算法进行删除key。

allkeys-lfu：4.0版本新增，当内存不足以容纳新写入数据时，从所有key中使用LFU算法进行淘汰；

volatile-random：当内存不足以容纳新写入数据时，从设置了过期时间的key中，随机淘汰数据；。

allkeys-random：当内存不足以容纳新写入数据时，从所有key中随机淘汰数据。

volatile-ttl：当内存不足以容纳新写入数据时，在设置了过期时间的key中，根据过期时间进行淘汰，越早过期的优先被淘汰；

noeviction：默认策略，当内存不足以容纳新写入数据时，新写入操作会报错。

怎么保证mysql和redis的数据一致性

1.延时双删策略

在写入数据库前后都进行缓存删除，并设置合理的超时时间

```
public void write(String key,Object data){
    redis.delKey(key);
    db.updateData(data);
    Thread.sleep(500);
    redis.delKey(key);
}
```

需要评估自己的项目的读数据业务逻辑的耗时。这么做的目的，就是确保读请求结束，写请求可以删除读请求造成的缓存脏数据。

2.异步更新缓存：先写mysql，通过binlog异步更新redis

MySQL binlog增量订阅消费+消息队列+增量数据更新到redis

- 1) **读Redis**: 热数据基本都在Redis
- 2) **写MySQL**: 增删改都是操作MySQL
- 3) **更新Redis数据**: MySQL的数据操作binlog, 来更新到Redis

Redis更新

- 1) 数据操作主要分为两大块:

一个是全量(将全部数据一次写入到redis)

一个是增量(实时更新)

这里说的是增量,指的是mysql的update、insert、delete变更数据。

- 2) 读取binlog后分析, 利用消息队列,推送更新各台的redis缓存数据。

这样一旦MySQL中产生了新的写入、更新、删除等操作, 就可以把binlog相关的消息推送至Redis, Redis再根据binlog中的记录, 对Redis进行更新。

其实这种机制, 很类似MySQL的主从备份机制, 因为MySQL的主备也是通过binlog来实现的数据一致性。

3.先写 MySQL, 再删除 Redis

- **比较推荐这种方式**, 删除 Redis 如果失败, 可以再多重试几次, 否则报警出来;
- 这个方案, 是实时性中最好的方案, 在一些高并发场景中, 推荐这种。

32.redis缓存异常问题:

缓存雪崩、缓存穿透、附加、缓存击穿、缓存预热、缓存降级等

缓存雪崩:

缓存雪崩是指缓存同一时间大面积的失效, 所以, 后面的请求都会落到数据库上, 造成数据库短时间内承受大量请求而崩掉。

解决方案

- 1、缓存数据的过期时间设置随机, 防止同一时间大量数据过期现象发生。
- 2、一般并发量不是特别多的时候, 使用最多的解决方案是加锁排队。
- 3、给每一个缓存数据增加相应的缓存标记, 记录缓存的是否失效, 如果缓存标记失效, 则更新数据缓存。

缓存穿透:

缓存穿透是指缓存和数据库中都没有的数据, 导致所有的请求都落到数据库上, 造成数据库短时间内承受大量请求而崩掉。

解决方案

- 1、接口层增加校验, 如用户鉴权校验, id做基础校验, id<=0的直接拦截;
- 2、从缓存取不到的数据, 在数据库中也没有取到, 这时也可以将key-value对写为key-null, 缓存有效时间可以设置短点, 如30秒(设置太长会导致正常情况也没法使用)。这样可以防止攻击用户反复用同一个id暴力攻击;
- 3、采用布隆过滤器, 将所有可能存在的数据哈希到一个足够大的 bitmap 中, 一个一定不存在的数据会被这个 bitmap 拦截掉, 从而避免了对底层存储系统的查询压力。

对于空间的利用到达了一种极致，那就是Bitmap和布隆过滤器(Bloom Filter)。

Bitmap：典型的的就是哈希表

缺点是，Bitmap对于每个元素只能记录1bit信息，如果还想完成额外的功能，恐怕只能靠牺牲更多的空间、时间来完成了。

布隆过滤器（推荐）

就是引入了 $k(k>1)$ 个相互独立的哈希函数，保证在给定的空间、误判率下，完成元素判重的过程。

它的优点是空间效率和查询时间都远远超过一般的算法，缺点是有一定的误识别率和删除困难。

Bloom-Filter算法的核心思想就是利用多个不同的Hash函数来解决“冲突”。

Hash存在一个冲突（碰撞）的问题，用同一个Hash得到的两个URL的值有可能相同。为了减少冲突，我们可以多引入几个Hash，如果通过其中的一个Hash值我们得出某元素不在集合中，那么该元素肯定不在集合中。只有在所有的Hash函数告诉我们该元素在集合中时，才能确定该元素存在于集合中。这便是Bloom-Filter的基本思想。

Bloom-Filter一般用于在大数据量的集合中判定某元素是否存在。

缓存击穿：

缓存击穿是指缓存中没有但数据库中有数据（一般是缓存时间到期），这时由于并发用户特别多，同时读缓存没读到数据，又同时去数据库去取数据，引起数据库压力瞬间增大，造成过大压力。和缓存雪崩不同的是，缓存击穿指并发查同一条数据，缓存雪崩是不同数据都过期了，很多数据都查不到从而查数据库。

解决方案

- 1、设置热点数据永远不过期
- 2、加互斥锁，互斥锁

缓存降级：

当访问量剧增、服务出现问题（如响应时间慢或不响应）或非核心服务影响到核心流程的性能时，仍然需要保证服务还是可用的，即使是有损服务。系统可以根据一些关键数据进行自动降级，也可以配置开关实现人工降级。

缓存降级的最终目的是保证核心服务可用，即使是有损的。而且有些服务是无法降级的（如加入购物车、结算）。

在进行降级之前要对系统进行梳理，看看系统是不是可以丢卒保帅；从而梳理出哪些必须誓死保护，哪些可降级；比如可以参考日志级别设置预案：

- 1、一般：比如有些服务偶尔因为网络抖动或者服务正在上线而超时，可以自动降级；
- 2、警告：有些服务在一段时间内成功率有波动（如在95~100%之间），可以自动降级或人工降级，并发送告警；
- 3、错误：比如可用率低于90%，或者数据库连接池被打爆了，或者访问量突然猛增到系统能承受的最大阈值，此时可以根据情况自动降级或者人工降级；
- 4、严重错误：比如因为特殊原因数据错误了，此时需要紧急人工降级。

服务降级的目的，是为了防止Redis服务故障，导致数据库跟着一起发生雪崩问题。因此，对于不重要的缓存数据，可以采取服务降级策略，例如一个比较常见的做法就是，Redis出现问题，不去数据库查询，而是直接返回默认值给用户。

缓存热点key

缓存中的一个Key(比如一个促销商品), 在某个时间点过期的时候, 恰好在这个时间点对这个Key有大量的并发请求过来, 这些请求发现缓存过期一般都会从后端DB加载数据并回设到缓存, 这个时候大并发的请求可能会瞬间把后端DB压垮。

解决方案

对缓存查询加锁, 如果KEY不存在, 就加锁, 然后查DB入缓存, 然后解锁; 其他进程如果发现有锁就等待, 然后等解锁后返回数据或者进入DB查询

缓存预热

缓存预热就是系统上线后, 将相关的缓存数据直接加载到缓存系统。这样就可以避免在用户请求的时候, 先查询数据库, 然后再将数据缓存的问题! 用户直接查询事先被预热的缓存数据!

解决方案

- 1、直接写个缓存刷新页面, 上线时手工操作一下;
- 2、数据量不大, 可以在项目启动的时候自动进行加载;
- 3、定时刷新缓存;

33.redis时间复杂度

<https://blog.csdn.net/lxw1844912514/article/details/120775818>

String类型

命令	时间复杂度
set	$O(1)$
get	$O(1)$
del	$O(k)$, k 是键的个数
mset	$O(k)$, k 是键的个数
mget	$O(k)$, k 是键的个数
incr	$O(1)$
decr	$O(1)$
incrby	$O(1)$
decrby	$O(1)$
incrbyfloat	$O(1)$
append	$O(1)$
strlen	$O(1)$
setrange	$O(n)$, n 为更改字符串长度
getrange	$O(n)$, n 为获取字符串长度

List列表

命令	时间复杂度
rpush	$O(k)$, k 是field的个数
lpush	$O(k)$, k 是field的个数
linsert	$O(n)$, n 是插入位置距离表头或表尾的距离
lrange	$O(s+n)$, s 是start的偏移量, n 是start到end的范围
lindex	$O(n)$, n 是索引的偏移量
llen	$O(1)$
lpop	$O(1)$
rpop	$O(1)$
lrem	$O(n)$, n 是列表的长度
ltrim	$O(n)$, n 是要裁剪的元素总数
lset	$O(n)$, n 是索引的偏移量
blpop	$O(1)$

Hash类型

命令	时间复杂度
hset	$O(1)$
hget	$O(1)$
hdel	$O(k)$, k是键的个数
hlen	$O(1)$
hgetall	$O(k)$, k是field的个数
hmget	$O(k)$, k是field的个数
hmset	$O(k)$, k是field的个数
hexists	$O(1)$
hkeys	$O(k)$, k是field的个数
hvals	$O(k)$, k是field的个数
hsetnx	$O(1)$
hincrby	$O(1)$
hincrbyfloat	$O(1)$
hstrlen	$O(1)$

Set集合

命令	时间复杂度
sadd	$O(k)$, k 为元素个数
srem	$O(k)$, k 为元素个数
scard	$O(1)$
sismember	$O(1)$
srndmember	$O(\text{count})$
spop	$O(1)$
smembers	$O(n)$, n 为元素总数
sinter	$O(m*k)$, k 为多个集合中元素较少的个数, m 是键个数
suinon	$O(k)$, k 为多个集合元素个数和
sdiff	$O(k)$, k 为多个集合元素个数和

命令	时间复杂度
zadd	$O(k \cdot \log(n))$, k为添加 成员个数, n为当前成员个数
zcard	$O(1)$
zscore	$O(1)$
zrank zrevrank	$O(\log(n))$, n为当前成员个数
zrem	$O(k \cdot \log(n))$, k为删除成员个数, n为当前成员个数
zincrby	$O(\log(n))$, n为当前成员个数
zrange zrevrange	$O(\log(n)+k)$, k为要获取成员个数, n为当前成员个数
zrangebyscore zrevrangebyscore	$O(\log(n)+k)$, k为要获取成员个数, n为当前成员个数
zcount	$O(\log(n)+k)$, k为要获取成员个数, n为当前成员个数

数据分析面经

介绍一下A/B测试

AB测试，是验证假设的好方法。

大家在实验中常用到实验思维，根据实验思维，我们要更好的进行归因实验，通常要满足以下条件：

- 对照：有其他对照组作为对比，就能真正看出来效果。而且不同组间的效果差异要足够明显，才能验证我们的判断
- 随机：为了排除实验条件以外的干扰因素，我们需要确保两个组的用户是随机选取的，这是为了排除用户差异对实验结果的影响
- 大样本：这里的样本量是指数据量，包括用户、行为和时间跨度，样本量越大，越容易排除个体差异的影响，也更容易验证统计上的显著性

实验设计：

1.单因素实验设计

所谓单因素实验设计，是指实验中只有一个影响因素变量，其他的内容都保持不变的实验方法。举个例子，两个实验组，一组用广告图A，一组用广告图B，这两个组进行实验对比，最后发现A组比B组效果好，那么我们就可以认为这是A广告图的作用。

2.多因素实验设计

多因素实验设计，是指实验中有多个影响因素变量。比如你想同时测试广告图（AB）和广告弹出方式

(AB) 对转化率的影响，这里面有两个变量，对应的有4种组合条件：

广告A，弹出方式A

广告A，弹出方式B

广告B，弹出方式A

广告B，弹出方式B

多因素实验设计的好处在于，除了可以检测同一个变量、不同实验条件之间的差异之外，还能对变量之间的交互效果进行检验。用上个例子做说明，如果在单因素AB实验里，我们发现广告A比广告B的效果好，弹出方式A比弹出方式B的效果好，但是广告A+弹出方式A的组合情况却不是最好的，因为他们之间的组合，产生了化学作用。这种情况下，就必须使用多因素实验设计来做。

如何理解产品经理

一个互联网产品的出现，一定是要满足一部分人的需求、需要，一个不能解决用户需求、需要的产品，只能是昙花一现，所以说，产品的核心在于需求。很多时候，用户的需求并不明显，往往表露出来的仅是表面需要，这个时候，我们需要一个人站出来，挖掘这个需求，把表面需求中的隐含需求找出来，形成可以用于软件设计、软件开发的文档，而我们需要这个人，就是产品经理。

一个普通产品经理的核心能力，就是需求挖掘。那么产品经理仅仅是要做需求挖掘吗？显然不是的，需求挖掘只是核心能力，不同团队中的产品经理，可能会遇到的情况是不同的。大公司的产品经理，还要兼有团队管理能力，市场分析，销售策略，盈利预测等。

归因分析

如何将用户行为的‘贡献’合理地分配到每一个渠道，这就是归因分析要解决的问题，通过渠道归因来衡量渠道的效果，反过来可以指导业务人员在渠道投放时合理分配投入。

对于归因分析而言，一个很重要的命题即是，针对当前的场景和目标，怎么把“贡献”合理分配到每一个坑位上。下面我们就以站内归因为例，普及一下几种常见的归因分析计算思路。假设一个用户一天内使用APP的行为顺序如下：

首先，启动APP，进入首页，先行搜索，在搜索结果列表页看到了商品A，浏览了商品A的详情，觉得不错，但是并未购买，退出APP。然后，再次启动APP，看到首页顶部Banner，点击进入活动分会场，浏览过程中再次看到商品A，点击再次查看商品A详情。接着，直接退出到了首页，底部推荐列表中推荐了一篇商品A的用户评论，点击进入，再次查看商品A的详细信息。最后，下定决心，购买了商品A。

数据分析步骤

数据抓取→数据清洗→数据分析→业务决策

产生数据：内部获取、第三方交易、数据爬取

数据清洗：缺失值处理（前后数字不全、平均值等、删除）、数据格式内容等一致性处理、逻辑错误处理（重复值、不合理值）

数据分析：描述性分析（What）、诊断性分析(Why)、预测性分析(Will)、规范性分析(How)

业务决策：通过利用对已发生事件的理解和事件发生的原因，以及各种可能发生的情况分析来帮助确定可采取的最佳行动方案。指导性分析通常是各类分析的组合。

用户画像怎么做，选择什么属性

基础属性（人口统计特征：年龄、性别、教育情况、收入情况）、兴趣属性（浏览内容、收藏内容、阅读咨询、购买物品偏好等）、行为属性（访问时间、浏览路径等用户在网站的行为日志数据）、社交属性（用户社交相关数据）、位置与场景（用户所处城市、所处居住区域、用户移动轨迹等）、消费属性、业务属性、资产属性、设备属性（使用的终端特征）

C++和python的区别

包括但不限于：

python是一种脚本语言，是解释执行的，而C++是编译语言，是需要编译后在特定平台运行的。
python可以很方便的跨平台，但是效率没有C++高。
python使用缩进来区分不同的代码块，C++使用花括号来区分
C++中需要事先定义变量的类型，而python不需要，python的基本数据类型只有数字，布尔值，字符串，列表，元组等等
python的库函数比C++的多，调用起来很方便

深拷贝和浅拷贝的区别（举例说明深拷贝的安全性）

浅拷贝就是将对象的指针进行简单的复制，原对象和副本指向的是相同的资源。
而深拷贝是新开辟一块空间，将原对象的资源复制到新的空间中，并返回该空间的地址。
深拷贝可以避免重复释放和写冲突。例如使用浅拷贝的对象进行释放后，对原对象的释放会导致内存泄漏或程序崩溃。

什么是lambda函数？它有什么好处？

答：lambda 表达式，通常是在需要一个函数，但是又不想费神去命名一个函数的场合下使用，也就是指匿名函数，是一个短小的函数

Python中pass语句的作用是什么？

pass语句不会执行任何操作，一般作为占位符或者创建占位程序，while False: pass

如何用Python来进行查询和替换一个文本字符串？

答：可以使用re模块中的sub()函数或者subn()函数来进行查询和替换，
格式：sub(replacement, string[, count=0]) (replacement是被替换成的文本，string是需要被替换的文本，count是一个可选参数，指最大被替换的数量)

Python里面match()和search()的区别？

答：re模块中match(pattern, string[, flags])，检查string的开头是否与pattern匹配。
re模块中search(pattern, string[, flags])，在string搜索pattern的第一个匹配值。

单引号，双引号，三引号的区别

答：单引号和双引号是等效的，如果要换行，需要符号(\)，三引号则可以直接换行，并且可以包含注释

计算机网络

1.tcp数据包的格式是什么？可选参数有哪些选项

偏移	位0—3	4—7	8—15	16—31
0	来源连接端口			目的连接端口
32	序列号码			
64	确认号码			
96	报头长度	保留	标志符	窗口大小
128	校验和			紧急指针
160	选项字段			
160/192+	数据			

端口号

序列号：代表该packet的数据区的第一个字节在传输中的序列号

确认号：ACK消息；=sequence number + 数据区长度

校验和：计算包括TCP header, IP header 和数据区的值；

窗口：消息发送方接收缓冲区的大小，表名已确认的字节数上限

紧急指针：U置位时，指出紧急消息在数据区中的位置

offset: options的大小

* 标志符

* URG—为1表示高优先级数据包，紧急指针字段有效。

* ACK—为1表示确认号字段有效

* PSH—为1表示是带有PUSH标志的数据，指示接收方应该尽快将这个报文段交给应用层而不用等待缓冲区装满。

* RST—为1表示出现严重差错。可能需要重现创建TCP连接。还可以用于拒绝非法的报文段和拒绝连接请求。

* SYN—为1表示这是连接请求或是连接接受请求，用于创建连接和使顺序号同步

* FIN—为1表示发送方没有数据要传输了，要求释放连接。

2.gRPC用的什么协议? TCP详解

gRPC基于http2.0，默认gRPC使用[protocol buffers](#)，Google成熟开源的序列化结构数据(尽管可以使用其他数据格式，比如JSON)。使用协议缓冲的第一步是在proto file里为数据定义你想序列化的结构：可以是普通的.proto扩展的文本文件。协议缓冲数据结构为messages，每条message是一个小的逻辑记录的信息包含一些name-value对名为fields。

grpc调用。。

tcp三次握手和四次挥手

注意每一个字段状态的建立过程

客户端

CLOSED

服务器

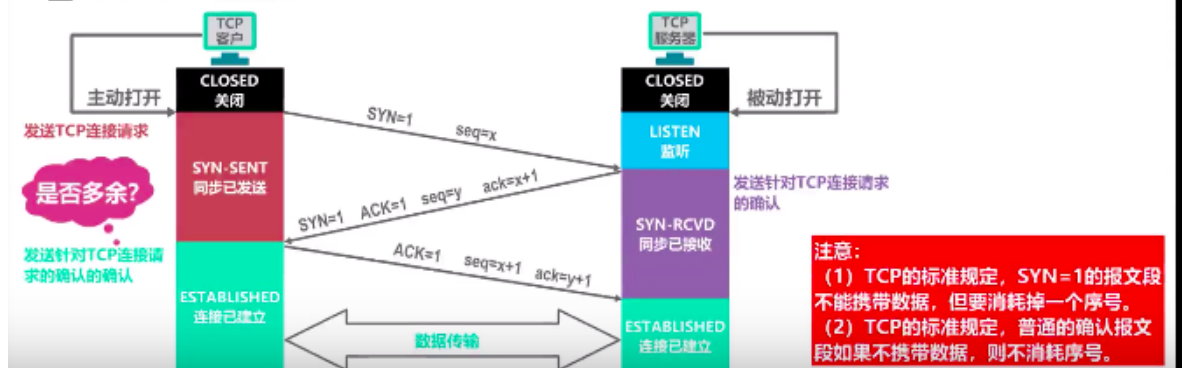
CLOSED

5.8.1 TCP的运输连接管理 —— TCP的连接建立

■ TCP的连接建立要解决以下三个问题：

- ① 使TCP双方能够确知对方的存在；
- ② 使TCP双方能够协商一些参数（如最大窗口值、是否使用窗口扩大选项和时间戳选项以及服务质量等）；
- ③ 使TCP双方能够对运输实体资源（如缓存大小、连接表中的项目等）进行分配。

■ TCP使用“三报文握手”建立连接



刚开始, 客户端和服务端都处于 CLOSE 状态.

此时, 客户端向服务器主动发出连接请求, 服务器被动接受连接请求.

1, TCP服务器进程先创建传输控制块TCB, 时刻准备接受客户端进程的连接请求, 此时服务器就进入了 LISTEN (监听) 状态

2, TCP客户端进程也是先创建传输控制块TCB, 然后向服务器发出连接请求报文, 此时报文首部中的同步标志位SYN=1, 同时选择一个初始序列号 $seq = x$, 此时, TCP客户端进程进入了 SYN-SENT (同步已发送状态) 状态. TCP规定, SYN报文段 (SYN=1的报文段) 不能携带数据, 但需要消耗掉一个序号。

3, TCP服务器收到请求报文后, 如果同意连接, 则发出确认报文。确认报文中的 ACK=1, SYN=1, 确认序号是 $x+1$, 同时也要为自己初始化一个序列号 $seq = y$, 此时, TCP服务器进程进入了 SYN-RCVD (同步收到) 状态。这个报文也不能携带数据, 但是同样要消耗一个序号。

4, TCP客户端进程收到确认后还, 要向服务器给出确认。确认报文的ACK=1, 确认序号是 $y+1$, 自己的序列号是 $x+1$ 。

5, 此时, TCP连接建立, 客户端进入 ESTABLISHED (已建立连接) 状态。当服务器收到客户端的确认后也进入 ESTABLISHED 状态, 此后双方就可以开始通信了。

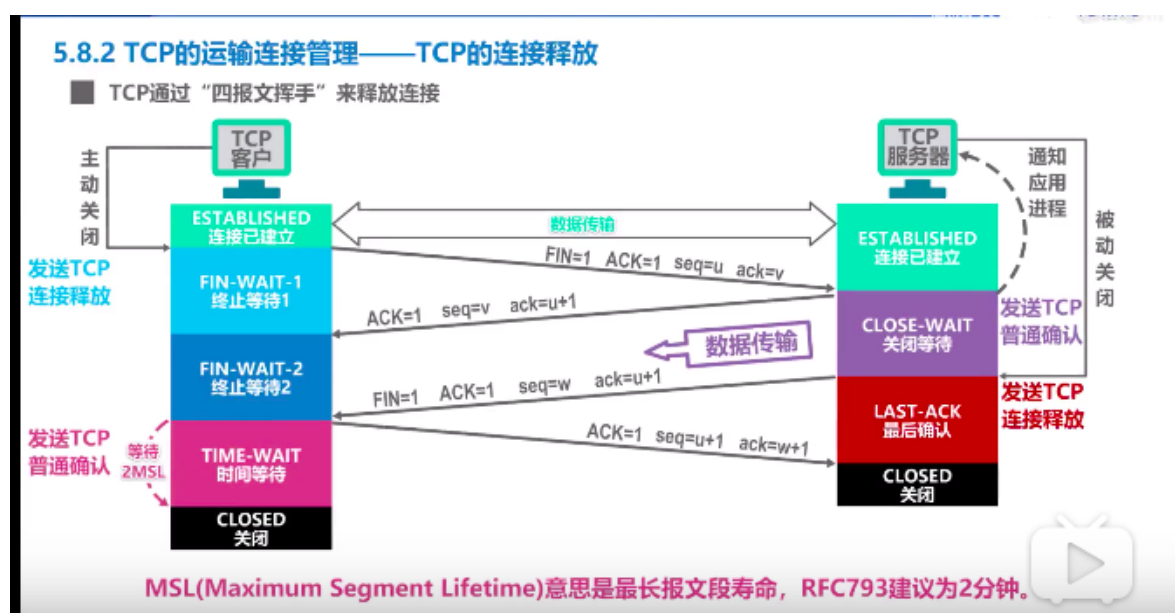
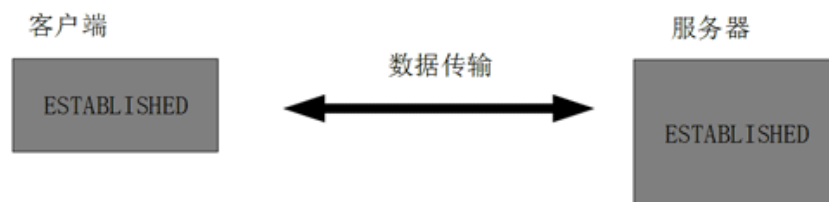
为什么不用两次? 客户端第二次发送response是否多余?

因为tcp是一个双向通信协议, 通信双方都有能力发送数据给对方, 并接收响应; 因此通信双方都需要随机产生一个序列号, 并告诉对方自己能够接收到他序列号之后的数据了; 因此需要三次握手, 如果只是两次握手, 则只能建立一个单方向的通道, 四次握手多余了;

为什么不用四次?

因为三次已经可以满足需要了, 四次就多余了。

四次挥手



此时客户端和服务端都是处于ESTABLISHED状态，然后客户端主动断开连接，服务器被动断开连接。

1, 客户端进程发出连接释放报文，并且停止发送数据。

释放数据报文首部，FIN=1，其序列号为seq=u（等于前面已经传送过来的数据的最后一个字节的序号加1），此时客户端进入FIN-WAIT-1（终止等待1）状态。TCP规定，FIN报文段即使不携带数据，也要消耗一个序号。

2, 服务器收到连接释放报文，发出确认报文，ACK=1，确认序号为u+1，并且带上自己的序列号seq=v，此时服务端就进入了CLOSE-WAIT（关闭等待）状态。

TCP服务器通知高层的应用进程，客户端向服务器的方向就释放了，这时候处于半关闭状态，即客户端已经没有数据要发送了，但是服务器若发送数据，客户端依然要接受。这个状态还要持续一段时间，也就是整个CLOSE-WAIT状态持续的时间。

3, 客户端收到服务器的确认请求后，此时客户端就进入FIN-WAIT-2（终止等待2）状态，等待服务器发送连接释放报文（在这之前还需要接受服务器发送的最终数据）

4, 服务器将最后的数据发送完毕后，就向客户端发送连接释放报文，FIN=1，确认序号为v+1，由于在半关闭状态，服务器很可能又发送了一些数据，假定此时的序列号为seq=w，此时，服务器就进入了LAST-ACK（最后确认）状态，等待客户端的确认。

5, 客户端收到服务器的连接释放报文后，必须发出确认，ACK=1，确认序号为w+1，而自己的序列号是u+1，此时，客户端就进入了TIME-WAIT（时间等待）状态。**注意此时TCP连接还没有释放，必须经过2*MSL（最长报文段寿命）的时间后，当客户端撤销相应的TCB后，才进入CLOSED状态。**

6, 服务器只要收到了客户端发出的确认, 立即进入CLOSED状态。同样, 撤销TCB后, 就结束了这次的TCP连接。可以看到, 服务器结束TCP连接的时间要比客户端早一些。

为什么最后客户端还要等待 $2 * MSL$ 的时间呢?

MSL (Maximum Segment Lifetime) , TCP允许不同的实现可以设置不同的MSL值。

MSL: 最长报文段寿命

第一, 保证客户端发送的最后一个ACK报文能够到达服务器, 因为这个ACK报文可能丢失, 站在服务器的角度来看, 我已经发送了FIN+ACK报文请求断开了, 客户端还没有给我回应, 应该是我发送的请求断开报文它没有收到, 于是服务器又会重新发送一次, 而客户端就能在这个 $2MSL$ 时间段内收到这个重传的报文, 接着给出回应报文, 并且会重启 $2MSL$ 计时器。

第二, 防止类似与“三次握手”中提到的“已经失效的连接请求报文段”出现在本连接中。客户端发送完最后一个确认报文后, 在这个 $2MSL$ 时间中, 就可以使本连接持续的时间内所产生的所有报文段都从网络中消失。这样新的连接中不会出现旧连接的请求报文。

time_wait状态

TCP协议规定,主动关闭连接的一方要处于 `TIME_WAIT` 状态,等待 $2 * MSL$ (maximum segment lifetime)的时间后才能回到CLOSED状态。

之所以不立即关闭, 主要为了让被动关闭方能足够的时间接收到最后的 Ack 包, 如果没有接收到, 被动方就会重新发送 Fin 包, 重新触发主动方发送最后的 Ack 包。这样的话, 就能尽量保证被动关闭方尽快关闭连接了, 毕竟主动关闭方需要承担起主要责任, 所以会有 `TIME_WAIT` 的等待了。

<https://www.liuuv.com/p/289c4599.html>

3.流量控制, 拥塞控制。滑动窗口

滑动窗口:

TCP 利用滑动窗口实现流量控制。流量控制是为了控制发送方发送速率, 保证接收方来得及接收。接收方发送的确认报文中的窗口字段可以用来控制发送方窗口大小, 从而影响发送方的发送速率。将窗口字段设置为 0, 则发送方不能发送数据。

发送窗口只有收到接收端对于本段发送窗口内字节的ACK确认, 才会移动发送窗口的左边界, 接收窗口只有在前面所有的段都确认的情况下, 才会移动左边界。当在前面还有字节未接收, 但收到后面的字节的情况下, 窗口是不会移动的, 并不对后续字节确认, 以此确保对端会对这些数据进行重传

之前学过的TCP报文中, 有一个字段叫Window, 我们也可以叫做AdvertisedWindow, **用于接收方通知发送方自己还有多少缓冲区可以接收数据, 发送方根据接收方的处理能力来发送数据, 不会导致接收方处理不过来, 这便是流量控制。**

拥塞控制四种办法: **慢开始**、**拥塞避免**、**快重传** 和 **快恢复**。

慢启动

TCP连接开始时, cwnd 的值通常被设为一个 MSS 的较小值, 这使得初始发送速率大约为 MSS/RTT 。但对于 TCP 发送方而言, 可用带宽可能远大于 MSS/RTT , 因此 TCP 发送方希望尽可能快地找到可用带宽的数量。

在慢启动状态, cwnd 的值以一个 MSS 开始并且每当传输的报文段首次被确认就增加一个 MSS。这样一来, 每经过一个 RTT, 发送速率就翻倍。TCP 发送速率呈现指数级增长。

每有一个报文被首次确认, 就增加一个MSS。这导致了每一个RTT内cwnd都会翻倍。

- ○ 有下列三种改变状态的情况


```
1 - **如果存在超时指示的丢包事件，将 cwnd 置为 1 并重新开始慢启动过程。还将第二个  
   状态变量的值 ssthresh（慢启动阈值）设置为 cwnd/2。**  
2  
3     > 相当于用ssthresh标记一下可能快要出现超时的位置  
4  
5 - **当 cwnd 的值等于 ssthresh 时，结束慢启动并令 TCP 转移到拥塞避免模式。**  
6  
7 - **如果检测到三个冗余 ACK，TCP 反应相对不那么剧烈，而是将 cwnd 的值减半，再为  
   三个冗余 ACK 加上三个 MSS。再将 ssthresh 的值记录为 cwnd 的一半。最终进入快速  
   恢复状态。**
```

拥塞避免

- 进入拥塞避免说明 `cwnd` 的值是上次遇到拥塞的值的一半，这意味着我们距离拥塞并不远。因此 TCP 不再选择每过一个 RTT 将 `cwnd` 的值翻番，而是每次只将 `cwnd` 的值增加一个 MSS。
 - 注意是一个RTT增加一个MSS。
- 有下列两种改变状态的情况
 - 当出现超时丢包时，与慢启动情况一样，`cwnd` 的值置为一个 MSS，`ssthresh` 的值被更新为 `cwnd` 值的一半。最终进入慢启动状态。
 - 当出现三个冗余 ACK 事件的丢包时，TCP 反应相对不那么剧烈，而是将 `cwnd` 的值减半，再为三个冗余 ACK 加上三个 MSS。再将 `ssthresh` 的值记录为 `cwnd` 的一半。最终进入快速恢复状态。

快速恢复

- 进入快速恢复阶段后，对收到的每个冗余 ACK，`cwnd` 的值增加一个 MSS。最终当丢失报文段的第一个 ACK 到达时，TCP 在降低 `cwnd` 后进入拥塞避免状态。
- 如果出现超时事件，与慢启动和拥塞避免一样，`cwnd` 的值置为一个 MSS，`ssthresh` 的值被更新为 `cwnd` 值的一半。最终进入慢启动状态。

快重传与快恢复：

在 TCP/IP 中，快速重传和快恢复（fast retransmit and recovery, FRR）是一种拥塞控制算法，它能快速恢复丢失的数据包。

没有 FRR，如果数据包丢失了，TCP 将会使用定时器来要求传输暂停。在暂停的这段时间内，没有新的或复制的数据包被发送。有了 FRR，如果接收机接收到一个不按顺序的数据段，它会立即给发送机发送一个重复确认。如果发送机接收到三个重复确认，它会假定确认件指出的数据段丢失了，并立即重传这些丢失的数据段。

有了 FRR，就不会因为重传时要求的暂停被耽误。当有单独的数据包丢失时，快速重传和快恢复（FRR）能最有效地工作。当有多个数据信息包在某一段很短的时间内丢失时，它则不能很有效地工作。

超时重传：数据包丢失或者确认应答丢失（SYN，ACK）超时时间应该略大于包的往返时间

快速重传：以数据驱动重传，3次同样的ACK就会启动重传机制，并重传之前的数据

4.输入url会发生什么

浏览器查找域名的IP地址（dns查找）——浏览器向web服务器发出一个http请求（tcp连接，http报文，IP协议，OSPF协议（IP数据包在路由器之间转发的路由选择），同时cookies也会随着浏览器的http请求一起发过来）——服务器处理请求——服务器返回一个html响应——浏览器开始显示html

1. DNS 解析：浏览器查询 DNS，获取域名对应的 IP 地址：具体过程包括**浏览器搜索自身的 DNS 缓存、搜索操作系统的 DNS 缓存、读取本地的 Host 文件和向本地 DNS 服务器进行查询等**。对于向本地 DNS 服务器进行查询，如果要查询的域名包含在本地配置区域资源中，则返回解析结果给客户端，完成域名解析(此解析具有权威性)；如果要查询的域名不由本地 DNS 服务器区域解析，

但该服务器已缓存了此网址映射关系，则调用这个 IP 地址映射，完成域名解析（此解析不具有权威性）。如果本地域名服务器并未缓存该网址映射关系，那么将根据其设置发起**递归查询或者迭代查询**；

2. TCP 连接：浏览器获得域名对应的 IP 地址以后，浏览器向服务器请求建立链接，发起三次握手；
3. 发送 HTTP 请求：TCP 连接建立起来后，浏览器向服务器发送 HTTP 请求；
4. 服务器处理请求并返回 HTTP 报文：服务器接收到这个请求，并根据路径参数映射到特定的请求处理器进行处理，并将处理结果及相应的视图返回给浏览器；
5. 浏览器解析渲染页面：浏览器解析并渲染视图，若遇到对 js 文件、css 文件及图片等静态资源的引用，则重复上述步骤并向服务器请求这些资源；浏览器根据其请求到的资源、数据渲染页面，最终向用户呈现一个完整的页面。
6. 连接结束。

5.dns解析过程

名称类型	说 明	示 例
根域	DNS域名中使用时，规定由尾部句点(.)来指定名称位于根或更高级别的域层次结构	单个句点(.)或句点用于末尾的名称
顶级域	用来指示某个国家/地区或组织使用的名称的类型名称	.com
第二层域	个人或组织在 Internet 上使用的注册名称	qq.com
子域	已注册的二级域名派生的域名，通俗的讲就是网站名	www.qq.com
主机名	通常情况下，DNS 域名的最左侧的标签标识网络上的特定计算机，如hl	hl.www.qq.com Blog

当 DNS 客户机需要查询程序中使用的名称时，它会查询本地DNS 服务器来解析该名称。客户机发送的每条查询消息都包括3条信息，以指定服务器应回答的问题。

- 指定的 DNS 域名，表示为完全合格的域名 (FQDN)。
- 指定的查询类型，它可根据类型指定资源记录，或作为查询操作的专门类型。
- DNS域名的指定类别。

对于DNS 服务器，它始终应指定为 Internet 类别。例如，指定的名称可以是计算机的完全合格的域名，如im.qq.com，并且指定的查询类型用于通过该名称搜索地址资源记录。

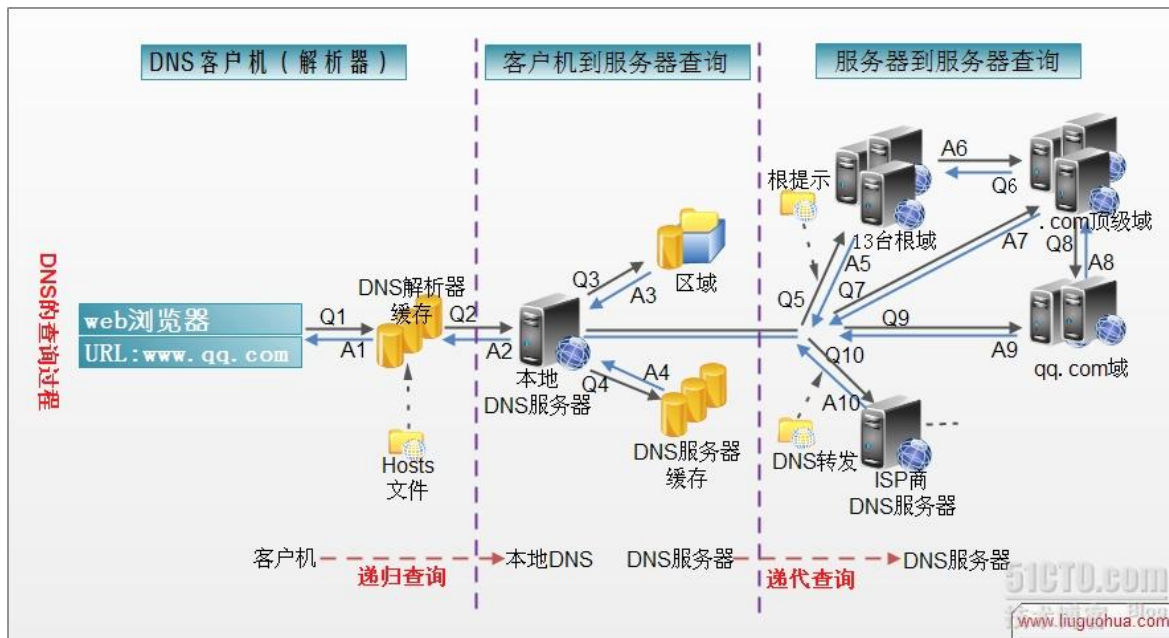
DNS 查询以各种不同的方式进行解析。客户机有时也可通过使用从以前查询获得的缓存信息就地应答查询。DNS 服务器可使用其自身的资源记录信息缓存来应答查询，也可代表请求客户机来查询或联系其他 DNS 服务器，以完全解析该名称，并随后将应答返回至客户机。这个过程称为递归。

另外，客户机自己也可尝试联系其他的 DNS 服务器来解析名称。如果客户机这么做，它会使用基于服务器应答的独立和附加的查询，该过程称作迭代，即DNS服务器之间的交互查询就是迭代查询。

请求过程

- 首先搜索浏览器的 DNS 缓存，缓存中维护一张域名与 IP 地址的对应表
- 若没有命中，则继续搜索操作系统的 DNS 缓存
- 若仍然没有命中，则操作系统将域名发送至本地域名服务器，本地域名服务器采用递归查询自己的 DNS 缓存，查找成功则返回结果
- 若本地域名服务器的 DNS 缓存没有命中，则本地域名服务器向上级域名服务器进行迭代查询
- - 首先本地域名服务器向根域名服务器发起请求，根域名服务器返回顶级域名服务器的地址给本地服务器

- 本地域名服务器拿到这个顶级域名服务器的地址后，就向其发起请求，获取权限域名服务器的地址
- 本地域名服务器根据权限域名服务器的地址向其发起请求，最终得到该域名对应的 IP 地址
- 本地域名服务器将得到的 IP 地址返回给操作系统，同时自己将 IP 地址缓存起来
- 操作系统将 IP 地址返回给浏览器，同时自己也将 IP 地址缓存起来
- 至此，浏览器就得到了域名对应的 IP 地址，并将 IP 地址缓存起来



6.为啥要IP还要mac

计算机网络常考

IP：是网络层及以上使用的地址

MAC：是数据链路层和物理层使用的地址

连接在通信链路中的设备（主机或**路由器**）在收到MAC帧时，根据MAC帧的首部中的硬件地址决定收下或丢弃。只有在剥去MAC帧的首部和尾部后把数据上传给网络层后，网络层在IP数据报的首部中找到源IP地址和目的IP地址。

路由器WAN口和LAN口各有一个**MAC地址**，WAN口MAC地址是对外通信的，LAN口地址是对内通信的。标准路由器上，每个端口各有一个自己的MAC地址，以进行各网段的通信。

由ARP决定在mac中填入什么样的IP地址

7.可不可以没有IP层

ip层主要常用协议：

- ARP:根据IP地址获取对应的MAC地址，通过一问一答实现交互，问是通过广播形式实现，答是通过单播形式。
- ICMP:在ip网络设备之间发送控制报文，传递差错、控制、查询等信息。可根据ICMP查看要发送的数据是否成功送达。若未成功送达的话，可查看发送失败的原因
- DHCP:给主机动态的分配ip地址，无需人为参与
- IP协议:寻址和路由

8.可不可以没有TCP传输层

要注意的是，IP不是可靠的协议，这是说，IP协议没有提供一种数据未传达以后的处理机制，这被认为是上层协议：TCP或UDP要做的事情。

TCP和UDP区别

- 1、TCP面向连接（如打电话要先拨号建立连接）；UDP是无连接的，即发送数据之前不需要建立连接
- 2、TCP提供可靠的服务。也就是说，通过TCP连接传送的数据，无差错，不丢失，不重复，且按序到达；UDP尽最大努力交付，即不保证可靠交付
- 3、TCP面向字节流，实际上是TCP把数据看成一连串无结构的字节流；UDP是面向报文的
UDP没有拥塞控制，因此网络出现拥塞不会使源主机的发送速率降低（对实时应用很有用，如IP电话，实时视频会议等）
- 4、每一条TCP连接只能是点到点的；UDP支持一对一，一对多，多对一和多对多的交互通信
- 5、TCP首部开销20字节；UDP的首部开销小，只有8个字节
- 6、TCP的逻辑通信信道是全双工的可靠信道，UDP则是不可靠信道

9.TCP保证传输的可靠性

校验和

序列号

确认应答：对应不同的tcp数据包会返回不同的ACK报文

拥塞控制：慢开始；拥塞避免；快重传与快恢复

流量控制：滑动窗口用来控制发送方发送数据速率

超时重传：超过一定等待时间则会认为另一端故障放弃传送

连接管理：三次握手四次挥手

确认应答

10.TCP粘包

TCP 是面向字节流的传输，它会根据接收方的包处理能力以及当前网络的拥塞情况来一部分一部分的加载数据发送，再加上有 Nagle 这种整合小数据包的算法存在。所以对于接收方来讲，接收到的数据有可能是粘合在一起的，也有可能是被拆开的，即所谓的粘包和拆包。

多个数据包被连续存储于连续的缓存中，在**对数据包进行读取时**由于**无法确定发送方的发送边界**，而**采用某一估测值大小**来进行数据读出，若**双方的size不一致**时就会使指发送方发送的若干包数据到接收方接收时粘成一包，从接收缓冲区看，后一包数据的头紧接着前一包数据的尾。

对于粘包和拆包现象，常用的解决方案有：

- 在包的首部添加当前要传输的数据包的长度，让接收方根据长度去切割。
- 将数据包封装为固定长度，不够的补 0，让接收方按固定长度解析。
- 人为的给数据包添加边界，比如在数据包结尾添加特殊字符，当解析到特殊字符时，接收方就认为读取到了一个完整有意义的数据段了。

UDP没有粘包：因为UDP有消息保护边界，不会发生粘包拆包问题

11.如何设计可靠的udp传输

在应用层设计可靠性机制

<https://www.jianshu.com/p/6c73a4585eba>

传输层无法保证数据的可靠传输，只能通过应用层来实现了。实现的方式可以参照tcp可靠性传输的方式，只是实现不在传输层，实现转移到了应用层。

最简单的方式是在应用层模仿传输层TCP的可靠性传输。下面不考虑拥塞处理，可靠UDP的简单设计。

- 1、添加seq/ack机制，确保数据发送到对端

- 2、添加发送和接收缓冲区，主要是用户超时重传。
- 3、添加超时重传机制。

详细说明：送端发送数据时，生成一个随机seq=x，然后每一片按照数据大小分配seq。数据到达接收端后接收端放入缓存，并发送一个ack=x的包，表示对方已经收到了数据。发送端收到了ack包后，删除缓冲区对应的数据。时间到后，定时任务检查是否需要重传数据。

常见的例子：RUDP,RTP,UDT

12.计算机网络的各层及数据结构



物理层：提供传输数据的物理通路，bit

数据链路层：建立相邻节点数据链路传输，数据组帧，差错检测；frame；主要设备交换机

网络层：基于IP地址的路由选择传输数据；拥塞控制，差错检测与恢复；packet，主要设备路由器

传输层：数据传递，面向连接或无连接；segment数据段

常见的应用层协议

DNS、TELNET（远程终端协议）、http（超文本传输协议）、ftp、smtp（电子邮件协议）、POP3（邮件读取）

13.http与https的区别

http是一个无状态协议，服务器不维护任何有关客户端过去所发请求的消息。TCP连接，默认端口为80.

https 是基于 HTTP 的，也是用 TCP 作为底层协议，并额外使用 SSL/TLS 协议用作加密和安全认证。默认端口号是 443.

核心要素是非对称加密

公钥私钥加密publickey，然后消息传输使用public key

CA 会给各个服务器颁发证书，证书存储在服务器上，并附有 CA 的**电子签名**（见下节）。

当客户端（浏览器）向服务器发送 HTTPS 请求时，一定要先获取目标服务器的证书，并根据证书上的信息，检验证书的合法性。

ssl/tls协议

SSL (Secure Sockets Layer 安全套接层), 及其继任者传输层安全 (Transport Layer Security, TLS) 是为网络通信提供安全及数据完整性的一种安全协议。TLS 与 SSL 在传输层对网络连接进行加密。

SSL 协议位于 TCP/IP 协议与各种应用层协议之间, 为数据通讯提供安全支持。SSL 协议可分为两层: SSL 记录协议 (SSL Record Protocol): 它建立在可靠的传输协议 (如 TCP) 之上, 为高层协议提供数据封装、压缩、加密等基本功能的支持。

安全传输层协议 (TLS) 用于在两个通信应用程序之间提供保密性和数据完整性。该协议由两层组成: TLS 记录协议 (TLS Record) 和 TLS 握手协议 (TLS Handshake)。较低的层为 TLS 记录协议, 位于某个可靠的传输协议 (例如 TCP) 上面, 与具体的应用无关, 所以, 一般把 TLS 协议归为传输层安全协议。

14.HTTP1.0, 1.1, 2.0, 3.0

1) http1.0

引入了新的命令POST和HEAD (http数据头部) 命令

每个TCP连接只能发送一个请求, 发送数据完毕, 连接就关闭, 如果还要请求其他资源, 就必须再新建一个连接

2) http1.1 新增方法: PUT、PATCH、OPTIONS、DELETE

引入了持久连接 (persistent connection), 即TCP连接默认不关闭, 可以被多个请求复用, 不用声明 Connection: keep-alive。对于同一个域名, 大多数浏览器允许同时建立6个持久连接引入了管道机制, 即在同一个TCP连接里, 客户端可以同时发送多个请求, 进一步改进了HTTP协议的效率

3) http2.0

1.头信息和数据体都是二进制, 称为头信息帧和数据帧

2.复用TCP连接, 在一个连接里, 客户端和浏览器都可以同时发送多个请求或回应, 且不用按顺序——对应, 避免了“队头堵塞”,此双向的实时通信称为多工 (Multiplexing)

3.引入头信息压缩机制 (header compression),头信息使用gzip或compress压缩后再发送; 客户端和服务器同时维护一张头信息表, 所有字段都会存入这个表, 生成一个索引号, 不发送同样字段, 只发送索引号, 提高速度

4.HTTP/2 允许服务器未经请求, 主动向客户端发送资源, 即服务器推送 (server push)

4) http3.0 HTTP/3.0

基于UDP实现, 实现了类似TCP的多路数据流、传输可靠性等功能, 称为QUIC(Quick UDP Internet Connection)协议

实现了类似TCP的流量控制、传输可靠性的功能

2、集成了TLS(Transport Layer Security 传输层安全性协议)加密功能

3、实现了HTTP/2.0中多路复用

不同点是QUIC实现了在同一物理连接上可以有多个独立的逻辑数据流, 实现了数据流的单独传输, 解决了TCP中队头阻塞问题

4、实现了快速握手功能

QUIC是基于UDP的, 所以QUIC可以实现使用0-RTT和1-RTT来建立连接

content_length字段作用

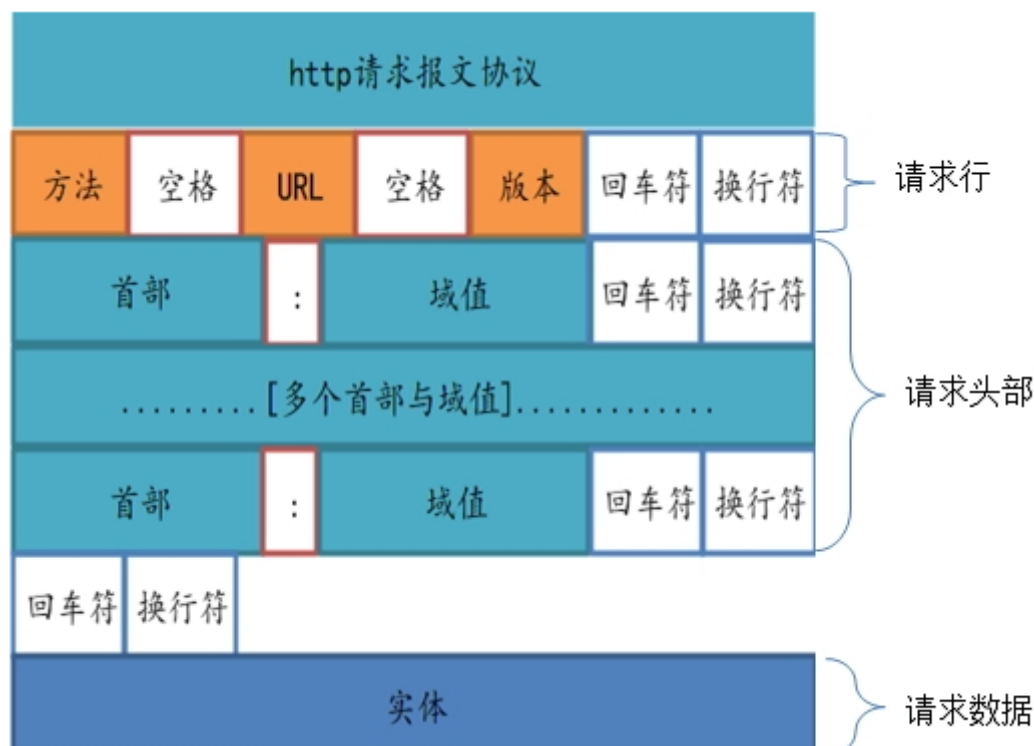
Content-Length如果存在并且有效的话, 则必须和消息内容的传输长度完全一致。否则就会导致异常 (特别地, HTTP1.0中这个字段可有可无)

content_length = 实际长度：正常加载网页

content_length < 实际长度：在http的响应体中被截去其他的部分了

content_length > 实际长度：网页无法正常运行，err_content_length_mismatch，localhost意外终止了连接

http报文



请求行（HTTP请求报文的最后一行）

请求行由方法字段、URL字段和HTTP协议版本字段。其中，方法字段严格区分大小写，当前HTTP协议中的方法都是大写，方法字段如下介绍如下：

方法字段

①GET:请求获取Request-URI(URI:通用资源标识符,URL是其子集，URI注重的是标识，而URL强调的是位置，可以将URL看成原始的URI),所标识的资源

②POST：在Request-URI所标识的资源后附加新的数据；支持HTML表单提交，表单中有用户添入的数据，这些数据会发送到服务器端，由服务器存储至某位置（例如发送处理程序）

③HEAD:请求Request-URI所标识的资源响应消息报头，HEAD方法可以在响应时不返回消息体。

④PUT：与GET相反，请求服务器存储一个资源，并用Request-URI做为其标识；例如发布系统。

⑤DELETE：请求删除URL指向的资源

⑥OPTIONS：请求查询服务器的性能，或者查询与资源相关的选项

⑦TRACE：跟踪请求要经过的防火墙、代理或网关等，主要用于测试或诊断

⑧CONNECT保留将来使用

• URL

一个完整的包括类型、主机名和可选路径名的统一资源引用名，如：<http://www.example.com/path/to/file.html>

请求头部：位于请求行的下面

请求报文中常见的标头有：

Connetion标头（连接管理）、Host标头（指定请求资源的主机）、Range标头（请求实体的字节范围）、User-Agent标头（包含发出请求的用户信息）、Accept标头（首选的媒体类型）、Accept-Language(首选的自然语言)

HTTP首部：

通用首部：请求和响应都可以使用的；

Connection：定义C/S之间关于请求/响应的有关选项

对于http/1.0, Connection: keep-alive

Via: 显示了报文经过的中间节点

Cache-Control: 缓存指示

实体首部：用于指定实体属性

实体主体用于POST方法中。用户向Web服务器提交表单数据的时候，需要使用POST方法，此时主体中包含用户添写在表单的各个属性字段的值，当Web服务器收到POST方法的HTTP请求报文后，可以从实体中取出需要的属性字段的值。

也就是说，当用户通过Web浏览器向Web服务器发送请求时,Web浏览器会根据用户的具体请求来选择不同的HTTP请求方法，再将相应的URL和HTTP协议版本及相关的标头填入头部行中，若是POST方法，还会将相关的表单数据填入实体主体中，产生一个HTTP请求报文，然后将这个报文发送给Web服务器。

Location: 资源的新位置

Allow: 允许对此资源使用的请求方法

1、内容首部：

Content-Encoding：支持的编码

Content-Language：支持的自然语言

Content-Length：文本长度

Content-Location：资源所在位置

Content-Range：在整个资源中此实体表示的字节范围

Content-Type：主体的对象类型

2、缓存首部：

ETag: 实体标签

Expires: 过期期限

Last-Modified: 上一次的修改时间

15.http状态码

1xx：表示通知信息，意思是接收到了，或者是正在处理；100、客户端应当继续发送请求；101、服务器已经理解了客户端的请求，并将通过Upgrade 消息头通知客户端采用不同的协议来完成这个请求

2xx：表示成功或者已经知道了；200、服务器已经将相应的页面返回；201、服务器已经接受了相应的请求，并创建了相应需要的资源；202、服务器已经接受了相应请求但是还未处理

3xx：表示重定向，如需完成当前请求还需要进一步操作；300、（多重选择）被请求的资源有一系列可供选择的回馈信息，每个都有自己特定的地址和浏览器驱动的商业信息。用户或浏览器能够自行选择一个首选的地址进行重定向；301、（永久移动）被请求的资源已永久移动到新位置，并且将来任何对此资源的引用都应该使用本响应返回的若干个 URI 之一；302、（临时移动）请求的资源现在临时从不同的 URI 响应请求。由于这样的重定向是临时的，客户端应当继续向原有地址发送以后的请求；305、使用代理

4xx：表示客户端发生错误，如客户端请求语法错误等

401：未授权，当前请求需要用户验证

403：拒绝服务

404：未找到资源

400 （请求错误）服务器不能够理解请求的语法。错误可能是：(1)语义有误，当前请求无法被服务器理解。除非进行修改，否则客户端不应该重复提交这个请求。(2)请求参数有误。

401 （未授权）当前请求需要用户验证。该响应必须包含一个适用于被请求资源的信息头用以询问用户信息。客户端可以重复提交一个包含恰当的 **Authorization** 头信息的请求。如果当前请求已经包含了 **Authorization** 证书，那么401响应代表着服务器验证已经拒绝了那些证书。如果401响应包含了与前一个响应相同的身份验证询问，且浏览器已经至少尝试了一次验证，那么浏览器应当向用户展示响应中包含的实体信息，因为这个实体信息中可能包含了相关诊断信息。

403 （拒绝服务）服务器已经理解请求，但是拒绝执行它。与401响应不同的是，身份验证并不能提供任何帮助，而且这个请求也不应该被重复提交。如果这不是一个 **HEAD** 请求，而且服务器希望能够讲清楚为何请求不能被执行，那么就应该在实体内描述拒绝的原因。当然服务器也可以返回一个404响应，假如它不希望让客户端获得任何信息

404 （未找到）请求失败，请求所希望得到的资源未被在服务器上发现。没有信息能够告诉用户这个状况到底是暂时的还是永久的。假如服务器知道情况的话，应当使用410状态码来告知旧资源因为某些内部的配置机制问题，已经永久的不可用，而且没有任何可以跳转的地址。404这个状态码被广泛应用于当服务器不想揭示到底为何请求被拒绝或者没有其他适合的响应可用的情况下。

405 （方法禁用）请求行中指定的请求方法不能被用于请求相应的资源。该响应必须返回一个 **Allow** 头信息用以表示出当前资源能够接受的请求方法的列表。 鉴于 **PUT**，**DELETE** 方法会对服务器上的资源进行写操作，因而绝大部分的网页服务器都不支持或者在默认配置下不允许上述请求方法，对于此类请求均会返回405错误。

406 （不接受）请求的资源的内容特性无法满足请求头中的条件，因而无法生成响应实体。除非这是一个 **HEAD** 请求，否则该响应就应当返回一个包含可以让用户或者浏览器从中选择最合适的实体特性以及地址列表的实体。实体的格式由 **Content-Type** 头中定义的媒体类型决定。浏览器可以根据格式及自身能力自行作出最佳选择。但是，规范中并没有定义任何作出此类自动选择的标准。

407 （需要代理授权）与401响应类似，只不过客户端必须在代理服务器上身份验证。代理服务器必须返回一个 **Proxy-Authenticate** 用以进行身份询问。客户端可以返回一个 **Proxy-Authorization** 信息头用以验证。

408 （请求超时）请求超时。客户端没有在服务器预备等待的时间内完成一个请求的发送。客户端可以随时再次提交这一请求而无需进行任何更改。

409 （冲突）由于和被请求的资源的当前状态之间存在冲突，请求无法完成。这个代码只允许在这样的情况下才能被使用：用户被认为能够解决冲突，并且会重新提交新的请求。该响应应当包含足够的信息以使用户发现冲突的源头。冲突通常发生于对 **PUT** 请求的处理中。例如，在采用版本检查的环境下，某次 **PUT** 提交的对特定资源的修改请求所附带的版本信息与之前的某个（第三方）请求向冲突，那么此时服务器就应该返回一个409错误，告知用户请求无法完成。此时，响应实体中很可能会包含两个冲突版本之间的差异比较，以使用户重新提交归并以后的新版本。

5xx：服务器错误

502：错误网关

503：服务不可用，临时的服务器过载导致服务器当前无法处理请求

500 (服务器内部出错) 服务器遇到了一个未曾预料的状态, 导致了它无法完成对请求的处理。一般来说, 这个问题都会在服务器的程序码出错时出现。

501 (尚未实施) 服务器不支持当前请求所需要的某个功能。当服务器无法识别请求的方法, 并且无法支持其对任何资源的请求。

502 (错误网关) 作为网关或者代理工作的服务器尝试执行请求时, 从上游服务器接收到无效的响应。

503 (服务不可用) 由于临时的服务器维护或者过载, 服务器当前无法处理请求。这个状态是临时的, 并且将在一段时间以后恢复。如果能够预计延迟时间, 那么响应中可以包含一个 `Retry-After` 头用以标明这个延迟时间。如果没有给出这个 `Retry-After` 信息, 那么客户端应当以处理500响应的方式处理它。注意: 503状态码的存在并不意味着服务器在过载的时候必须使用它。某些服务器只不过是希望拒绝客户端的连接。

504 (网关超时) 作为网关或者代理工作的服务器尝试执行请求时, 未能及时从上游服务器 (URI标识出的服务器, 例如 HTTP、FTP、LDAP) 或者辅助服务器 (例如DNS) 收到响应。注意: 某些代理服务器在DNS查询超时时会返回400或者500错误

505 (HTTP版本不支持) 服务器不支持, 或者拒绝支持在请求中使用的 HTTP 版本。这暗示着服务器不能或不愿使用与客户端相同的版本。响应中应当包含一个描述了为何版本不被支持以及服务器支持哪些协议的实体。

16.http五种方法

get: 从服务器获取数据, 请求数据放在url中

post: 向服务器提交数据, 数据存放在body中, POST 请求可能会导致新的资源的建立和/或已有资源的修改。

put: 向服务器提交数据, 发送一个修改数据的请求, 需求数据全部更新

delete: 向服务器端提交数据, 请求数据放在报文body中, 删除数据

patch: 向服务器端提交数据, 请求数据放在body中, 修改数据, 需求数据部分更新, 局部更新

17.了解restful吗

Restful风格指的是网络应用中就是资源定位和资源操作的风格。不是标准也不是协议。

Rest即Representational State Transfer的缩写, 可译为"表现层状态转化"。Restful风格最大的特点为: 资源、统一接口、URI和无状态。

资源: 所有的互联网上的事物都可以被抽象为资源, 并被http方法操作

统一接口: http动作

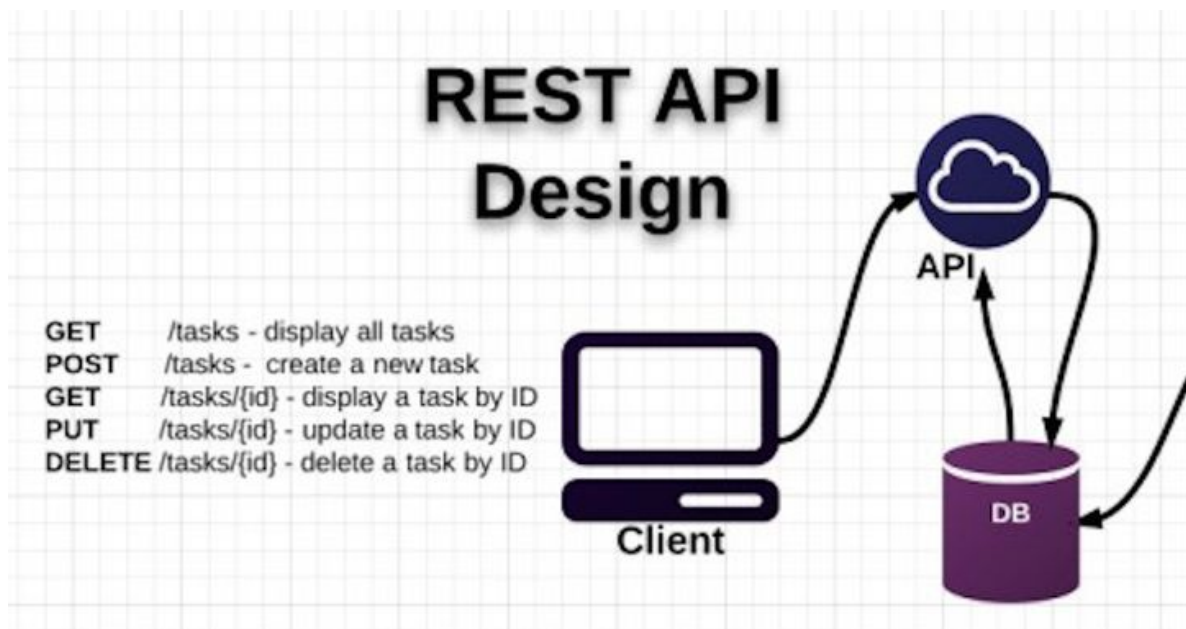
URI: 每一个URI (统一资源定位符) 指向一个特定的资源。通过URI来访问资源。最典型的URI就是URL。

URI定位资源, http动词描述对资源的操作

无状态:

所有的资源, 都可以通过URI定位, 而且这个定位与其他资源无关。例如无需登录就可以通过URL查看, 就是无状态。需要登录才能查看, 是有状态。

资源在http动作的操作下发生的状态转移变化



Server和Client之间传递某资源的一个表现形式，比如用JSON，XML传输文本，或者用JPG，WebP传输图片等。当然还可以压缩HTTP传输时的数据（on-wire data compression）。

用 HTTP Status Code传递Server的状态信息。比如最常用的 200 表示成功，500 表示Server内部错误等。

rest的几种method

get

put

post

delete

HTTP协议中并无PUT和DELETE的操作因此需在web.xml添加HiddenHttpMethodFilter，用于识别PUT和DELETE操作

原理是：从POST操作中额外分出PUT和DELETE操作

优点

使请求路径变得更加简洁

传递、获取参数值更加方便，框架会自动进行类型转换

通过路径变量@PathVariable的类型，可以约束访问参数。

若参数值与定义类型不匹配，则访问不到对应的方法，报错400错误的请求。

安全，请求路径中直接传递参数值，并用斜线/分隔，不会暴露传递给方法的参数变量名。

高效，更易于缓存的实现，让响应更加高效。

18.cookie和session

1) cookie

在浏览某些网站时,这些**网站会把一些数据存在客户端,用于使用网站等跟踪用户,实现用户自定义功能**.比如你登录以后，服务器设置了 cookie用户名(username),那么，当你再次请求服务器的时候，浏览器会将username一块发送给服务器，这些变量有一定的特殊标记。

服务器会解释为 cookie变量。

所以只要不关闭浏览器，那么 cookie变量便一直是有效的，所以能够保证长时间不掉线。

一般保存在内存,不在硬盘;

如果**设置了过期时间**,浏览器会把cookie保存在硬盘上,关闭再打开浏览器,这些cookie依然有效直到超过的设置过期时间;

2) session

Session是存放在服务器端的类似于HashTable结构; **实现网页之间数据传递, 是一个存储在服务器端的对象集合**。每个客户端用户访问时, 服务器都为每个用户分配一个唯一的会话ID (Session ID) . 她的生存期是用户持续请求时间再加上一段时间(一般是20分钟左右).Session中的信息保存在Web服务器内容中,保存的数据量可大可小.当 Session超时或被关闭时将自动释放保存的数据信息.

session工作原理

- (1)当有Session启动时, 服务器生成一个唯一值, 称为Session ID (好像是通过取进程ID的方式取得的)。
- (2)然后, 服务器开辟一块内存, 对应于该Session ID。
- (3)服务器再将该Session ID写入浏览器的cookie。
- (4)服务器内有一进程, 监视所有Session的活动状况, 如果有Session超时或是主动关闭, 服务器就释放改内存块。
- (5)当浏览器连入IIS时并请求的ASP内用到Session时, IIS就读浏览器Cookie中的Session ID。
- (6)然后, 服务检查该Session ID所对应的内存是否有效。
- (7)如果有效, 就读出内存中的值。
- (8)如果无效, 就建立新的Session。

cookie和session的区别

1.cookie存放在客户的浏览器上, session存放在服务器上简单的说, 当你登录一个网站的时候, 如果web服务器端使用的是session,那么所有的数据都保存在服务器上面,

客户端每次请求服务器的时候会发送 当前会话的session_id, 服务器根据当前session_id判断相应的用户数据标志, 以确定用户是否登录, 或具有某种权限。

2.cookie不是很安全, 别人可以分析存放在本地的COOKIE并进行COOKIE欺骗考虑到安全应当使用session。

3.session会在一定时间内保存在服务器上。当访问增多, 会比较占用你服务器的性能考虑到减轻服务器性能方面, 应当使用COOKIE。

4.单个cookie保存的数据不能超过4K, 很多浏览器都限制一个站点最多保存20个cookie。(Session对象没有对存储的数据量的限制, 其中可以保存更为复杂的数据类型)

19.对称加密和非对称加密

对称加密使用相同的key, 非对称加密使用不同的key, 它使用了一对密钥, 公钥 (public key) 和私钥 (private key)。私钥只能由一方安全保管, 不能外泄, 而公钥则可以发给任何请求它的人。非对称加密使用这对密钥中的一个进行加密, 而解密则需要另一个密钥。(RSA算法)

同时具备安全性和加解密快速性的解决的办法是将对称加密的密钥使用非对称加密的公钥进行加密, 然后发送出去, 接收方使用私钥进行解密得到对称加密的密钥, 然后双方可以使用对称加密来进行沟通。

20.HTTPS的CA证书工作原理

WEB服务存在http和https两种通信方式, http默认采用80作为通讯端口, 对于传输采用不加密的方式, https默认采用443, 对于传输的数据进行加密传输

https=http+ssl，顾名思义，https是在http的基础上加上了SSL保护壳，信息的加密过程就是在SSL中完成的

http通信原理

客户端发送一句client hello给服务器端，服务器端返回一句serverhello给客户端，鉴于本文讨论是https的加密主题，我们只讨论信息传输的加密问题

实现客户端和服务端发送的信息client hello 和server hello，即使中间的包被窃取了，也无法解密传输的内容



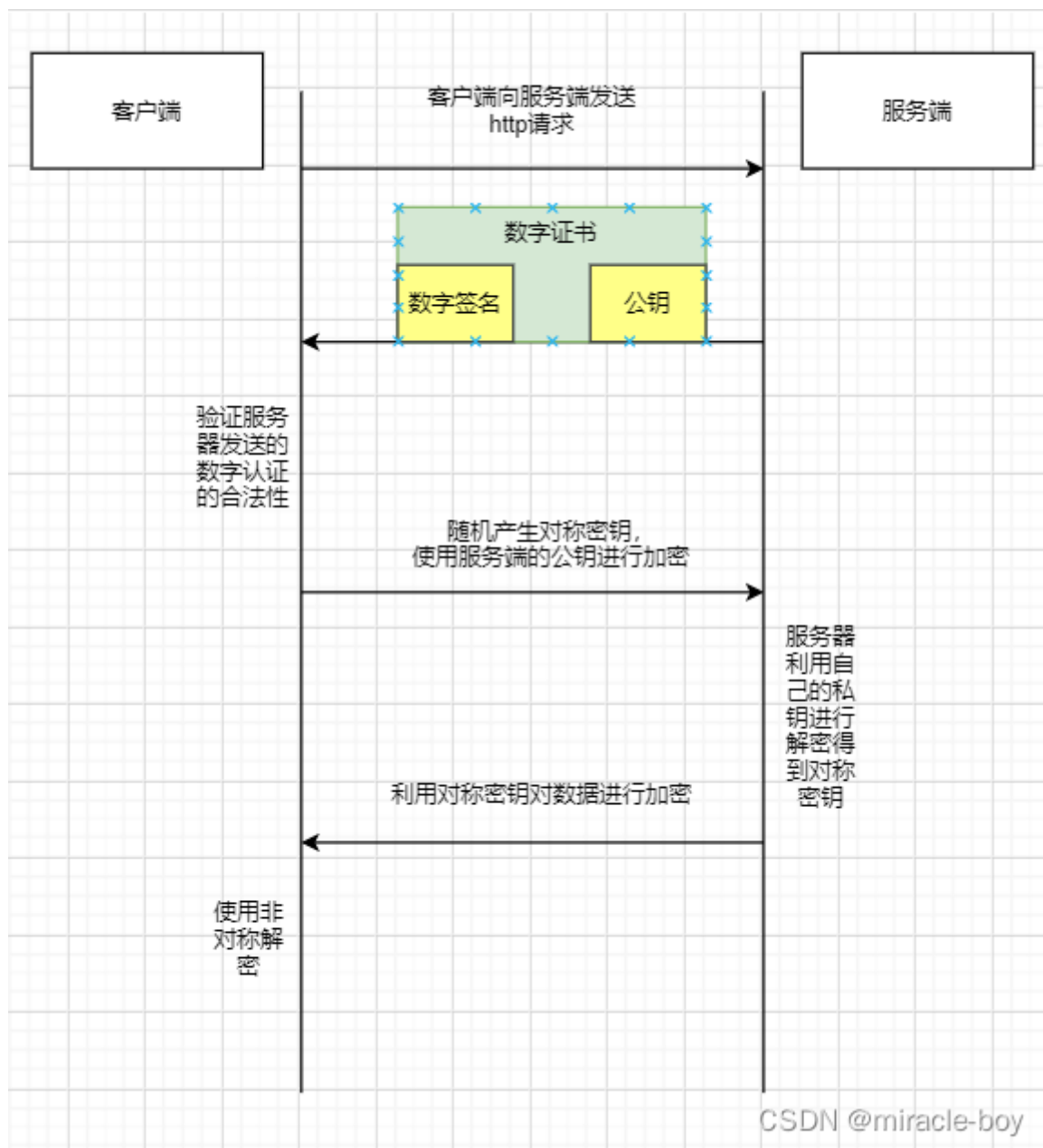
可以看到，http的信息传输中，信息很容易被xxx给劫持，更有甚者，xxx可以伪装服务器将篡改后的信息返回给用户，试想一下，如果xxx劫持的是你的银行信息，是不是很可怕。所以对于http传出存在的问题可以总结如下：

- (1) 信息篡改：修改通信的内容
- (2) 信息劫持：拦截到信息通信的内容

这些是http不安全的体现

HTTPS的通信步骤

- 客户端发送报文给服务器开始进行SSL通信。报文中包含客户端支持的SSL指定版本、客户端生成的随机数、加密组件(加密算法以及密钥长度)。
- 服务器发送响应报文给客户端。和客户端一样，报文中包含SSL版本、服务器生成的随机数、加密组件(从接收到的客户端加密组件内筛选出来的)。
- 服务器把公钥证书发送给客户端，自己保存着私钥。服务器把自己的公钥登陆至数字证书认证机构，数字证书认证机构用自己的私钥向服务器的公钥署数字签名并颁发公钥证书。
- 服务器发送报文告知客户端最初阶段的SSL握手协商部分结束。
- 客户端用服务器的公钥对生成的随机数加密后发送给服务器。客户端拿到服务器的公钥证书后，使用数字证书认证机构的公钥向数字证书认证机构验证公钥证书的数字签名，以确认服务器的公钥的真实性，然后生成一个新的随机数，用公钥加密这个随机数发送给服务器。
- 服务器用私钥解密后获得客户端发来的随机数。
- 客户端和服务端根据约定的加密方法，使用前面三个随机数生成对话密钥，用来加密接下来的通信过程



SSL过程

SSL 握手有三个目的。第一，客户端与服务器需要就一组用于保护数据的算法达成一致。第二，它们需要确立一组由那些算法所使用的加密密钥。第三，握手还可以选择对客户端进行认证。整个过程工作如下：

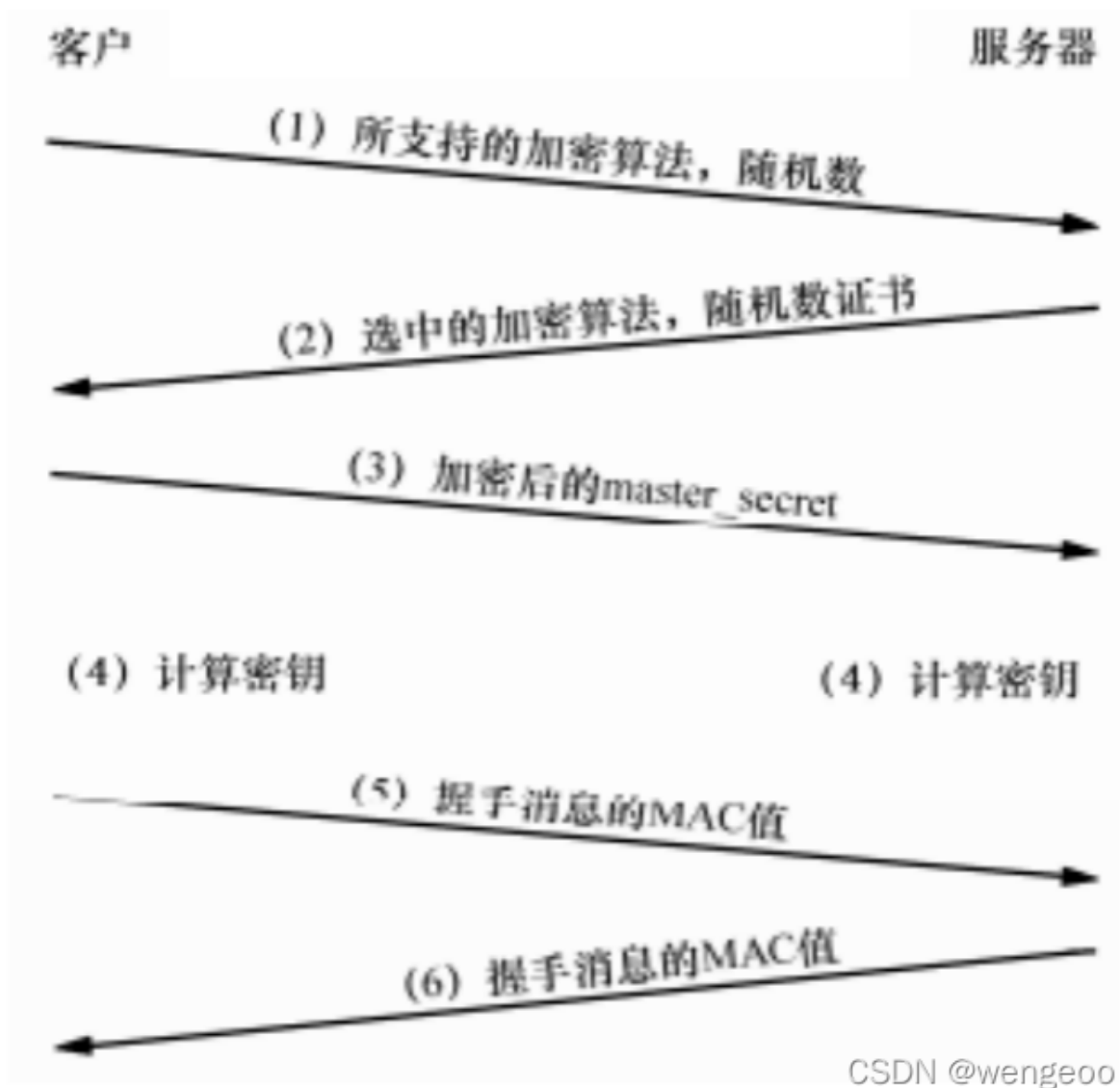


图2 SSL握手概述

(1) 客户端将它所支持的算法列表连同一个密钥产生过程用作输入的随机数发送给服务器。

(2) 服务器根据从列表的内容中选择一种加密算法，并将其连同一份包含服务器公用密钥的证书发回给客户端。该证书还包含了用于认证目的的服务器标识，服务器同时还提供了一个作为密钥产生过程部分输入的随机数。

(3) 客户端对服务器的证书进行验证，并抽取服务器的公用密钥。然后，再产生一个称做 `pre_master_secret` 的随机密码串，并使用服务器的公用密钥对其进行加密。最后，客户端将加密后的信息发送给服务器。

(4) 客户端与服务器端根据 `pre_master_secret` 以及客户端与服务器的随机数值独立计算出加密和 MAC 密钥。

(5) 客户端将所有握手消息的 MAC 值发送给服务器。

(6) 服务器将所有握手消息的 MAC 值发送给客户端。

在此过程结束时，客户端与服务器已就使用的加密算法达成一致，并拥有了一组与那些算法一起使用的密钥。

非对称加密

基于上述的特点，我们可以得出如下结论：

(1) 公钥是开放给所有人的，但私钥是需要保密的，存在于服务端

(2) 服务器端server向client端 (A、B...) 的信息传输是不安全的：因为所有人都可以获取公钥

(3) 但client端 (A、B...) 向server端的信息传输确实安全的：因为私钥只有server端存在

如上图所示，在第 (2) 步时服务器发送了一个SSL证书给客户端，SSL 证书中包含的具体内容有证书的颁发机构、有效期、公钥、证书持有者、签名，通过第三方的校验保证了身份的合法，解决了公钥获取的安全性

以浏览器为例说明如下整个的校验过程：

(1) 首先浏览器读取证书中的证书所有者、有效期等信息进行——校验

(2) 浏览器开始查找操作系统中已内置的受信任的证书发布机构CA，与服务器发来的证书中的颁发者CA比对，用于校验证书是否为合法机构颁发

(3) 如果找不到，浏览器就会报错，说明服务器发来的证书是不可信任的。

(4) 如果找到，那么浏览器就会从操作系统中取出 颁发者CA 的公钥，然后对服务器发来的证书里面的签名进行解密

(5) 浏览器使用相同的hash算法计算出服务器发来的证书的hash值，将这个计算的hash值与证书中签名做对比

(6) 对比结果一致，则证明服务器发来的证书合法，没有被冒充

(7) 此时浏览器就可以读取证书中的公钥，用于后续加密了

14.一个包怎么能从一台电脑到另一台电脑

https://blog.csdn.net/qg_35826156/article/details/103383053

计算机网络的五层模型 ()

具体过程如下-----

1. “ping”命令所产生的数据包，我们归类为ICMP协议。说白了就是向目的地发送一个数据包，然后等待回应，如果回应正常则目的地的网络就是通的。

2. “小德”已经将ICMP协议打包到数据段里了，可是还不能发送，因为一个数据要想向外面传送，还得经过“有关部门”的批准-----IP协议。IP要将数据的源IP地址和目的IP地址分别打包在“小德”的头部和尾部，这样一来，大家才知道你的数据是要送到哪里。

3. 准备工作还没有完。接下来还有部门要审核-----ARP。ARP属于数据链路层协议，主要负责把IP地址对应到硬件地址。交换机就经常保留一张IP地址与硬件地址的对应表以便其查找目的地。而ARP就是用来生成这张表的。

4. 经过一番折腾，“小德”终于要走出这个倒霉的局域网了。可在此之前，它们还没忘给“小德”屁股后面盖个“戳”，说是什么CRC校验值，注：很多人弄不清FCS和CRC。所谓的CRC是一种校验方法，用来确保数据在传输过程中不会丢包，损坏等等，FCS是数据包（准确的说是frame）里的一个区域，用来存放CRC的计算结果的。到了目的地之后，目的计算机要检查FCS里的CRC值，如果与原来的相同，则说明数据在途中没有损坏。

5. 在走出去之前，那些家伙最后折磨了一次“小德”-----把小德身上众多的0和1，弄成了什么“高电压”“低电压”，在双绞线上传送了出去。晕~~出趟门就这么麻烦吗？

6. 坐着双绞线旅游，爽！可当看到很多人坐着同轴电缆，还有坐光纤的时候，小德又感觉不是那么爽了。就在这时，来到了旅途的中转站-----路由器。这地方可是高级场所，人家直接查看IP地址！剩下的一概不管，交给下面的人去做。够牛吧？路由器的内部也有一张表，叫做路由表，里面标识着哪一个网络的IP对应着路由器的哪一个端口。这个表也不是天生就有的，而是靠路由器之间互相“学习”之后生成的，当然也可以由管理员手工设定。这个“学习”的过程是依靠路由协议来完成的，比如RIP，EIGRP，OSPF等等。

7. 当路由器查看了“小德”的IP地址以后，根据路由表知道了小德要去的网络，接着就把小德转到了相应的端口了。至此，路由器的主要工作完成，下面又是打包，封装成frame，转换成电压信号等一系列“折腾”的活，就由数据链路层和物理层的模块去干吧。

8. 小德从路由器的出口出来，便来到了目的地---电脑B---所属的网络的默认网关。默认网关可以是路由器的一个端口，也可以是局域网里的各种服务器。不管怎样，下面的过程还是一样的：到交换机里的ARP表查询“小德”的IP地址，看看属于哪个局域网段或端口，然后就转发到B了。

9. 进了B的网卡之后，还要层层“剥皮”，基本上和从A出来的程序是一样的-----电脑B先校验一下CRC值，看看数据是否完整；然后检查一下frame的封装，看到是IP协议之后，就把“小德”交给IP“部门”了；IP协议一看目的地址，正确，再看看应用协议，是ICMP。于是知道了该怎么做了-----产生一个回应数据包，（可以命名为“回应小德”），并准备以同样的顺序向远端的A发送。。至于刚刚收到的那个数据包就丢弃了。（从底向上）

10. “回应小德”这个数据包又开始了上述同样的循环，只不过这次发送者是B而接收者是A了。

以上是一个最简单的路由过程，任何复杂的网络都是在次基础之上实现的。

15.get和post请求的区别

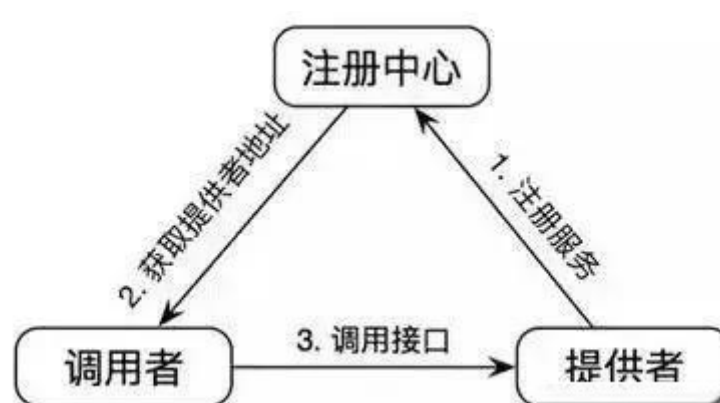
- GET在浏览器回退时是无害的，而POST会再次提交请求。
- GET产生的URL地址可以被Bookmark，而POST不可以。
- GET请求会被浏览器主动cache，而POST不会，除非手动设置。
- GET请求只能进行url编码，而POST支持多种编码方式。
- GET请求参数会被完整保留在浏览器历史记录里，而POST中的参数不会被保留。
- GET请求在URL中传送的参数是有长度限制的，而POST没有。
- 对参数的数据类型，GET只接受ASCII字符，而POST没有限制。
- GET比POST更不安全，因为参数直接暴露在URL上，所以不能用来传递敏感信息。
- GET参数通过URL传递，POST放在Request body中。

但是get和post的本质都是tcp连接，能做的事情是一样的。

16.懂不懂RPC

当垂直应用越来越多，应用之间交互不可避免，将核心业务抽取出来，作为独立的服务，逐渐形成稳定的服务中心，使前端应用能更快速的响应多变的市场需求。

此时，用于提高业务复用及整合的分布式服务框架(RPC)，提供统一的服务是关键。RPC服务的原因



RPC就是从一台机器(客户端)上通过参数传递的方式调用另一台机器(服务器)上的一个函数或方法(可以统称为服务)并得到返回的结果

一个基本的RPC架构里面应该至少包含以下4个组件：

1、客户端（Client）：

服务调用方（服务消费者）

2、客户端存根 (Client Stub) :

存放服务端地址信息, 将客户端的请求参数数据信息打包成网络消息, 再通过网络传输发送给服务端

3、服务端存根 (Server Stub) :

接收客户端发送过来的请求消息并进行解包, 然后再调用本地服务进行处理

4、服务端 (Server) :

服务的真正提供者

调用过程

- 1、服务消费者 (client客户端) 通过调用本地服务的方式调用需要消费的服务;
- 2、客户端存根 (client stub) 接收到调用请求后负责将方法、入参等信息序列化 (组装) 成能够进行网络传输的消息体;
- 3、客户端存根 (client stub) 找到远程的服务地址, 并且将消息通过网络发送给服务端;
- 4、服务端存根 (server stub) 收到消息后进行解码 (反序列化操作);
- 5、服务端存根 (server stub) 根据解码结果调用本地的服务进行相关处理;
- 6、本地服务执行具体业务逻辑并将处理结果返回给服务端存根 (server stub);
- 7、服务端存根 (server stub) 将返回结果重新打包成消息 (序列化) 并通过网络发送至消费方;
- 8、客户端存根 (client stub) 接收到消息, 并进行解码 (反序列化);
- 9、服务消费方得到最终结果;

当访问量越来越大, 我们需要通过不断添加服务器的方式来应对越来越大的访问量, 产生了集群; 或是将应用拆分成几个不相干的应用部署在不同的服务器上, 产生了分布式系统。

集群: 将相同的程序、功能部署在两台或是多台服务器上, 这些服务器对外提供的功能是完全一样的, 集群就是通过不同横向扩展增加服务器的方式, 以提高服务的能力。

分布式系统: 将两个或多个程序、功能分别运行在两台或多台主机服务器上, 这些服务对外提供的功能并不一样, 它们通过相互协作最终完成某一服务或是功能。

包含注册中心的RPC调用过程

服务提供者在启动时, 向注册中心注册自己提供的服务。

服务消费者在启动时, 向注册中心订阅自己所需的服务。

注册中心返回服务提供者地址列表给消费者, 如果有变更, 注册中心将基于长连接推送变更数据给消费者。

服务消费者, 从提供者地址列表中, 基于软负载均衡算法, 选一台提供者进行调用, 如果调用失败, 再选另一台调用。

服务消费者和提供者, 在内存中累计调用次数和调用时间, 定时每分钟发送一次统计数据到监控中心。

17.gRPC为什么高效

gRPC 进程间通信与 RESTful 服务不同的是, 它没有使用文本传输, 而是使用基于 protocol buffers 的二进制协议, 二进制传输的效率远远高于文本传输的效率, 并且 gRPC 是基于 HTTP/2 实现的 protocol buffers 协议, 从而使进程间通信更加高效。

gRPC 与 RESTful 服务不同的是, gRPC 先要定义服务接口, 然后再去实现细节。因此, gRPC 可以约束多语言开发的分布式应用程序, 使分布式应用程序更加可靠, 可扩展。

gRPC 使用 protocol buffers 定义服务接口，可以支持多种语言，并且强制约束了不同语言的分布式应用程序之间进程间通信使用的类型，可以使分布式应用程序更加稳定。

18.grpc调用（写一个实例）

<https://www.topgoer.cn/docs/grpc/grpc-1d2ud3fh1d74h>

proto文件中的message相当于go中的struct，会生成对应的工具方法（getter,setter）

Service

定义一个简单的Service，`TestService` 有一个方法 `Test`，接收一个 `Request` 参数，返回 `Response`：

```
1 // TestService 测试服务
2 service TestService {
3     // Test 测试方法
4     rpc Test(Request) returns (Response) {};
5 }
6
7 // Request 请求结构
8 message Request {
9     string name = 1;
10 }
11
12 // Response 响应结构
13 message Response {
14     string message = 1;
15 }
```

转换结果：

```
1 // 客户端接口
2 type TestServiceClient interface {
3     // Test 测试方法
4     Test(ctx context.Context, in *Request, opts ...grpc.CallOption)
5     (*Response, error)
6 }
7
8 // 服务端接口
9 type TestServiceServer interface {
10    // Test 测试方法
11    Test(context.Context, *Request) (*Response, error)
12 }
```

生成的go代码中包含该Service定义的接口，客户端接口已经自动实现了，直接供客户端使用者调用，服务端接口需要由服务提供方实现。

19.http长连接和短链接的应用

在 HTTP/1.0 中默认使用短连接。也就是说，客户端和服务端每进行一次 HTTP 操作，就建立一次连接，任务结束就中断连接。当客户端浏览器访问的某个 HTML 或其他类型的 Web 页中包含有其他的 Web 资源（如：JavaScript 文件、图像文件、CSS 文件等），每遇到这样一个 Web 资源，浏览器就会重新建立一个 HTTP 会话。

而从 HTTP/1.1 起，默认使用长连接，用以保持连接特性。使用长连接的 HTTP 协议，会在响应头加入这行代码：

Connection:keep-alive

1

在使用长连接的情况下，当一个网页打开完成后，客户端和服务端之间用于传输 HTTP 数据的 TCP 连接不会关闭，客户端再次访问这个服务器时，会继续使用这一条已经建立的连接。

Keep-Alive 不会永久保持连接，它有一个保持时间，可以在不同的服务器软件（如：Apache）中设定这个时间。实现长连接需要客户端和服务端都支持长连接。

操作系统

1..进程，线程，协程联系和区别

一个进程可以有多个执行线程，执行不同的任务

对操作系统来说，线程是最小的执行单元，进程是最小的资源管理单元。每个线程都有执行栈，多个线程共享数据空间

总介绍

1.进程是资源分配的单位；

2.线程是CPU调度的单位；

3.协程，是一种比线程更加轻量级的存在，协程不是被操作系统内核所管理，而完全是由程序所控制（也就是在用户态执行）。这样带来的好处就是性能得到了很大的提升，不会像线程切换那样消耗资源。

一个线程可以有多个协程

协程在子程序内部是可中断的，然后转而执行别的子程序，在适当的时候再返回来接着执行。效率高，不需要多线程的锁机制

协程中断是由程序内部的实现机制控制，与线程阻塞不同，线程阻塞是由操作系统内核控制的

线程与进程的区别：

- 1) 地址空间:线程是进程内的一个执行单元，进程内至少有一个线程，它们共享进程的地址空间，而进程有自己独立的地址空间
- 2) 资源拥有:进程是资源分配和拥有的单位,同一个进程内的线程共享进程的资源
- 3) 线程是处理器调度的基本单位,但进程不是
- 4) 二者均可并发执行
- 5) 每个独立的线程有一个程序运行的入口、顺序执行序列和程序的出口，但是线程不能够独立执行，必须依存在应用程序中，由应用程序提供多个线程执行控制

协程与线程的区别：

- 1) 一个线程可以多个协程，一个进程也可以单独拥有多个协程。
- 2) 线程进程都是同步机制，而协程则是异步。
- 3) 协程能保留上一次调用时的状态，每次过程重入时，就相当于进入上一次调用的状态。
- 4) 线程是抢占式，而协程是非抢占式的，所以需要用户自己释放使用权来切换到其他协程，因此同一时间其实只有一个协程拥有运行权，相当于单线程的能力。
- 5) 协程并不是取代线程,而且抽象于线程之上,线程是被分割的CPU资源,协程是组织好的代码流程,协程需要线程来承载运行,线程是协程的资源,但协程不会直接使用线程,协程直接利用的是执行器(Interceptor),执行器可以关联任意线程或线程池,可以使当前线程,UI线程,或新建新程。
- 6) 线程是协程的资源。协程通过Interceptor来间接使用线程这个资源。

23.golang协程为什么比线程更轻量

1. go协程切换比线程切换效率高

线程切换需要切换内核栈和硬件上下文的，而协程切换只发生在用户态，没有时钟中断、系统调用等机制，效率更高。协程切换发生在用户态，线程切换发生在内核态。

2. go协程占用的内存少

执行go协程只需要极少的栈内存（大概是4~5KB），默认情况下，而线程栈的大小为1MB。

goroutine就是一段代码，一个函数入口，以及在堆上为其分配的一个堆栈。所以它非常廉价，我们可以很轻松的创建上万个goroutine，但它们并不是被操作系统所调度执行。

协程切换比较简单，就是把当前协程的 CPU 寄存器状态保存起来，然后将需要切换进来的协程的 CPU 寄存器状态加载的 CPU 寄存器上就 ok 了。而且完全在用户态进行，一般来说一次协程上下文切换最多就是几十 ns 这个量级。

2.进程调度算法

1) 先到先服务

先来先去服务调度算法是一种最简单的调度算法，也称为先进先出或严格排队方案。当每个进程就绪后，它加入就绪队列。当前正运行的进程停止执行，选择在就绪队列中存在时间最长的进程运行。该算法既可以用于作业调度，也可以用于进程调度。先来先去服务比较适合于常作业（进程），而不利于段作业（进程）。

2) 最短进程优先

最短进程优先是一个非抢占策略，他的原则是下一次选择预计处理时间最短的进程，因此短进程将会越过长作业，跳至队列头。该算法即可用于作业调度，也可用于进程调度。但是他对长作业不利，不能保证紧迫性作业（进程）被及时处理，作业的长短只是被估算出来的。

3) 时间片轮转

轮转法是基于适中的抢占策略的，以一个周期性间隔产生时钟中断，当中断发生后，当前正在运行的进程被置于就绪队列中，然后基于先来先去服务策略选择下一个就绪作业的运行。这种技术也称为时间片，因为每个进程再被抢占之前都给定一片时间。

4) 优先级调度算法

按照进程的优先级高低来进行调度，使高优先级进程优先得到处理机的调度算法称为优先级调度算法。进程的优先级可以由操作系统按一定原则赋予，也可以在操作系统外部安排，甚至可由用户支付高额费用来购买。

但在许多采用优先级调度算法的系统中，通常使用动态优先级。一个进程的优先级不是固定的，可能会随许多因素的变化而变化，例如，进程的等待时间、已使用的处理机时间或其他资源的使用情况。

5) 最高响应比优先算法

这是一个非剥夺的算法。按照此算法，每个进程都有一个动态优先数，该优先数不但是要求的服务时间的函数，而且是该进程得到服务所花费的等待时间的函数。进程的动态优先数计算公式如下：

优先数=（等待时间+要求的服务时间）/要求的服务时间

“等待时间+要求的服务时间”是系统对作业的响应时间，所以在优先数公式中，优先数的值实际上也是响应时间与服务时间的比值，称为响应比。响应比高者得到优先调度。

6) 多级反馈队列调度算法

3.进程之间的通信方式

7种常见通信方式

1. 管道：用于具有亲缘关系的父子进程间或者兄弟进程间的通信
2. 有名管道：严格遵循先进先出规则，以磁盘文件形式存在，实现本机任意两个进程之间通信

3. 信号：用于通知接收进程某个事件已经发生
4. 消息队列：消息队列是由消息的链表，存放在内核中并由消息队列标识符标识。消息队列克服了信号传递信息少、管道只能承载无格式字节流以及缓冲区大小受限等缺点。
5. 信号量：用于多进程对数据的访问，意图在于进程间同步
6. 共享内存：多个进程可以访问同一块内存空间，需要进行同步操作；映射一段能被其他进程所访问的内存，这段共享内存由一个进程创建，但多个进程都可以访问。往往与其他通信方式，如信号量一起搭配使用
7. 套接字：cs之间网络通信，它可用于不同机器间的进程通信。

4.线程之间的通信方式

共享内存：线程之间共享程序的公共状态，线程之间通过读-写内存中的公共状态来隐式通信

消息传递（java中使用wait/notify的方式）：线程之间没有公共的状态，线程之间必须通过明确的发送信息来显示的进行通信

管道输入输出流

临界区：通过多线程的串行化来访问公共资源或一段代码，速度快，适合控制数据访问；

互斥量Synchronized/Lock：采用互斥对象机制，只有拥有互斥对象的线程才有访问公共资源的权限。因为互斥对象只有一个，所以可以保证公共资源不会被多个线程同时访问

信号量Semaphore：为控制具有有限数量的用户资源而设计的，它允许多个线程在同一时刻去访问同一个资源，但一般需要限制同一时刻访问此资源的最大线程数目。

5.linux下进程，线程实现，如何通信

最早是用管道实现，分为有名管道和匿名管道，用fork创建

现在最好的方式是使用共享内存

共享内存是最快的IPC形式，共享内存进行进程间通信的时候不涉及内核。

共享内存就是允许两个不相关的进程访问同一个逻辑内存。

共享内存是在两个正在运行的进程之间共享和传递数据的一种非常有效的方式。

不同进程之间共享的内存通常安排为同一段物理内存。进程可以将同一段共享内存连接到它们自己的地址空间中，所有进程都可以访问共享内存中的地址，就好像它们是由用C语言函数malloc分配的内存一样。

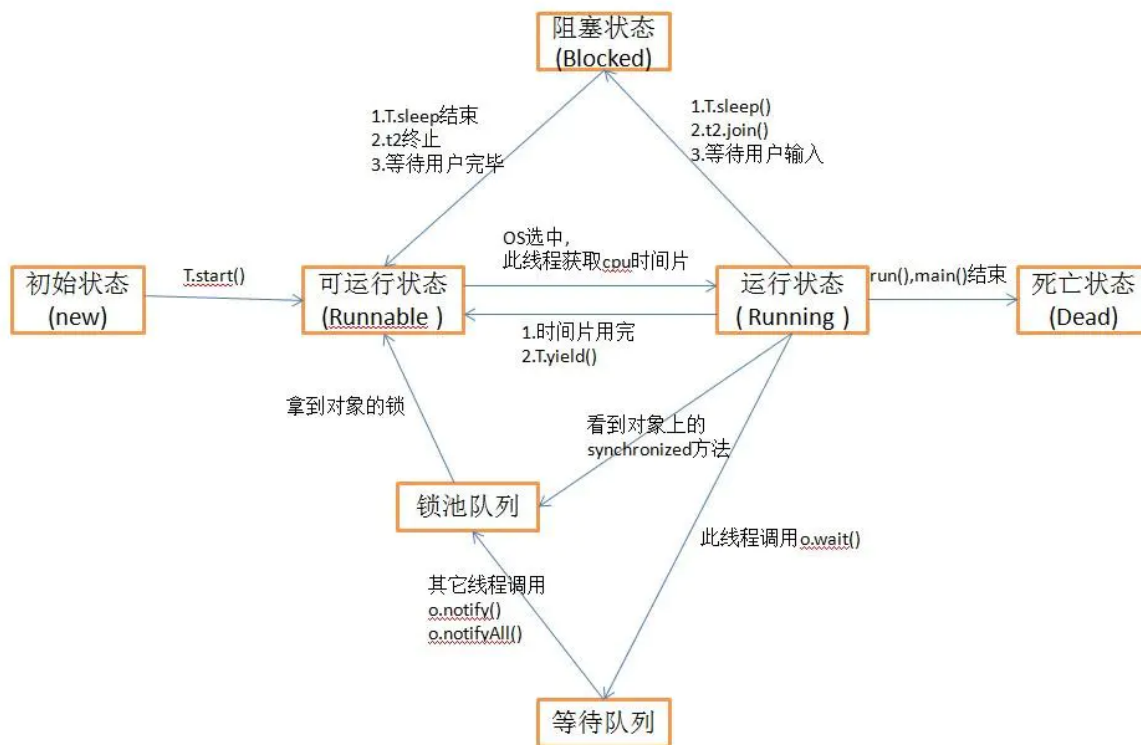
如果某个进程向共享内存写入数据，所做的改动将立即影响到可以访问同一段共享内存的任何其他进程。

6.僵尸进程，孤儿进程

僵尸进程：一个父进程利用fork创建子进程，如果子进程退出，而父进程没有利用wait 或者 waitpid 来获取子进程的状态信息，那么子进程的状态描述符依然保存在系统中。

孤儿进程：一个父进程退出，而它的一个或几个子进程仍然还在运行，那么这些子进程就会变成孤儿进程，孤儿进程将被init进程（进程号为1）所收养，并由init进程对它们完成状态收集的工作

7.线程的不同状态和状态之间的切换



CSDN @小則沐风

8. 介绍一下虚拟内存和好处

虚拟内存的核心原理

为每个程序设置一段"连续"的虚拟地址空间，把这个地址空间分割成多个具有连续地址范围的页 (Page)，并把这些页和物理内存做映射，在程序运行期间动态映射到物理内存。当程序引用到一段在物理内存的地址空间时，由硬件立刻执行必要的映射；而当程序引用到一段不在物理内存中的地址空间时，由操作系统负责将缺失的部分装入物理内存并重新执行失败的指令。

其实虚拟内存技术从某种角度来看的话，很像是糅合了基址寄存器和界限寄存器之后的新技术。它使得整个进程的地址空间可以通过较小的虚拟单元映射到物理内存，而不需要为程序的代码和数据地址进行重定位。

好处

虚拟内存让每个进程都产生了一种自己在独享主存的错觉，定义了一个连续的虚拟地址空间，并且把内存扩展到硬盘空间

更有效的管理内存，并减少出错

9. 讲一下分段和分页-内存划分的种类

1) 分段：主存分为段，每段的空间比页小，段具有实际意义。段表。

内存不同的区域有不同的性质，如有的区域用于存放指令，这种区域只允许读，有的区域用于存放数据，这种区域可读可写。

为了方便管理不同性质的区域，需要将内存进行分段处理，让不同的段之间不会相互干扰。

如何分段

首先在内存中找出一个空闲区域，然后将这片区域分成数据段，代码段，堆栈段等几个区域（段），并把这几个区域的基地址及其属性（如读写属性）保存放到该进程的LDT和PCB中。最后把程序中的指令部分放到代码段中，把全局变量放到数据段中等等。当CPU执行这个进程时，先将PCB中的寄存器内容（包括段寄存器）“扣到”寄存器中，若遇到偏移地址则根据指令找到对应的段基地址，用段基地址 + 偏

移地址就可以找到内存的物理地址了。

2) 分页：主存分为大小相等的页，页比较小，提高了内存利用率，减少了碎片。页表对应逻辑地址和物理地址(linux中大小一般为4KB)

- 将物理内存分成等大的块，这里把每一个块叫做一个**帧 (frame)**；
- 与此同时把一个进程的逻辑地址空间也分成等大的块，这里把每一个块叫做一个**页 (page)**；
- **进程分页，内存分帧**，页帧等大小；
- 为当前的进程分配物理内存的时候，实际上是把内存中一系列的**非连续的帧**分配给**对应的页**上。这样的映射关系叫做**页表**；

页表记录着当前进程的逻辑地址空间不同的页对应的物理帧的帧号是多少。也就是说由页表来记录当前进程内存分配的过程；

- 采用分页的内存分配方案时，由CPU去访问逻辑地址时（或者CPU产生的逻辑地址），此时进程的**逻辑地址=页号+页偏移量**；
- 分页这种内存分配方案不会产生外部碎片，因为都是按块进行分配的，但是其有内部碎片问题。

分页的好处

- 多个进程在运行的过程当中，可共享可重入代码 (re-entrant code)；
- 所谓可重入代码，是指在程序运行过程中保持不变的代码。

10.死锁产生的四个必要条件

产生死锁的四个必要条件：死锁是因为多个进程并发争夺系统资源而互相等待的现象。死锁要满足四个必要条件才会产生，分别是：

- **互斥**：某种资源一次只允许一个进程访问，即该资源一旦分配给某个进程，其他进程就不能再访问，直到该进程访问结束。
- **占有且等待**：一个进程本身占有资源（一种或多种），同时还有资源未得到满足，正在等待其他进程释放该资源。
- **不可抢占**：别人已经占有了某项资源，你不能因为自己也需要该资源，就去把别人的资源抢过来。
- **循环等待**：存在一个进程链，使得每个进程都占有下一个进程所需的至少一种资源。

如何预防和避免线程死锁？

如何预防死锁？ 破坏死锁的产生的必要条件即可：

1. **破坏请求与保持条件**：一次性申请所有的资源。
2. **破坏不剥夺条件**：占用部分资源的线程进一步申请其他资源时，如果申请不到，可以主动释放它占有的资源。
3. **破坏循环等待条件**：靠按序申请资源来预防。按某一顺序申请资源，释放资源则反序释放。破坏循环等待条件。

如何避免死锁？

避免死锁就是在资源分配时，借助于算法（比如银行家算法）对资源分配进行计算评估，使其进入安全状态。

安全状态 指的是系统能够按照某种线程推进顺序 (P1、P2、P3.....Pn) 来为每个线程分配所需资源，直到满足每个线程对资源的最大需求，使每个线程都可顺利完成。称 $\langle P1、P2、P3.....Pn \rangle$ 序列为安全序列。

我们对线程 2 的代码修改成下面这样就不会产生死锁了。

```

1  new Thread(() -> {
2      synchronized (resource1) {
3          System.out.println(Thread.currentThread() + "get
resource1");
4          try {
5              Thread.sleep(1000);
6          } catch (InterruptedException e) {
7              e.printStackTrace();
8          }
9          System.out.println(Thread.currentThread() + "waiting get
resource2");
10         synchronized (resource2) {
11             System.out.println(Thread.currentThread() + "get
resource2");
12         }
13     }
14     }, "线程 2").start();

```

输出:

```

1  Thread[线程 1,5,main]get resource1
2  Thread[线程 1,5,main]waiting get resource2
3  Thread[线程 1,5,main]get resource2
4  Thread[线程 2,5,main]get resource1
5  Thread[线程 2,5,main]waiting get resource2
6  Thread[线程 2,5,main]get resource2
7
8  Process finished with exit code 0

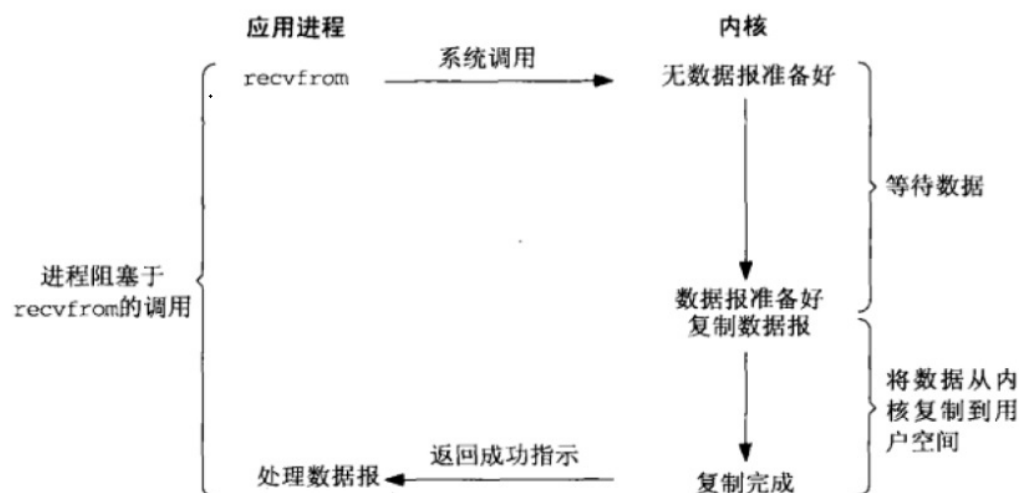
```

我们分析一下上面的代码为什么避免了死锁的发生?

线程 1 首先获得到 resource1 的监视器锁,这时候线程 2 就获取不到了。然后线程 1 再去获取 resource2 的监视器锁, 可以获取到。然后线程 1 释放了对 resource1、resource2 的监视器锁的占用, 线程 2 获取到就可以执行了。这样就破坏了破坏循环等待条件, 因此避免了死锁。

11.I/O多路复用

什么是阻塞IO, 所谓阻塞IO就是当应用B发起读取数据申请时, 在内核数据没有准备好之前, 应用B会一直处于等待数据状态, 直到内核把数据准备好了交给应用B才结束。



知乎 @勤劳的小手

所谓非阻塞IO就是当应用B发起读取数据申请时，如果内核数据没有准备好会即刻告诉应用B，不会让B在这里等待。

术语：非阻塞IO是在应用调用recvfrom读取数据时，如果该缓冲区没有数据的话，就会直接返回一个EWOULDBLOCK错误，不会让应用一直等待中。在没有数据的时候会即刻返回错误标识，那也意味着如果应用要读取数据就需要不断的调用recvfrom请求，直到读取到它数据要的数据为止。

区分阻塞IO和非阻塞IO主要看第一个阶段中硬件到内核态的阶段中是否存在阻塞用户的操作

流程：

- 1、应用进程向内核发起recvfrom读取数据。
- 2、没有数据报准备好，即刻返回EWOULDBLOCK错误码。
- 3、应用进程向内核发起recvfrom读取数据。
- 4、已有数据包准备好就进行一下步骤，否则还是返回错误码。
- 5、将数据从内核拷贝到用户空间。
- 6、完成后，返回成功提示。

1) BIO

BIO是一个典型的网络编程模型，是通常我们实现一个服务端程序的过程，步骤如下：

- 主线程accept请求阻塞
- 请求到达，创建新的线程来处理这个套接字，完成对客户端的响应。
- 主线程继续accept下一个请求

这种模型有一个很大的问题是：当客户端连接增多时，服务端创建的线程也会暴涨，系统性能会急剧下降。因此，在此模型的基础上，类似于tomcat的bio connector，采用的是线程池来避免对于每一个客户端都创建一个线程。有些地方把这种方式叫做伪异步IO(把请求抛到线程池中异步等待处理)

(阻塞模式)

2) NIO

JDK1.4开始引入了NIO类库，这里的NIO指的是Non-block IO，主要是使用Selector多路复用器来实现。Selector在Linux等主流操作系统上是通过epoll实现的。(非阻塞模式)

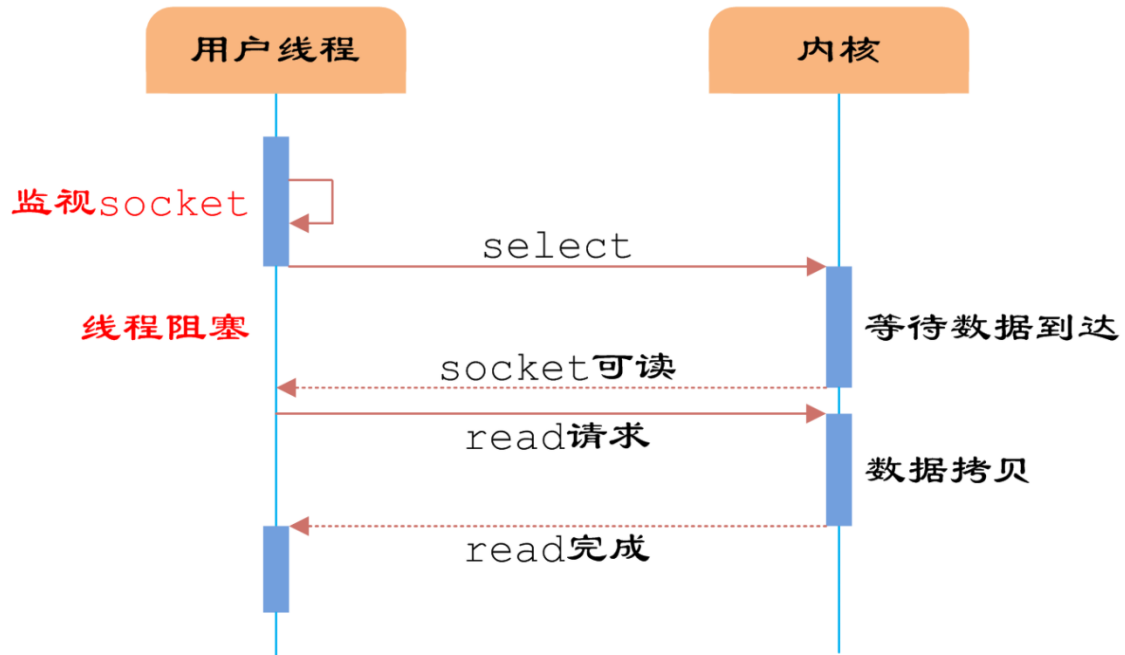
NIO的实现流程，类似于select：

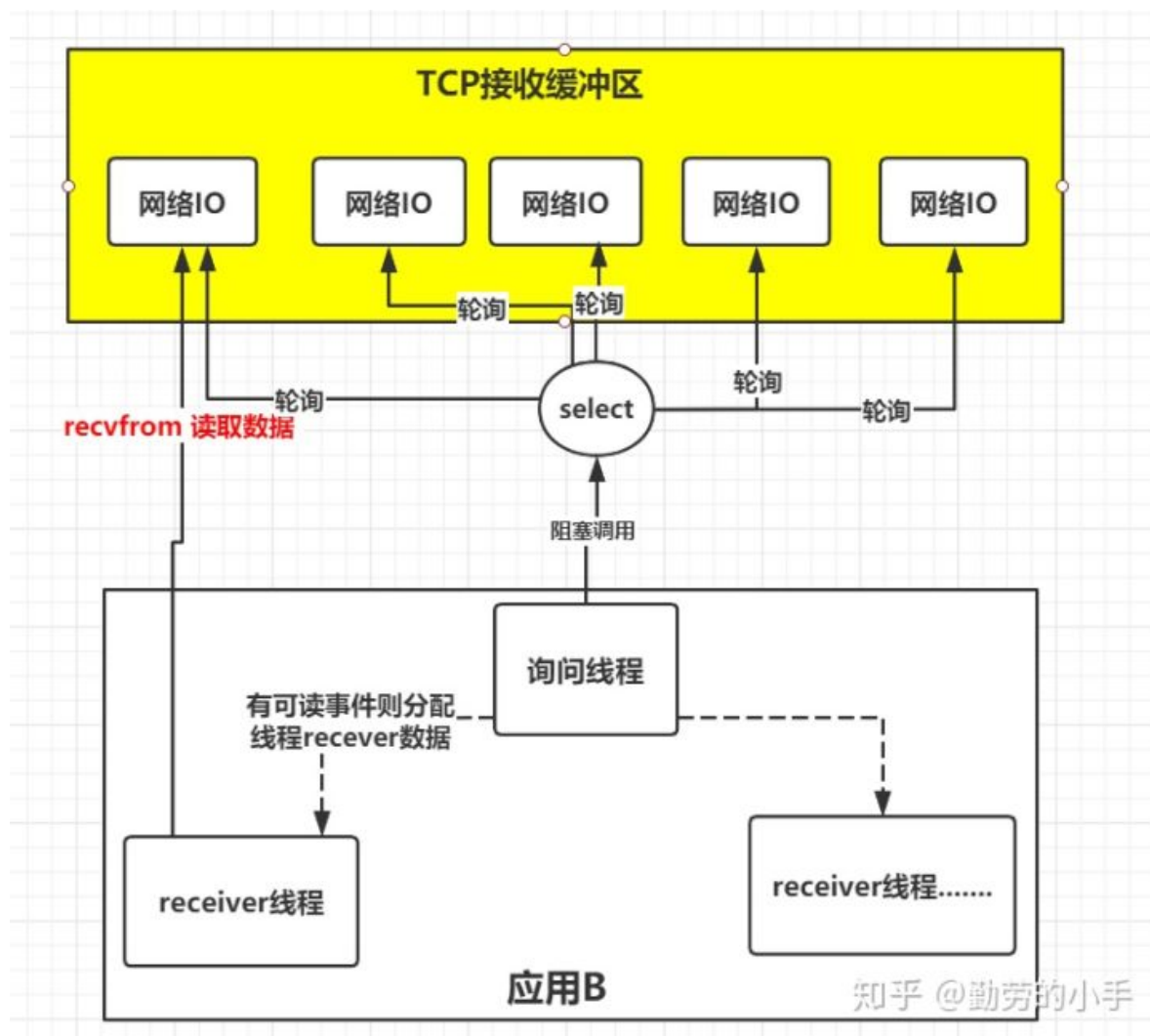
- 创建ServerSocketChannel监听客户端连接并绑定监听端口，设置为非阻塞模式。
- 创建Reactor线程，创建多路复用器(Selector)并启动线程。
- 将ServerSocketChannel注册到Reactor线程的Selector上。监听accept事件。
- Selector在线程run方法中无限循环轮询准备就绪的Key。
- Selector监听到新的客户端接入，处理新的请求，完成tcp三次握手，建立物理连接。
- 将新的客户端连接注册到Selector上，监听读操作。读取客户端发送的网络消息。
- 客户端发送的数据就绪则读取客户端请求，进行处理。

相比BIO，NIO的编程非常复杂。

3) IO多路复用

建立在reactor模式上，并且使用系统内置的select函数完成





用户首先将需要进行IO操作的socket添加到select中，然后阻塞等待select系统调用返回。当数据到达时，socket被激活，select函数返回。用户线程正式发起read请求，读取数据并继续执行。

从流程上来看，使用select函数进行IO请求和同步阻塞模型没有太大的区别，甚至还多了添加监视socket，以及调用select函数的额外操作，效率更差。但是，使用select以后最大的优势是用户可以在一个线程内同时处理多个socket的IO请求。用户可以注册多个socket，然后不断地调用select读取被激活的socket，即可达到在同一个线程内同时处理多个IO请求的目的。而在同步阻塞模型（或者是同步非阻塞模型）中，必须通过多线程的方式才能达到这个目的。

select是一个阻塞的IO模型，但是IO多路复用是一个非阻塞调用的模型，因为当前线程并不会一直停留在这里等待对面返回，而是继续做其他操作。（只是对这一个事件阻塞，对其他操作可以正常执行）

复用IO的基本思路就是通过select或poll、epoll来监控多fd，来达到不必为每个fd创建一个对应的监控线程，从而减少线程资源创建的目的。只要有一个fd成功，则select函数就会准备执行

4) AIO

JDK1.7引入NIO2.0，提供了异步文件通道和异步套接字通道的实现。其底层在windows上是通过IOCP，在Linux上是通过epoll来实现的

(LinuxAsynchronousChannelProvider.java, UnixAsynchronousServerSocketChannelImpl.java)。

- 创建AsynchronousServerSocketChannel，绑定监听端口
- 调用AsynchronousServerSocketChannel的accept方法，传入自己实现的CompletionHandler。包括上一步，都是非阻塞的
- 连接传入，回调CompletionHandler的completed方法，在里面，调用AsynchronousSocketChannel的read方法，传入负责处理数据的CompletionHandler。
- 数据就绪，触发负责处理数据的CompletionHandler的completed方法。继续做下一步处理即可。

- 写入操作类似，也需要传入CompletionHandler。

其编程模型相比NIO有了不少的简化。

(异步非阻塞-最先进)

go轮询器

IO阻塞：当我们执行 `read` 系统调用时，应用程序会从用户态陷入内核态，内核会检查文件描述符是否可读；当文件描述符中存在数据时，操作系统内核会将准备好的数据拷贝给应用程序并将控制权交回。

IO非阻塞：

当进程把一个文件描述符设置成非阻塞时，执行 `read` 和 `write` 等 I/O 操作就会立刻返回。在 C 语言中，我们可以使用如下所示的代码片段将一个文件描述符设置成非阻塞的：

```
int flags = fcntl(fd, F_GETFL, 0);fcntl(fd, F_SETFL, flags | O_NONBLOCK);
```

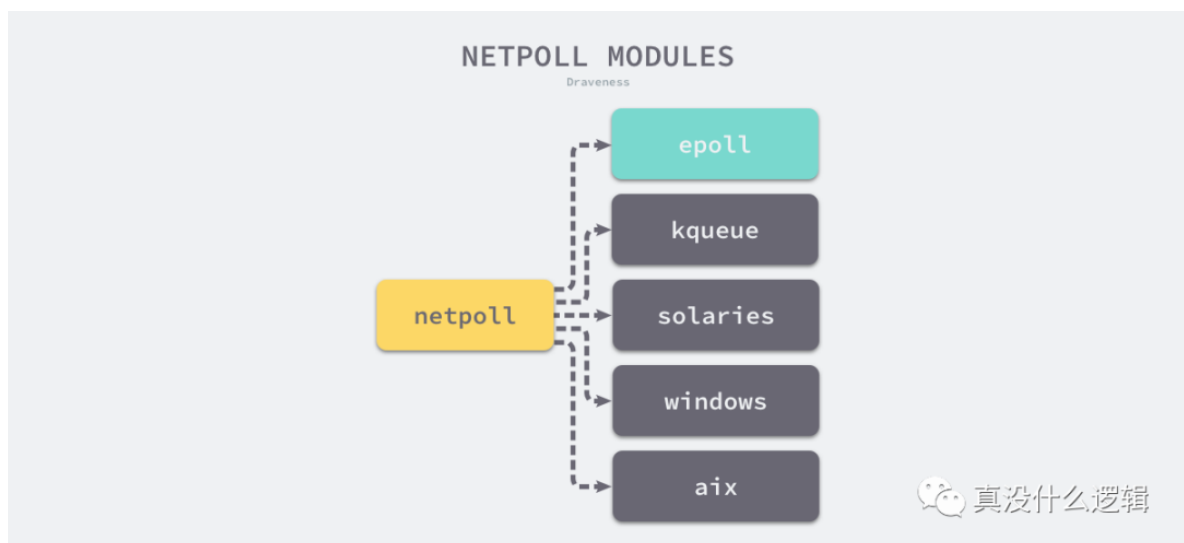
第一次从文件描述符中读取数据会触发系统调用并返回 EAGAIN 错误，EAGAIN 意味着该文件描述符还在等待缓冲区中的数据；随后，应用程序会不断轮询调用 `read` 直到它的返回值大于 0，这时应用程序就可以对读取操作系统缓冲区中的数据并进行操作。进程使用非阻塞的 I/O 操作时，可以在等待过程中执行其他的任务，增加 CPU 资源的利用率。

IO多路复用：I/O 多路复用被用来处理同一个事件循环中的多个 I/O 事件。I/O 多路复用需要使用特定的系统调用，最常见的系统调用就是 `select`，该函数可以同时监听最多 1024 个文件描述符的可读或者可写状态：

```
int select(int nfds, fd_set restrict readfds, fd_set restrict writefds, fd_set restrict errorfds, struct timeval restrict timeout);
```

多路复用函数会阻塞的监听一组文件描述符，当文件描述符的状态转变为可读或者可写时，`select` 会返回可读或者可写事件的个数，应用程序就可以在输入的文件描述符中查找哪些可读或者可写，然后执行相应的操作。

GO语言不使用SELECT函数，使用epoll, kqueue, evport等，在不同的平台上实现不同函数操作。



上述函数在网络轮询器中分别扮演了不同的作用：

`runtime.netpollinit` — 初始化网络轮询器，通过 `sync.Once` 和 `netpollinit` 变量保证函数只会调用一次；

`runtime.netpollopen` — 监听文件描述符上的边缘触发事件，创建事件并加入监听；

`runtime.netpoll` — 轮询网络并返回一组已经准备就绪的 Goroutine，传入的参数会决定它的行为[^3]；

如果参数小于 0，无限期待文件描述符就绪；

如果参数等于 0，非阻塞地轮询网络；

如果参数大于 0，阻塞特定时间轮询网络；

runtime.netpollBreak — 唤醒网络轮询器，例如：计时器向前修改时间时会通过该函数中断网络轮询器[^4]；

runtime.netpollIsPollDescriptor — 判断文件描述符是否被轮询器使用；

初始化网络轮询器->加入待监控的任务->获取触发的事件

初始化：调用epoll函数创建文件描述符，创建用于通信的管道，并且使用epollctl将读取数据的文件描述符打包成事件加入监听

加入新事件：netpollopen初始化轮询事件，加入事件描述符的可读可写状态，当然这个事件也是可删除的

获取事件：当我们在文件描述符上执行读写操作时，如果文件描述符不可读或者不可写，当前 Goroutine 就会执行 runtime.poll_runtime_pollWait 检查 runtime.pollDesc 的状态并调用 runtime.netpollblock 等待文件描述符的可读或者可写：

`runtime.netpollblock` 是 Goroutine 等待 I/O 事件的关键函数，它会使用运行时提供的 `runtime.gopark` 让出当前线程，将 Goroutine 转换到休眠状态并等待运行时的唤醒。

Go 语言的运行时会在调度或者系统监控中调用 runtime.netpoll 轮询网络，该函数的执行过程可以分成以下几个部分：

根据传入的 delay 计算 epoll 系统调用需要等待的时间；

调用 epollwait 等待可读或者可写事件的发生；epollwait函数返回的值>0时，显示有待处理的事件在循环中依次处理 epollevnt 事件；

处理的事件总共包含两种，一种是调用 `runtime.netpollBreak` 函数触发的事件，该函数的作用是中断网络轮询器；另一种是其他文件描述符的正常读写事件，对于这些事件，我们会交给 `runtime.netpollready` 处理：

runtime.netpollunblock 会在读写事件发生时，将 runtime.pollDesc 中的读或者写信号量转换成 pdReady 并返回其中存储的 Goroutine；如果返回的 Goroutine 不会为空，那么该 Goroutine 就会被加入 toRun 列表，运行时会将列表中的全部 Goroutine 加入运行队列并等待调度器的调度。

原文链接：https://blog.csdn.net/weixin_39542889/article/details/112663999

12.select、poll、epoll区别

1.select的句柄数目受限，在linux/posix_types.h头文件有这样的声明：`#define _FD_SETSIZE 1024`表示**select最多同时监听1024个fd**。而epoll没有，它的限制是系统对整个进程限制的最大文件描述符。

poll本质上和select没有区别，它将用户传入的数组拷贝到内核空间，然后查询每个fd对应的设备状态，**但是它没有最大连接数的限制**，原因是它是基于链表来存储的。

2.epoll的最大好处是不会随着FD的数目增长而降低效率，在select中采用轮询处理，其中的数据结构类似一个数组的数据结构，而epoll 是维护一个队列，直接看队列是不是空就可以了。不同于忙轮询和无差别轮询，epoll会把哪个流发生了怎样的I/O事件通知我们。所以我们说epoll实际上是**事件驱动（每个事件关联上fd）**的，此时我们对这些流的操作都是有意义的。（**复杂度降低到了O(1)**）**epoll只会对"活跃"的socket进行操作**---这是因为在内核实现中epoll是根据每个fd上面的callback函数实现的。那么，只有"活跃"的socket才会主动的去调用 callback函数（把这个句柄加入队列），其他idle状态句柄则不会，在这点上，epoll实现了一个"伪"AIO。**因此在实际的网络环境中，Epoll的效率就会远远高于select和poll**。但是如果绝大部分的I/O都是“活跃的”，每个I/O端口使用率很高的话，epoll效率不一定比select高（可能是要维护队列复杂）。

3.epoll比select更高效

epoll使用mmap加速内核与用户空间的消息传递。无论是select,poll还是epoll都需要内核把FD消息通知给用户空间，如何避免不必要的内存拷贝就很重要，在这点上，epoll是通过内核于用户空间mmap同一块内存实现的。（内核与用户空间共享一起实现的）**epoll的实现中使用了mmap调用使得内核空间和用户空间共享内存，从而避免了过多的内核和用户空间的切换引起的开销。**

mmap 是一种内存映射文件的方法，即将一个文件或者其它对象映射到进程的地址空间，实现文件磁盘地址和进程虚拟地址空间中一段虚拟地址的一一对映关系。实现这样的映射关系后，进程就可以采用指针的方式读写操作这一段内存，而系统会自动回写脏页面到对应的文件磁盘上，即完成了对文件的操作而不必再调用 read、write 等系统调用函数。

综上所述，select和epoll对比如下表所示

	select	epoll
性能	随着连接数增加，急剧下降。处理成千上万并发连接数时，性能很差。	随着连接数增加，性能基本上没有下降。处理成千上万并发连接时，性能很好。
连接数	连接数有限制，处理的最大连接数不超过1024。如果要处理超过1024个连接数，则需要修改FD_SETSIZE宏，并重新编译。	连接数无限制。
内在处理机制	线性轮询	回调callback
开发复杂性	低	中

在不阻塞 IO 模式里，如果要监控的文件描述符比较多的话。可以使用 select 模型。select 将监控多个文件描述符，再将结果统一的返回给处理程序。

由于 select 能监视的文件描述符有限，一般为 1024 个，为了处理这个瓶颈，后面实现了 poll 这种结构来监控文件描述符的数据到达情况。即 poll 没有最大数量的限制。

select 和 poll 本质上都需要遍历的去操作文件描述符，效率不高。为此, linux 后面有设计了 epoll 模型，通过在内核里使用一个红黑树的结构来管理注册进来的文件描述符，同时使用了一个就绪链表维护了就绪事件，一旦基于某个文件描述符就绪时，内核会采用类似 callback 的回调机制，将事件通知给用户进程

epoll的两种触发模式

epoll有EPOLLLET和EPOLLET两种触发模式，LT是默认的模式，ET是“高速”模式。LT模式下，只要这个fd还有数据可读，每次 epoll_wait都会返回它的事件，提醒用户程序去操作，而在ET（边缘触发）模式中，它只会提示一次，直到下次再有数据流入之前都不会再提示了，无论fd中是否还有数据可读。所以在ET模式下，read一个fd的时候一定要把它的buffer读光，也就是说一直读到read的返回值小于请求值，或者遇到EAGAIN错误。还有一个特点是，epoll使用“事件”的就绪通知方式，通过epoll_ctl注册fd，一旦该fd就绪，内核就会采用类似callback的回调机制来激活该fd，epoll_wait便可以收到通知。

epoll为什么要有EPOLLET触发模式？

如果采用EPOLLLET模式的话，系统中一旦有大量你不需要读写的就绪文件描述符，它们每次调用 epoll_wait都会返回，这样会大大降低处理程序检索自己关心的就绪文件描述符的效率。而采用EPOLLET这种边沿触发模式的话，当被监控的文件描述符上有可读写事件发生时，epoll_wait()会通知处理程序去读写。如果这次没有把数据全部读写完(如读写缓冲区太小)，那么下次调用epoll_wait()时，它不会通知你，也就是它只会通知你一次，直到该文件描述符上出现第二次可读写事件才会通知你!!! **这种模式比水平触发效率高，系统不会充斥大量你不关心的就绪文件描述符**

13.虚拟内存、物理内存、共享内存介绍下？有什么区别和使用场景

1) linux系统的物理内存被划分为许多相同大小的部分，也称作内存页。内存页的大小取决于CPU的架构和操作系统的配置，一般为4KB。

问题

内部碎片（单个页框内的碎片）

物理内存是按页来分配的，这样当实际只需要很小内存的时候，也会分配至少是 4K 大小的页面，而内核中有很多需要以字节为单位分配内存的场景，这样本来只想要几个字节而已却不得不分配一页内存，除去用掉的字节剩下的就形成了内部碎片。

外部碎片（页框间的碎片）

当需要分配大块内存的时候，要用好几页组合起来才够，而系统分配物理内存页的时候会尽量分配连续的内存页面，频繁的分配与回收物理页导致大量的小块内存夹杂在已分配页面中间，形成外部碎片

2) 虚拟内存实际上并不存在，它只是存在于这套巧妙的内存管理机制中。当一个进程启动时，内核会给新的进程建立一个虚拟地址空间。这个虚拟地址空间代表了该进程可能使用到的所有内存，当然它是可以动态变化的。

`malloc` 用于申请用户空间的虚拟内存，当申请小于 128KB 小内存的时，`malloc` 使用 `sbrk`或`brk` 分配内存；当申请大于 128KB 的内存时，使用 `mmap` 函数申请内存；

问题：由于 `brk/sbrk/mmap` 属于系统调用，如果每次申请内存都要产生系统调用开销，`cpu` 在用户态和内核态之间频繁切换，非常影响性能。

而且，堆是从低地址往高地址增长，如果低地址的内存没有被释放，高地址的内存就不能被回收，容易产生内存碎片。

解决：因此，`malloc` 采用的是内存池的实现方式，先申请一大块内存，然后将内存分成不同大小的内存块，然后用户申请内存时，直接从内存池中选择一块相近的内存块分配出去。

3) 进程在运行过程中，会加载许多操作系统的动态库，比如 `libc.so`、`libld.so`等。这些库对于每个进程而言都是公用的，它们在内存中实际只会加载一份，这部分称为共享内存。

14.用户态和内核态区分

内核态（Kernel Mode）：运行操作系统程序，操作硬件

用户态（User Mode）：运行用户程序

- 内核态与用户态是操作系统的两种运行级别，当程序运行在3级特权级上时，就可以称之为运行在用户态。**因为这是最低特权级，是普通的用户进程运行的特权级，大部分用户直接面对的程序都是运行在用户态；**
- 当程序运行在0级特权级上时，就可以称之为运行在内核态。
- 运行在用户态下的程序不能直接访问操作系统内核数据结构和程序。当我们在系统中执行一个程序时，大部分时间是运行在用户态下的，在其需要操作系统帮助完成某些它没有权力和能力完成的工作时就会切换到内核态（比如操作硬件）。
- 这两种状态的主要差别是
 - 处于用户态执行时，进程所能访问的内存空间和对象受到限制，其所处于占有的处理器是可被抢占的
 - 处于内核态执行时，则能访问所有的内存空间和对象，且所占有的处理器是不允许被抢占的。

通常来说，以下三种情况会导致用户态到内核态的切换

- 系统调用

这是用户态进程主动要求切换到内核态的一种方式，用户态进程通过系统调用申请使用操作系统提供的服务程序完成工作。比如前例中`fork()`实际上就是执行了一个创建新进程的系统调用。

而系统调用的机制其核心还是使用了操作系统为用户特别开放的一个中断来实现，例如Linux的int 80h中断。

用户程序通常调用库函数，由库函数再调用系统调用，因此有的库函数会使用户程序进入内核态（只要库函数中某处调用了系统调用），有的则不会。

- 异常

当CPU在执行运行在用户态下的程序时，发生了某些事先不可知的异常，这时会触发由当前运行进程切换到处理此异常的内核相关程序中，也就转到了内核态，比如缺页异常。

- 外围设备的中断

当外围设备完成用户请求的操作后，会向CPU发出相应的中断信号，这时CPU会暂停执行下一条即将要执行的指令转而去执行与中断信号对应的处理程序，

如果先前执行的指令是用户态下的程序，那么这个转换的过程自然也就发生了由用户态到内核态的切换。比如硬盘读写操作完成，系统会切换到硬盘读写的中断处理程序中执行后续操作等。

linux常用命令

top：能够实时显示系统中各个进程的资源占用状况，常用于服务端性能分析

- `-i<时间>`：设置间隔时间；
- `-u<用户名>`：指定用户名；
- `-p<进程号>`：指定进程；
- `-n<次数>`：循环显示的次数

chmod给文件修改权限，+x (1) 可执行，+r (4) 可读，+w (2) 可写

seq命令可以输出连续的数字，固定间隔，指定格式的数字

-s 指定输出的分隔符，默认为\n，即默认为回车换行

-w 指定为定宽输出，不能和-f一起用

-f 按照指定的格式输出，不能和-w一起使用

先使用echo命令输出制表符，然后用输出的制表符作为seq命令输出数字的连接符

awk命令：对文本进行格式化

awk [options] 'Pattern{Action}' file

awk是逐行处理的，逐行处理的意思就是说，当awk处理一个文本时，会一行一行进行处理，处理完当前行，再处理下一行，awk默认以“换行符”为标记，识别每一行，也就是说，awk跟我们人类一样，每次遇到“回车换行”，就认为是当前行的结束，新的一行的开始。如果没有指定分隔符则默认使用一个空格作为分隔符

\$符号可以索引第几列

awk是一种编程语言，用于在linux系统中编写脚本

grep命令：是一个可以利用“正则表达式”进行“全局搜索”的工具，grep会在文本文件中按照指定的正则进行全局搜索，并将搜索出的行打印出来

--color -n -i

-B+数字：表示输出匹配字符串的之前的几行

-A+数字同理，之后几行

vmstat命令是最常见的Linux/Unix监控工具，属于sysstat包。可以展现给定时间间隔的服务器的状态值,包括服务器的CPU使用率，内存使用，虚拟内存交换情况,IO读写情况。

df：用于显示目前在 Linux 系统上的文件系统磁盘使用情况统计。

Linux touch命令用于修改文件或者目录的时间属性，包括存取时间和更改时间。若文件不存在，系统会建立一个新的文件。ls -l可以查看文件的更改时间

ps：显示当前进程的状态，相当于任务管理器

ps -aux：显示所有包含其他使用者的进程

free：用于显示内存占用状态

log日志查询

linux文件数字权限

```
444 r--r--r--
600 rw-----
644 rw-r--r--
666 rw-rw-rw-
700 rwx-----
744 rwxr--r--
755 rwxr-xr-x
777 rwxrwxrwx
```

从左至右，

1-3位数字代表文件所有者的权限，

4-6位数字代表同组用户的权限，

7-9数字代表其他用户的权限。

读取权限：等于4 用 r 表示

写入权限：等于2 用 w 表示

执行权限：等于1 用 x 表示

场景题

1.select * from table limit 80000,20会出现什么问题

扫描满足条件的800020行，扔掉前面的800000行，返回最后的20行，可想而知这时会很慢

解决方式：

方法1：子查询，id>=的形式：

```
1 | select * from product
2 | where ID >= ( select id from product order by id limit 80000,1 ) limit 20
```

查询时间为0.2秒，简直是一个质的飞跃啊。

方法2：利用join（推荐这种方法）

```
1 | select * from product a
2 | JOIN ( select id from product order by id limit 80000,20 ) b on a.id = b.id
```

上面两个一个是子查询，一个是关联查询，那么这两种方案的查询性能哪一个更好呢？

- 子查询：把内层查询结果当作外层查询的比较条件，使用 IN() 函数、EXISTS 运算符等，示例如下：

```
1 select goods_id,goods_name from goods where goods_id = (select
2   max(goods_id) from goods);
1
```

执行子查询时，MYSQL需要创建临时表，查询完毕后再删除这些临时表，所以，子查询的速度会受到一定的影响，这里多了一个创建和销毁临时表的过程。

- 连接查询 (JOIN) 连接查询不需要建立临时表，因此其速度比子查询快。另外注意：能过滤先过滤，过滤好了再链接

2.海量数据处理问题

原则：放的进内存就遍历或者排序 放不进内存就hash分而治之

能装进内存

计数排序

- 高考考生成绩排序

待排序的数组的n个元素都在 (0, k) 区间内，k为某个正整数

新建一个辅助数组C[0...k]用来统计每个元素出现的次数（并且是按照顺序进行统计）

不用考虑桶排序，能用桶就能用计数排序

快速排序

- 寻找中位数

快排有轴划分的特点，每次可以把左边或者右边的排除掉

位图算法

- 寻找中位数-排序
- 寻找重复数-重复

每个bit代表一个数字，一个bitmap有多长根据内存换算

$512M = 8 * 1024 * 1024 * 512 = 2^{32}$

遍历一次，即可排序，但前提要是不重复数字

遍历一次，即可查重

不能装进内存

哈希取模+分而治之

- 海量日志数据，提取出某日访问百度次数最多的那个IP

哈希取模成各个小文件后，对小文件进行处理，最后再将结果归并一次即可

堆排序

- 所有求前最多和第K多

借助堆，只需要topK大小，时间复杂度 $n\log K$ 。内存里面只需要维护K堆大小即可

一个堆装不下K的时候，可以分多个堆，遍历一次，建一个堆，取堆顶数字，再遍历第二次，建一个堆
额外数据结构

布隆过滤器

- 保证内容不重复过滤器，可以是用户记录，也可以是url

当布隆过滤器说某个值存在时，这个值可能不存在；当它说不存在时，那就肯定不存在。

可能会丢掉一些不重复的值，但判断为不重复的一定没问题

3.设计一个限流器

朴素计数器

- 维护一个计数器count和起始时间startTime，count表示[startTime ... curTime]的请求数；
startTime过期后，重新设置startTime，并重置count；

假定一个user限制每分钟请求3次；我需要维护一个hash，key=userID，value={count, startTime}；收到一个请求后，rate limiter有如下步骤

```
1  `# step 1
2  if userID not in hash:
3      hash[userID] = {1, current_time}
4      accept req
5  # step 2
6  `if userID in hash and current_time - hash[userID].startTime >= 1min:
7      # 赋值startTime
8      hash[userID] = {1, current_time}
9      accept req
10 # step 3
11 if userID in hash and current_time - hash[userID].startTime < 1min:
12     if hash[userID].count < 3:
13         accept req
14         hash[userID].count++
15     else:
16         reject req
17
```

滑动窗口

如果我们可以跟踪每个用户的每个请求，我们可以维护一个滑动窗口。我们可以将每个请求的时间戳存储在散列表的“value”字段的Redis排序集中。

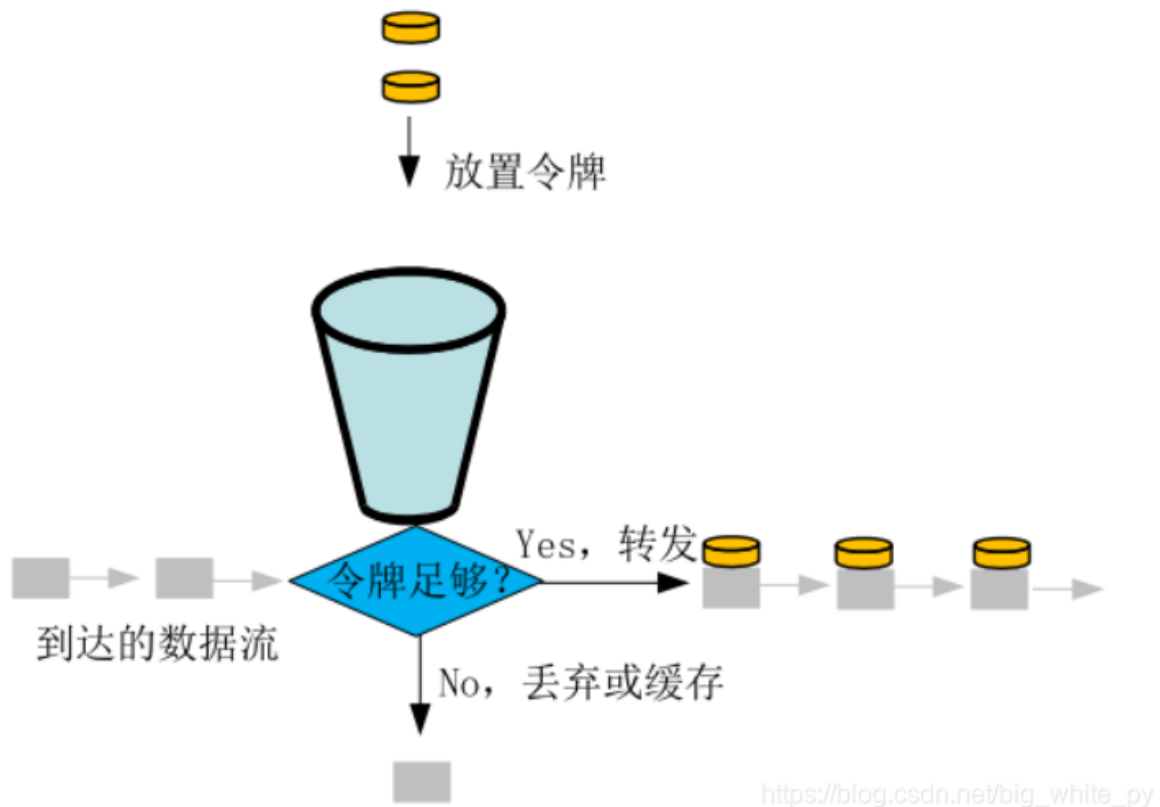
- 存储每个请求的时间戳，按照时间戳有序，每来一个请求，查询队列首的时间戳T，if gap <= T - curTime 弹出 else 计算队列中元素数量size；size < N 接收请求 else 丢弃；

令牌桶算法

令牌桶能够在限制请求平均速率的同时还允许一定程度的突发调用。在令牌桶算法中，存在一个桶，用来存放固定数量的令牌。该算法以一定的速率往桶中放入令牌。**每次请求需要先获取到桶中的令牌才能继续执行，否则等待可用的令牌，或者直接拒绝。**

放令牌的动作是持续不断进行的，如果桶中令牌数达到上限，则丢弃令牌，因此桶中可能一直持有大量的可用令牌。此时请求进来可以直接拿到令牌执行。比如设置qps为100，那么限流器初始化完成1秒后，桶中就已经有100个令牌了，如果此前还没有请求过来，这时突然来了100个请求，该限流器可以抵挡瞬时的100个请求。由此可见，只有桶中没有令牌时，请求才会进行等待，最终表现的效果即为

以一定的速率执行。



当请求到达服务端时首先会根据请求数据的大小从令牌桶中取出与数据大小相当的令牌数量用来传输数据。也就是说要使请求数据被传输必须保证令牌桶里有足够多的令牌，如果令牌数量不够，则数据就不会正常被服务器处理。这就可以限制报文的流量只能小于等于令牌生成的速度，达到限制流量的目的。

golang的limiter包封装令牌桶算法，主要调用allow，wait方法。每次调用消耗一个令牌，limiter可以指定放令牌的频率。没有令牌的时候，allow方法就会返回false，导致无法继续执行任务，起到限流作用。

3.如何处理一亿次请求

可以采用限流器，使用限流算法对处理的请求量进行限制

削峰：可以添加kafka等中间件起到缓冲作用，将用户提交的修改申请放在消息队列中，系统按照数据库最大能支持的并发数量从消息队列中慢慢拉取消息。**高峰期积压变少**

缓存：可以添加中间件缓存，例如redis将请求缓存起来，等下游积压请求处理完成后再读取redis中存储的请求

涉及到缓存雪崩和缓存击穿的问题：

缓存击穿：

缓存击穿是指缓存中没有但数据库中有数据（一般是缓存时间到期），这时由于并发用户特别多，同时读缓存没读到数据，又同时去数据库去取数据，引起数据库压力瞬间增大，造成过大压力。和缓存雪崩不同的是，缓存击穿指并发查同一条数据，缓存雪崩是不同数据都过期了，很多数据都查不到从而查数据库。

缓存雪崩：

缓存雪崩是指缓存同一时间大面积的失效，所以，后面的请求都会落到数据库上，造成数据库短时间内承受大量请求而崩掉。

解决方案

1、缓存数据的过期时间设置随机，防止同一时间大量数据过期现象发生。

2、一般并发量不是特别多的时候，使用最多的解决方案是加锁排队。

3、给每一个缓存数据增加相应的缓存标记，记录缓存的是否失效，如果缓存标记失效，则更新数据缓存。

解决方案

1、设置热点数据永远不过期

2、加互斥锁，互斥锁

加kafka：看kafka的第20条

4.100个G的log文件，里面写了访问ip地址，找出访问次数最多的前100个ip。

Hash分桶法：

- 将100G文件分成1000份，将每个IP地址映射到相应文件中： $\text{file_id} = \text{hash}(\text{ip}) \% 1000$ ；相同的IP会映射到同一个文件

- 在每个文件中分别求出最高频的IP，再合并 Hash分桶法：接下来把每个小文件载入内存，建立[哈希表](#)`unordered_map<string,int>`，将每个IP作为关键字映射为出现次数，这个哈希表建好之后也得先写入硬盘，因为内存就那么多，一共要统计100个文件呢。

- 使用Hash分桶法把数据分发到不同文件

- 各个文件分别统计top K，使用大顶堆维护出现频率最高的K个IP

- 最后Top K汇总

Linux命令，假设top 10：`sort log_file | uniq -c | sort -nr k1,1 | head -10`

5.可不可以用学号做主键索引，有什么坏处，那如果用自增的id呢？两者的区别，是你你怎么选？性别可不可以做索引

学号不能用做主键索引，因为学号字段太长了，会额外占用很多的数据页的空间，会导致同等数据下数据页变多了。所以不建议用学号作为主键。所以考虑要么给学号建立唯一索引，要么就是建立普通索引。如果业务上已经保证了不会有重复的学号的，那么这两个选哪一个都行。

从性能上考虑，普通索引更好；当需要更新一个数据页的时候，如果数据页还在内存中就直接更新，如果数据页在磁盘中就先更新的内容缓存在change buffer中，这样就节省了将数据页从磁盘读取到内存这个操作。下次需要访问这个数据页的时候，将数据页读取到内存中，然后执行change buffer中与这个数据页相关的更新操作，可以保证这个数据页的正确性。唯一索引每次更新数据的时候，要将数据页读到内存中，然后找到相关的位置判断要更新的数据存在否，所以**唯一索引在更新数据的时候总是要把数据页读到内存中，change buffer对它起不到任何作用**。普通索引用到change buffer，[将数据从磁盘读入内存涉及随机 IO 的访问](#)，是数据库里面成本最高的操作之一。change buffer 因为减少了随机磁盘访问，所以对更新性能的提升是很明显的。所以普通索引性能更好。

自增ID可以用作主键索引，因为在InnoDB存储引擎中，主键索引是作为聚簇索引存在的，也就是说，主键索引的B+树叶子节点上存储了主键索引以及全部的数据(按照顺序)，如果主键索引是自增ID，那么只需要不断向后排列即可；如果mysql没有指定主键，innodb会选择一个唯一键主键，没有唯一键也会创建一个隐式的。

UUID不推荐，因为如果是UUID，由于到来的ID与原来的大小不确定，会造成非常多的数据插入，数据移动，然后导致产生很多的内存碎片，进而造成插入性能的下降。

性别字段也不可以；因为你访问索引需要付出额外的IO开销，你从索引中拿到的只是地址，要想真正访问到数据还是要对表进行一次IO。假如你要从表的100万行数据中取几个数据，那么利用索引迅速定位，访问索引的这IO开销就非常值了。但如果你是从100万行数据中取50万行数据，就比如性别字段，那你相对需要访问50万次索引，再访问50万次表，加起来的开销并不会比直接对表进行一次完整扫描小。只有2种取值的字段，建了索引数据库也不一定会用，只会白白增加索引维护的额外开销，因为索引也是需要存储的，所以插入和更新的写入操作，同时需要插入和更新你这个字段的索引的。维护的成本和扫描的成本加在一起可能并不少于扫描全表的成本。

6.扫码登录的流程

1、首先准备工作

网站应用微信登录是基于OAuth2.0协议标准构建的微信OAuth2.0授权登录系统。

在进行微信OAuth2.0授权登录接入之前，在微信开放平台注册开发者帐号，并拥有一个已审核通过的网站应用，并获得相应的AppID和AppSecret

申请微信登录且通过审核后，可开始接入流程。就是你的网站要想实现微信扫码登陆功能首先要在微信备案通过，它才会给你个AppID和AppSecret。

2、时序流程图

时序图总结流程：

- 1、用户点击网站微信登陆图标。
- 2、网站带上AppID和AppSecret和回调域名参数请求微信OAuth2.0授权登陆。
- 3、第二步通过后，微信返回二维码供用户扫码。
- 4、用户扫码确定授权。
- 5、微信带上code参数回调java后端相关接口。
- 6、java后端获取到code后，在带上带上AppID和AppSecret和code再去调微信接口。
- 7、获取access_token后就可以解析用户的一些基本信息，比如：微信用户头像、用户名、性别、城市等一些基本参数。

7.将长URL转换为短URL

短网址映射算法的理论（网上找到的资料）：

- ① 将长网址用md5算法生成32位签名串，分为4段，每段8个字符；
- ② 对这4段循环处理，取每段的8个字符，将他看成16进制字符串与0x3fffffff(30位1)的位与操作，超过30位的忽略处理；
- ③ 将每段得到的这30位又分成6段，每5位的数字作为字母表的索引取得特定字符，依次进行获得6位字符串；
- ④ 这样一个md5字符串可以获得4个6位串，取里面的任意一个就可作为这个长url的短url地址。

我们并不一定说得到的URL是唯一的，但是我们能够取出4组URL，这样几乎不会出现太大的重复。

8.1000w数据中找出一个随机数

使用bitmap

存储新数字

```
1 public void set(int number) {
2     //相当于对一个数字进行右移动3位，相当于除以
8     int index = number >> 3;
3
4     //相当于 number % 8 获取到byte[index]的位
置     int position = number & 0x07;
5
6     //进行|或运算 参加运算的两个对象只要有一个为1，其值为
1.    bytes[index] |= 1 << position;
7 }
```

判断数字是否存在

```
1 public boolean contain(int number) {  
2     int index = number >> 3;  
3     int position = number & 0x07;  
4     return (bytes[index] & (1<<position)) !=0;  
5 }
```

位图通过数组下标来定位数据，所以，访问效率非常高。而且，每个数字用一个二进制位来表示，在数字范围不大的情况下，所需要的内存空间非常节省。

但是当数据取值范围更大时，使用布隆过滤器。采用多个hash函数计算，判断数据是否存在。

个人介绍

互联网：

面试官你好，我叫祁肖宇，目前就读于北京邮电大学计算机学院，专业是计算机科学与技术；我的本科就读于北京邮电大学国际学院，专业是电子商务及法律；在校期间与实验室同学合作完成了一个空地一体化多维数据查询系统；在2021年11月我在知乎进行了一段为期三个月的实习，主要工作是协助开发一个数据调度转发平台。在2022年6月-8月在华为进行了一段为期两个月的暑期实习，主要工作是开发了一个对网关网络产品进行补丁漏洞规则检查的工具；此外我曾参加过英语四六级考试和托福的考试，并且曾获得一等奖学金和优秀研究生称号；我的自我介绍到此结束，谢谢！

国企：

面试官你好，我叫祁肖宇，目前就读于北京邮电大学计算机学院，专业是计算机科学与技术。本科就读于北京邮电大学国际学院，专业是电子商务及法律，并以优异成绩推免报送到本校攻读研究生学位；在研究生期间与实验室同学合作完成了一个空地一体化多维数据查询系统；2021年11月我在知乎进行了一段为期三个月的实习，主要工作是协助开发一个数据调度转发平台。在2022年6月-8月在华为进行了一段为期两个月的暑期实习，主要工作是开发了一个对网关网络产品进行补丁漏洞规则检查的工具。此外我曾在本科多次获得校级学业奖学金和优秀团员称号，并曾获得校级优秀毕业生称号；在研究生阶段曾获得校级一等奖学金和优秀研究生称号。我曾参加大学英语四六级考试和托福考试，并取得优异成绩。我的自我介绍到此结束，谢谢！

项目介绍

知乎

doris与clickhouse等其他数仓的区别

Doris更优的方面1. 使用更简单，如建表更简单，SQL标准支持更好，Join性能更好，导数功能更强大2. 运维更简单，如灵活的扩缩容能力，故障节点自动恢复，社区提供的支持更好3. 分布式更强，支持事务和幂等性导入数据，物化视图自动聚合，查询自动路由，全面元数据管理

ClickHouse更优的方面1. 性能更佳，导入性能和单表查询性能更好，同时可靠性更好2. 功能丰富，非常多的表引擎，更多类型和函数支持，更好的聚合函数以及庞大的优化参数选项3. 集群管理工具更多，更好多租户和配额管理，灵活的集群管理，方便的集群间迁移工具

那么两者之间如何选择呢？1. 业务场景复杂数据规模巨大，希望投入研发力量做定制开发，选ClickHouse2. 希望一站式的分析解决方案，少量投入研发资源，选择Doris

Hive 主流数仓产品：基于hadoop分布式架构；

原理是将SQL (Hivesql) 通过解释器转换为mapreduce 的作业执行模型，提交到Hadoop集群上，返回作业执行结果给用户。hive 在加载数据过程中不会对数据进行任何的修改，只是将数据移动到HDFS中hive 设定的目录下，因此，hive 不支持对数据的改写和添加，所有的数据都是在加载的时候确定的。hive 的数据存储在 HDFS 中，大部分的查询由 MapReduce 完成（不包含 * 的查询，比如 select * from tbl 不会生成 MapReduce 任务）

优点：适用于海量数据批处理场景，使用Hiveql 语法

离线批处理 是其主场

缺点：hive 需要安装集群和运维

HBase NoSQL数据库--更适合存储半结构化数据

常用场景：

- 高并发业务查询；
- 实时流处理的业务存储；
- 非结构化半结构化数据存储；
- DDL频繁变动的场景；

数据仓库建设

Data warehouse（可简写为DW或者DWH）数据仓库，是在数据库已经大量存在的情况下，它是一整套包括了etl、调度、建模在内的完整的理论体系。

数据仓库并不是数据的最终目的地，而是为数据最终的目的地做好准备，这些准备包含：清洗、转义、分类、重组、合并、拆分、统计等

实习中使用的百度的doris数据仓库，doris的架构（说一下）

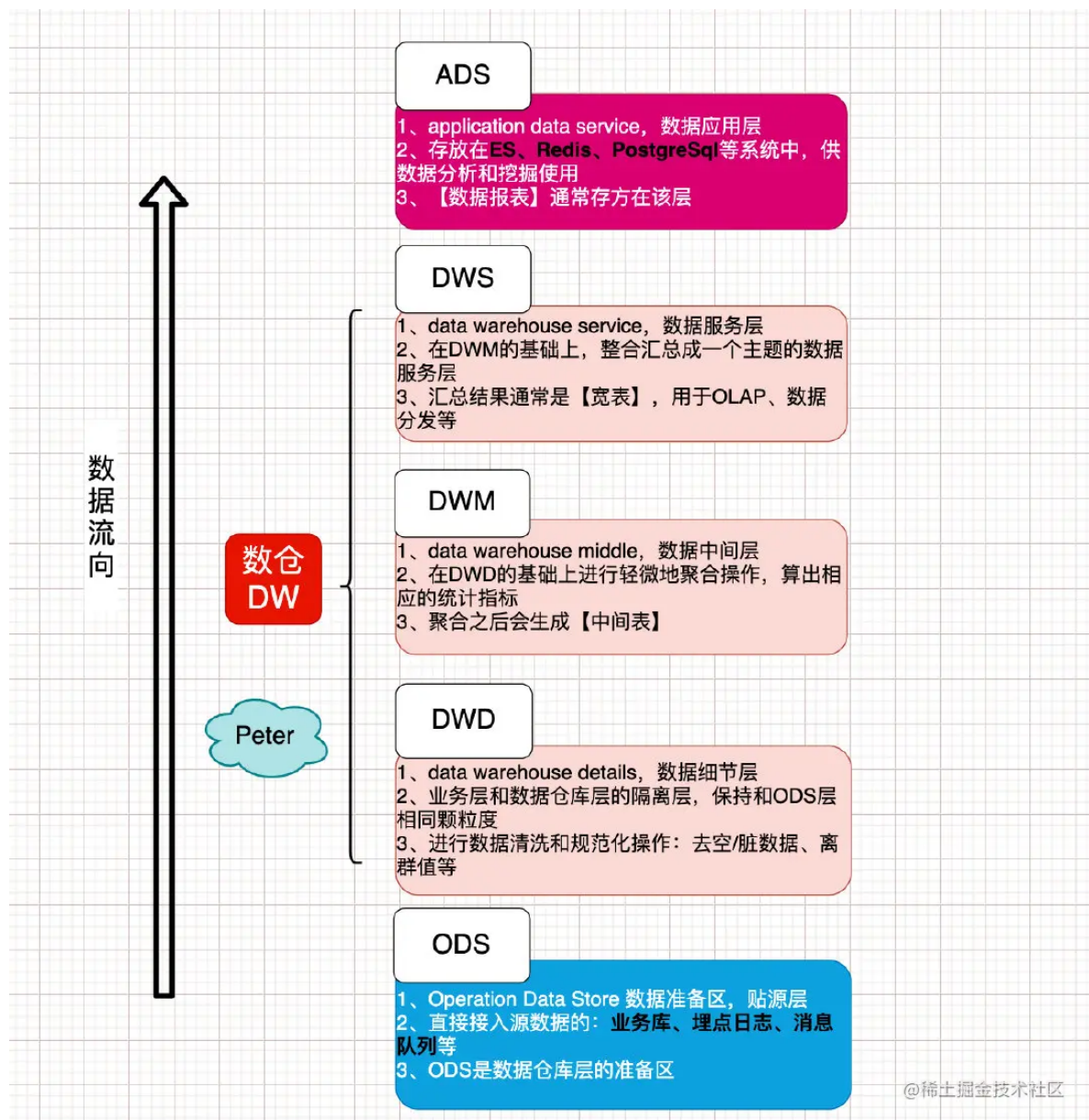
ODS

DWD

DWM

DWS

ADS



数据仓库就是在数据库已经大量存在的情况下, 对etl、调度、建模等操作的完整的操作流程

数据仓库并不是数据的最终目的地, 而是为数据的最终目的地做好准备, 也就是: 清洗、转义、分类、重组、合并、拆分、统计等

ODS数据运营层: 数据源经过抽取、洗净、传输 (也就是etl操作) 后进入本层, 本层除此之外并不会对数据进行额外的处理; 最好是原封不动的接入原始数据;

数据仓库层: 从上到下分为三个层: DWD、DWM、DWS

DWD:

该层是业务层和数据仓库的隔离层, 保持和ODS层一样的数据颗粒度; 主要是对ODS数据层做一些数据的清洗和规范化的操作, 比如去除空数据、脏数据、离群值等。

为了提高数据明细层的易用性, 该层通常会才采用一些维度退化方法, 将维度退化至事实表中, 减少事实表和维表的关联。

DWM:

在dwd基础上对数据进行一些轻微的聚合操作, 减少重复的加工工作; 并给出一些列的中间结果表, 算出相应的统计指标

DWS:

基于DWM，整合汇总分析某一个主题领域的数据服务层；是宽表；用于提供后续的业务查询、olap分析、数据分发等

ADS:

该层主要是提供给数据产品和数据分析使用的数据，一般会存放在ES、Redis、PostgreSQL等系统中供线上系统使用；也可能存放在hive或者Druid中，供数据分析和数据挖掘使用，比如常用的数据报表就是存在这里的。

知乎业务建设

首先有一张dwd_user_behavior_detail表，用于记录用户的行为；我们在业务中没有采用dwm层，而是直接将dwd表进行聚合并存储在dws表内；ads表用于存放数据集成业务的数据；同时所有的数据业务都会在hdfs中完成一个备份，用于后续业务中断后恢复数据；

数据需求

大概分为几种：

- 1.数据查询：聚合天级别、小时级别数据增量，并用一段时间内的数据增量对比离线数据数据增量，必须对齐；还涉及到多表、多字段计算后字段等
- 2.数据写入：从用户行为表以及其他相关表中完成数据集成后，写入doris数据库中新建的表。

数据平台建设：doris中取出数据，调度数据

数据平台目的：

在doris上存储的数据需要根据业务需求导出，因为doris的数据导出只能做到单任务单配置，不能做到动态修改（比如某个字段的计算公式变更，或者新增导出字段或者时间变更）和部分分区重试（对单次的任务执行记录进行重试，并不是对任务整体进行重试）；因此在doris基础上搭建了一个数据调度转发平台

数据调度聚合：

数据调度转发功能是在doris基础上，对doris数据库中的一些数据进行聚合整理，从doris数据库中取出，并通过配置工作在前端网页中完成，**数据聚合任务大致分为两种**，一种是单次执行的调度任务；另一种是定时执行的数据调度任务；（两种任务区别在于定时任务是例如每小时执行一次、或者每天执行一次；但单次执行任务，例如单次导出任务，执行一次就不会再执行了）

我在实习中对该部分内容进行了梳理和代码串讲、在mentor的指导下对该部分内容进行了学习和研究；了解了数据仓库的建设、学习了数据调度系统中对任务调度、数据存储的业务分层；

doris数据库中存储的用户行为数据是实时数据（更新的频率不了解），前端埋点会将用户行为捕捉，并通过数据导入的方式写入doris数据库

模仿doris数据导入的底层原理，用调度层（FE）和worker（BE）来实现任务的调度和数据存储转发功能

sql在worker中被执行，调度器中完成新增任务记录和sql拼接

worker执行情况在前端页面中可以看到

检测kafka的topic偏移量是否有延迟。

数据任务作为对下游业务的明显指标，能够用数据最直观的反映出业务的能力。同时，也需要对数据任务的执行情况进行监控并实时对不正常的业务内容报警处理。

在数据集成转发平台中，数据业务出现执行失败的原因主要有以下几点：

- 1.kafka的offset延迟了

2.进行任务执行的broker load挂了

3.网络故障。后两种原因是更为客观的系统安全性问题，但第一种原因是能够通过报警监控并完成及时纠正的。

数据导入任务中，doris仓库存在从kafka订阅的topic数据，并将这些数据通过routine load导入仓库。**在导入数据中可能会出现kafka的offset与当前消费到的offset对不上的情况**；所以需要观察当前kafka的对应topic的offset和routine load的offset的偏移量。在获取kafka的offset时，由于一个topic有多个partition，所以会获得每个partition在不同时刻的offset。与routine load的不同partition进行比较，如果routine load即消费的offset小于生产的offset，则当前数据消费出现延迟，需要取得最大的延迟时间；也就是找到能够满足当前offset<生产offset的最小偏移量，得到最大的延迟时间。因此能够得到当前任务的偏移量，并及时回溯数据；

关于数据回滚：每次数据回滚中都会将上一次有问题的部分分区进行删除，再进行一次重新执行数据导入任务，可以获得正常的数据。

理解kafka的topic和partition

Topic 是一个存储消息的逻辑概念，可以认为是一个消息集合。每条消息发送到 Kafka 集群的消息都有一个类别。物理上来说，不同的 Topic 的消息是分开存储的，每个 Topic 可以有多个生产者向它发送消息，也可以有多个消费者去消费其中的消息。

每个 Topic 可以划分多个分区（每个 Topic 至少有一个分区），同一 Topic 下的不同分区包含的消息是不同的。每个消息在被添加到分区时，都会被分配一个 offset，它是消息在此分区中的唯一编号，Kafka 通过 offset 保证消息在分区内的顺序，offset 的顺序不跨分区，即 **Kafka 只保证在同一个分区内的消息是有顺序的。**

消息是每次追加到对应的 Partition 的后面；**topic只是一个逻辑概念，真正存储还是通过partition存储；**

每个partition(目录)相当于一个巨型文件被平均分配到多个大小相等segment(段)数据文件中，但每个段segment file消息数量不一定相等；每个partiton只需要支持顺序读写就行了，segment文件生命周期由服务端配置参数决定，这样做的好处就是能快速删除无用文件，有效提高磁盘利用率。

前面提到了每个topic被分成了多个partition分布到各个broker上，而每个partition的文件夹中又又多个小文件组成。

- segment file组成：由2大部分组成，分别为index file和data file，此2个文件——对应，成对出现，后缀".index"和".log"分别表示为segment索引文件、数据文件。
- segment文件命名规则：partion全局的第一个segment从0开始，后续每个segment文件名为上一个全局partion的最大offset(偏移message数)。数值最大为64位long大小，19位数字字符长度，没有数字用0填充。

导致在每个消费者上保留的元数据只有offset，它是消费者在日志上的位置。这个offset受消费者控制：正常情况下，随着消费者读取消息offset是线性增长的，但是消费者可以控制offset，**消费者可以以任何顺序消费消息。**例如，一个消费者可以重置到老的offset来从新执行。kafka将每个分区下的消息分配给消费组里的消费者消费，每个分区都有一个消费者

这些特性表明，Kafka的消费者是非常廉价的，一个消费者的创建、销毁不会对集群或其他消费者产生多大的影响。

测试环境

目的：使用百度的doris，但因为doris版本在持续升级，为了保证新版本能够完全支持知乎的数据业务，先建立了一个测试的数据库，之后的doris更新也在这个上面先进行。

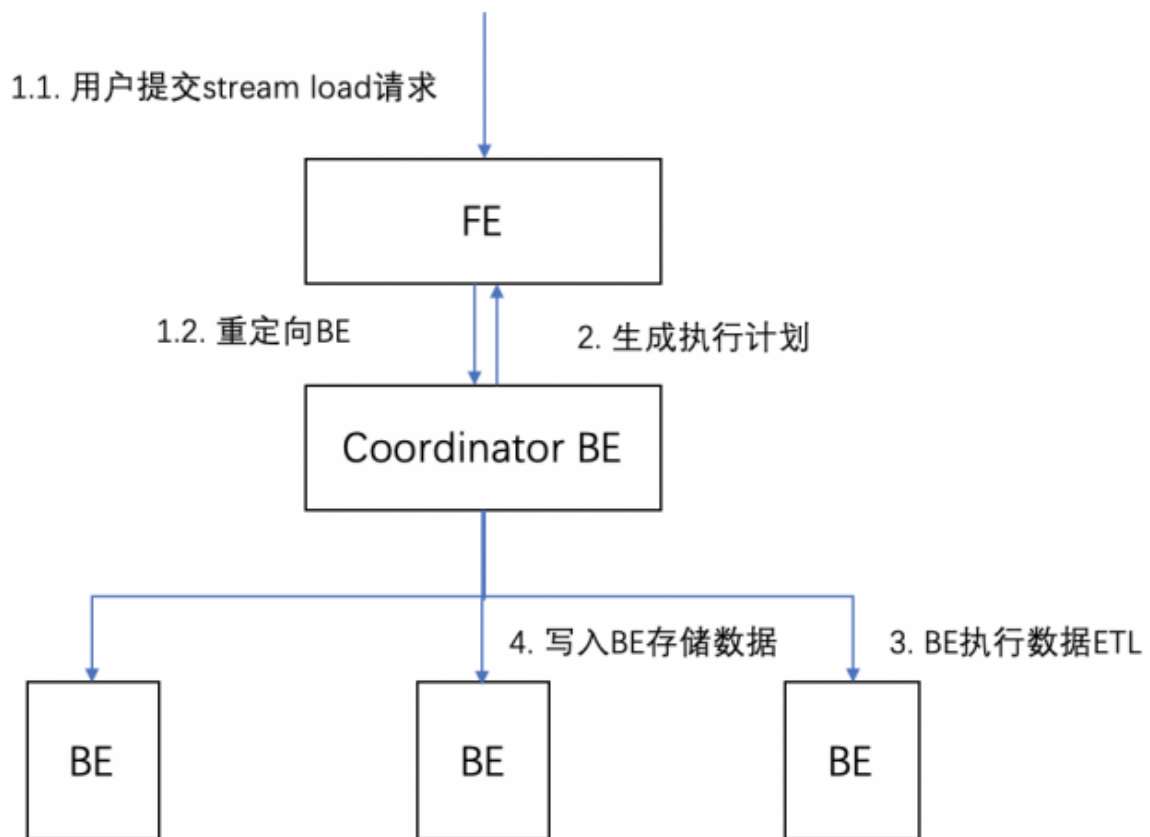
工作：将原数据库的几张表的数据导出到测试数据库中，并执行sql测试：是否出现问题；是否满足doris升级后的功能（如提升了查询效率）

doris系统架构

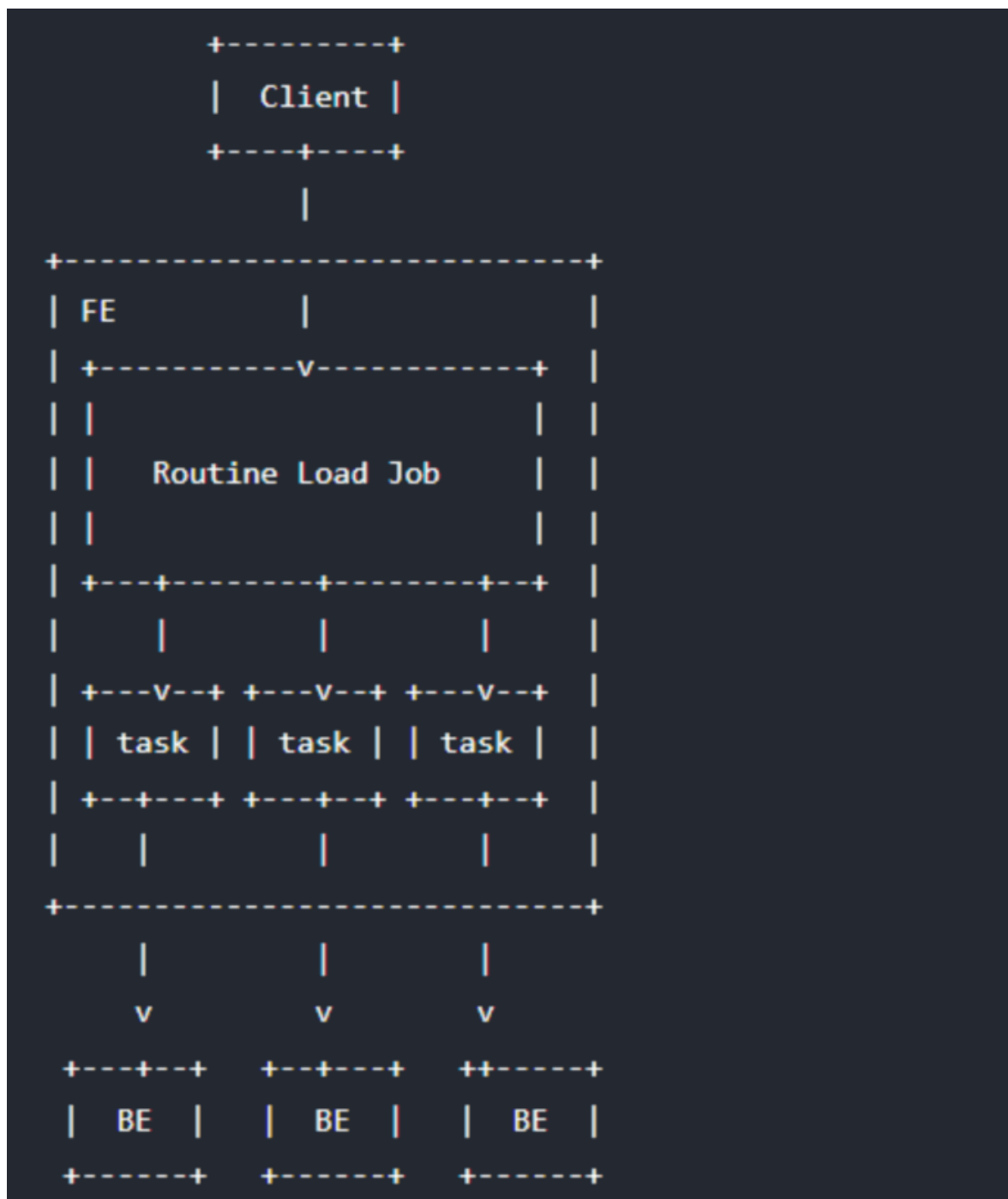
FE：即 Doris 的前端节点。主要负责接收和返回客户端请求、元数据以及集群管理、查询计划生成等工作。

BE：Backend，即 Doris 的后端节点。主要负责数据存储与管理、查询计划执行等工作。

broker：从BROKER中拉取



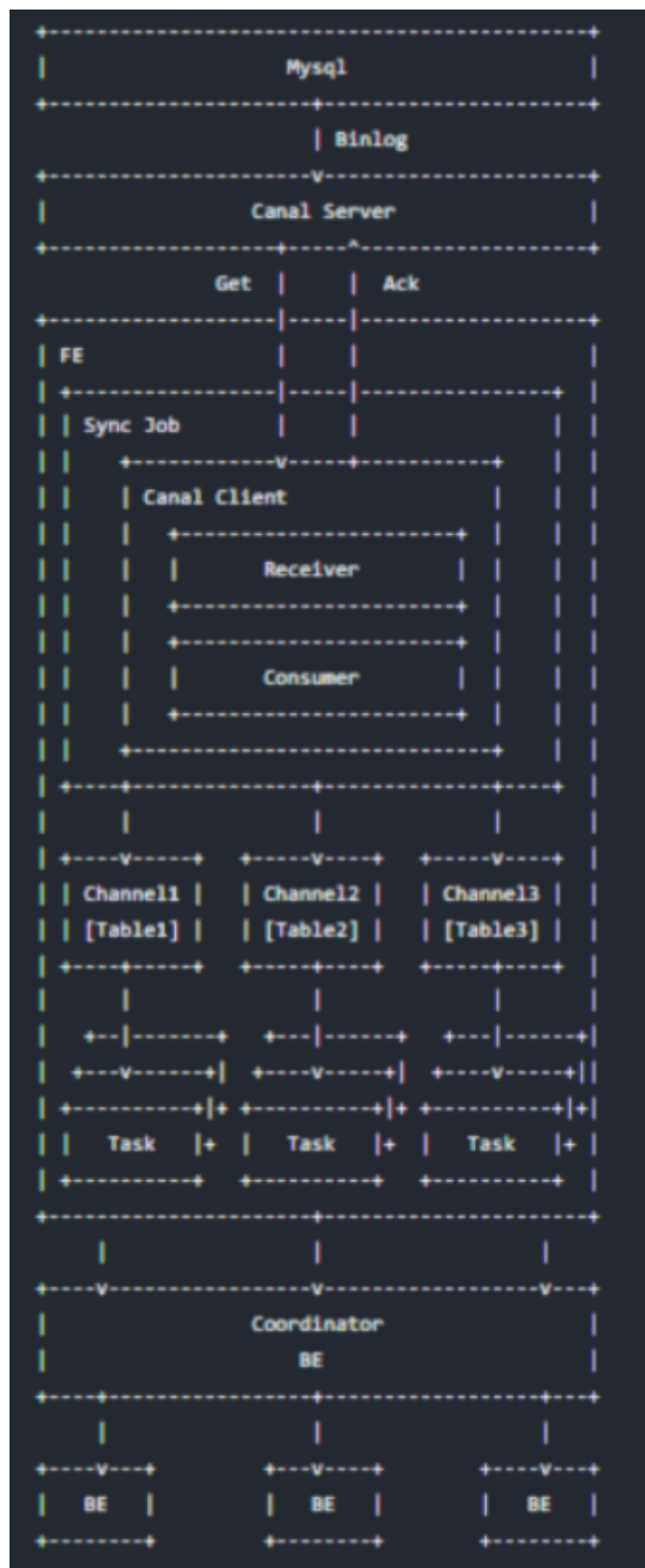
导入数据分为两种方式：实时数据从kafka或mysql binlog 中导入；doris的routineload能够从指定的数据源中导入数据



1. FE 通过 JobScheduler 将一个导入作业拆分成若干个 Task。每个 Task 负责导入指定的一部分数据。Task 被 TaskScheduler 分配到指定的 BE 上执行。
2. 在 BE 上，一个 Task 被视为一个普通的导入任务，通过 Stream Load 的导入机制进行导入。导入完成后，向 FE 汇报。
3. FE 中的 JobScheduler 根据汇报结果，继续生成后续新的 Task，或者对失败的 Task 进行重试。
4. 整个例行导入作业通过不断的产生新的 Task，来完成数据不间断的导入。

在使用routine load时，doris可以支持从kafka的某个partition 和offset开始消费

或者从mysql 的 binlog 订阅，增量同步用户在MYSQL数据库中对数据的更新操作CDC（实时数据）数据更新

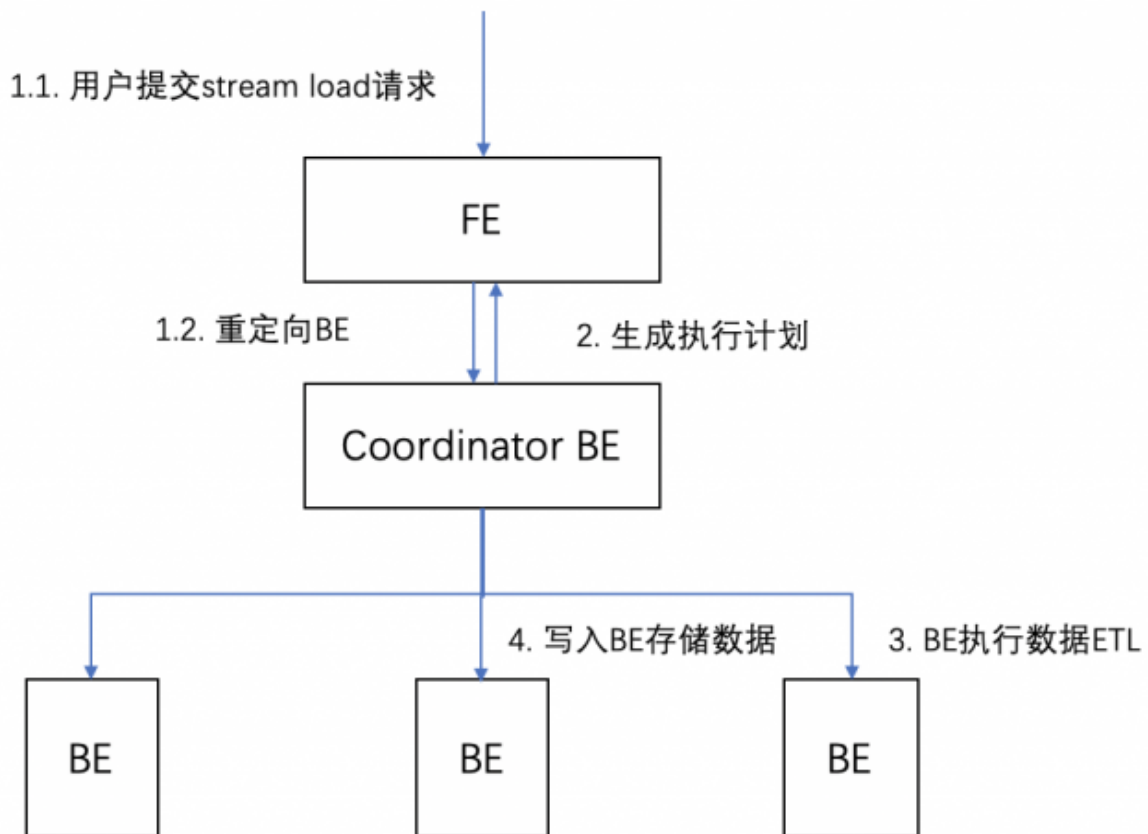


解析binlog数据需要搭建一个中间的canal层从节点，doris从canal上获取mysql主节点的binlog并解析

1. FE会为每个数据同步作业启动一个canal client，来向canal server端订阅并获取数据。
2. client中的receiver将负责通过Get命令接收数据，每获取到一个数据batch，都会由consumer根据对应表分发到不同的channel，每个channel都会为此数据batch产生一个发送数据的子任务Task。
3. 在FE上，一个Task是channel向BE发送数据的子任务，里面包含分发到当前channel的同一个batch的数据。
4. channel控制着单个表事务的开始、提交、终止。一个事务周期内，一般会从consumer获取到多个batch的数据，因此会产生多个向BE发送数据的子任务Task，在提交事务成功前，这些Task不会实际生效。

5. 满足一定条件时（比如超过一定时间、达到提交最大数据大小），consumer将会阻塞并通知各个channel提交事务。
6. 当且仅当所有channel都提交成功，才会通过Ack命令通知canal并继续获取并消费数据。
7. 如果有任意channel提交失败，将会重新从上一次消费成功的位置获取数据并再次提交（已提交成功的channel不会再次提交以保证幂等性）。
8. 整个数据同步作业中，FE通过以上流程不断的从canal获取数据并提交到BE，来完成数据同步。

streamload写入数据



doris底层数据存储

doris底层中存在segment文件，包括：

Data Region：用于存储各个列的数据信息，这里的数据是按需分page加载的

Index Region: doris中将各个列的index数据统一存储在Index Region，这里的数据会按照列粒度进行加载，所以跟列的数据信息分开存储

文件中的数据按照page的方式进行组织，page是编码和压缩的基本单位。

data page：存储通过encoding和compress之后的数据，分为nullable和non-nullable两种

Bloom Filter Pages

针对每个bloom filter列,会在page的粒度相应的生成一个bloom filter的page，保存在bloom filter pages区域

Ordinal Index Page

针对每个列，都会按照page粒度，建立行号的稀疏索引。内容为这个page的起始行的行号到这个block的指针（包括offset和length）

Short Key Index page

我们会每隔N行（可配置）生成一个short key的稀疏索引，索引的内容为：short key->行号(ordinal)

了解过大数据吗

Mapreduce方法：

MapReduce合并了两种经典函数：

- 映射（Mapping）对集合里的每个目标应用同一个操作。即，如果你想把表单里每个单元格乘以二，那么把这个函数单独地应用在每个单元格上的操作就属于mapping。这是一个化繁为简的过程，对复杂的输入进行分步简单的操作；

简单的操作的特点：

- 一是数据或计算的规模相对原任务要大大**缩小**；
- 二是**就近计算原则**，即任务会分配到存放着所需数据的节点上进行计算；
- 三是这些小任务**可以并行计算，彼此间几乎没有依赖关系**。

- 化简（Reducing）遍历集合中的元素来返回一个综合的结果。即，输出表单里一列数字的和这个任务属于reducing。对前面分步的结果进行汇总得到输出

并行处理数据----->合并处理的结果

华为

使用python语言，对开发的网络产品进行补丁规则检查，检查的结果输出为检查报告

主要分几种规则类型

1. docker_tag检查：对比了网关产品中两个文件夹下docker tag，如果有不一致的，则在基础版本中找到对应docker tag并检查是否一致，不一致则记录报错信息
2. 字段检查：对网关产品中的文件的特定字段进行检查，查看是否为空、是否与当前产品版本号一致等功能

除此之外，因为产品种类不同、文件结构和某些字段的约束不同，检查工具对其他版本的网关产品也做了适配

空地一体化数据系统简介

这个系统中，有一个数据表存储了飞机的航迹信息，航迹信息包括很多字段；这个系统主要的功能就是完成在客户端界面配置查询条件，并提交到后端程序中完成sql拼接，执行和数据处理；

我负责的主要是数据库的搭建、使用qt与oracle数据库进行连接和sql脚本的编写

索引：在select和where中出现较多的字段上加索引了

系统可靠性：会对查询条件的输入数据合法性进行检验：例如检查是否为数字值、是否在范围内；否则会提醒用户非法输入，并建议重新输入查询条件

程序是否崩溃：关于数据量过大：我们对数据量进行了限制，因为是死数据不会有新增，限制了一次查询只能查询最多5000条记录，如果用户需要多于5000条记录，则需要用户批量输入查询条件

分布式系统概要

节点：在具体的工程项目中，一个节点往往是一个操作系统上的进程。

副本：指在分布式系统中为数据或服务提供的冗余。对于数据副本指在不同的节点上持久化同一份数据，当出现某一个节点的存储的数据丢失时，可以从副本上读到数据。数据副本是分布式系统解决数据丢失异常的唯一手段。另一类副本是服务副本，指数个节点提供某种相同的服务，这种服务一般并不依赖于节点的本地存储，其所需数据一般来自其他节点。

分布式系统通过副本控制协议，使得从系统外部读取系统内部各个副本的数据在一定的约束条件下相同，称之为副本一致性

- 强一致性 (strong consistency)：任何时刻任何用户或节点都可以读到最近一次成功更新的副本数据。强一致性是程度最高的一致性要求，也是实践中最难以实现的一致性。
- 单调一致性 (monotonic consistency)：任何时刻，任何用户一旦读到某个数据在某次更新后的值，这个用户不会再读到比这个值更旧的值。单调一致性是弱于强一致性却非常实用的一种一致性级别。因为通常来说，用户只关心从己方视角观察到的一致性，而不会关注其他用户的一致性情况。
- 会话一致性 (session consistency)：任何用户在某一次会话内一旦读到某个数据在某次更新后的值，这个用户在这次会话过程中不会再读到比这个值更旧的值。会话一致性通过引入会话的概念，在单调一致性的基础上进一步放松约束，会话一致性只保证单个用户单次会话内数据的单调修改，对于不同用户间的一致性和同一用户不同会话间的一致性没有保障。实践中有许多机制正好对应会话的概念，例如PHP中的session概念。
- 最终一致性 (eventual consistency)：最终一致性要求一旦更新成功，各个副本上的数据最终将达到完全一致的状态，但达到完全一致状态所需要的时间不能保障。对于最终一致性系统而言，一个用户只要始终读取某一个副本的数据，则可以实现类似单调一致性的效果，但一旦用户更换读取的副本，则无法保障任何一致性。
- 弱一致性 (weak consistency)：一旦某个更新成功，用户无法在一个确定时间内读到这次更新的值，且即使在某个副本上读到了新的值，也不能保证在其他副本上可以读到新的值。弱一致性系统一般很难在实际中使用，使用弱一致性系统需要应用方做更多的工作从而使得系统可用。

分布式系统的原理

单机系统与分布式系统的最大的区别在于问题的规模，即计算、存储的数据量的区别。将一个单机问题使用分布式解决，首先要解决的就是如何将问题拆解为可以使用多机分布式解决，使得分布式系统中的每台机器负责原问题的一个子集。由于无论是计算还是存储，其问题输入对象都是数据，所以如何拆解分布式系统的输入数据成为分布式系统的基本问题。

hash分布的问题：1.可扩展性不高：一旦集群规模需要扩展，则几乎所有的数据需要被迁移并重新分布。工程中，扩展哈希分布数据的系统时，往往使得集群规模成倍扩展，按照数据重新计算哈希，这样原本一台机器上的数据只需迁移一半到另一台对应的机器上即可完成扩展。 2.出现数据倾斜问题：某数据的特征值不均，则会出现某个特征值下出现许多元素

其他

ES搜索时各个节点怎么工作的

Elasticsearch ⇒ 索引(Index) ⇒ 类型(type) ⇒ 文档(Documents) ⇒ 字段(Fields)

Elasticsearch的交互，可以使用Java API，也可以直接使用HTTP的Restful API方式，比如我们打算插入一条记录，可以简单发送一个HTTP的请求：

```
PUT /megacorp/employee/1 { "name": "John", "sex": "Male", "age": 25, "about": "I love to go rock climbing", "interests": [ "sports", "music" ] }
```

ES：倒排索引

Elasticsearch分别为每个field都建立了一个倒排索引，Kate, John, 24, Female这些叫term，而[1,2]就是**Posting List**。Posting list就是一个int的数组，存储了所有符合某个term的文档id

客户端节点

当主节点和数据节点配置都设置为false的时候，**该节点只能处理路由请求，处理搜索，分发索引操作等**，从本质上来说该客户节点表现为智能负载均衡器。

数据节点

数据节点主要是存储索引数据的节点，**主要对文档进行增删改查操作，聚合操作等**。

主节点

主资格节点的主要职责是和集群操作相关的内容，如**创建或删除索引，跟踪哪些节点是群集的一部分，并决定哪些分片分配给相关的节点**。

讲一下微服务

<https://www.cnblogs.com/imyalost/p/6792724.html>

一、微服务架构介绍

微服务架构（Microservice Architecture）是一种架构概念，旨在通过将功能分解到各个离散的服务中以实现**对解决方案的解耦**。你可以将其看作是在架构层次而非获取服务的

类上应用很多SOLID原则。微服务架构是个很有趣的概念，它的主要作用是将功能分解到离散的各个服务当中，从而降低系统的耦合性，并提供更加灵活的服务支持。

概念：把一个大型的单个应用程序和服务拆分为数个甚至数十个的支持微服务，它可扩展单个组件而不是整个的应用程序堆栈，从而满足服务等级协议。

定义：围绕业务领域组件来创建应用，这些应用可独立地进行开发、管理和迭代。在分散的组件中使用云架构和平台式部署、管理和服务功能，使产品交付变得更加简单。

本质：用一些功能比较明确、业务比较精练的服务去解决更大、更实际的问题。

先来看看传统的web开发方式，通过对比比较容易理解什么是Microservice Architecture。和Microservice相对应的，这种方式一般被称为Monolithic（单体式开发）。

所有的功能打包在一个 WAR包里，基本没有外部依赖（除了容器），部署在一个JEE容器（Tomcat, JBoss, WebLogic）里，包含了 DO/DAO, Service, UI等所有逻辑。

微服务的开发：有效拆分应用，实现敏捷开发和部署

- 1、一些列的独立的服务共同组成系统
- 2、单独部署，跑在自己的进程中
- 3、每个服务为独立的业务开发
- 4、分布式管理
- 5、非常强调隔离性

ES了解多少

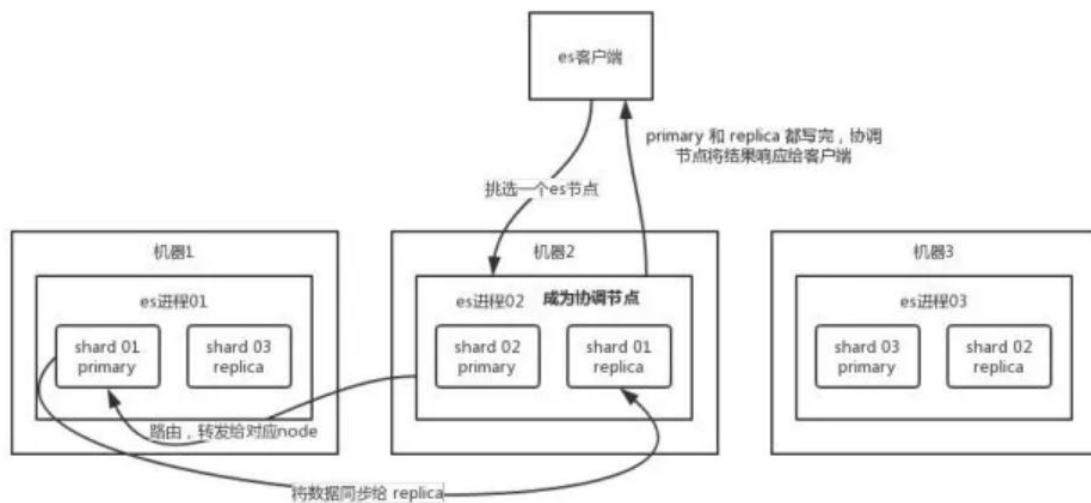
es 写入数据的工作原理是什么啊？ es 查询数据的工作原理是什么啊？底层的 [lucene](#) 介绍一下呗？倒排索引了解吗？

看看你了解不了解 es 的一些基本原理，因为用 es 无非就是写入数据，搜索数据。

es 写数据过程

- 客户端选择一个 node 发送请求过去，这个 node 就是 `coordinating node`（协调节点）。

- `coordinating node` 对 document 进行路由，将请求转发给对应的 node（有 primary shard）。[路由的算法是？]
- 实际的 node 上的 `primary shard` 处理请求，然后将数据同步到 `replica node`。
- `coordinating node` 如果发现 `primary node` 和所有 `replica node` 都搞定之后，就返回响应结果给客户端。



<https://blog.csdn.net/abcd1101>

es 读数据过程

可以通过 `doc id` 来查询，会根据 `doc id` 进行 hash，判断出来当时把 `doc id` 分配到了哪个 shard 上面去，从那个 shard 去查询。

- 客户端发送请求到任意一个 node，成为 `coordinate node`。
- `coordinate node` 对 `doc id` 进行哈希路由，将请求转发到对应的 node，此时会使用 `round-robin` 随机轮询算法，在 `primary shard` 以及其所有 `replica` 中随机选择一个，让读请求负载均衡。
- 接收请求的 node 返回 document 给 `coordinate node`。
- `coordinate node` 返回 document 给客户端。

写请求是写入 `primary shard`，然后同步给所有的 `replica shard`；读请求可以从 `primary shard` 或 `replica shard` 读取，采用的是随机轮询算法。

10.基础查表建表问题

建表要满足第三范式

反向代理和正向代理

设计模式中的代理模式：给某个对象提供一个代理对象，并由代理对象控制原对象的引用。

代理简单来说，就是如果我们想做什么，但又不想直接去做，那么这时候就找另外一个人帮我们去做。那么这个例子里面的中介公司就是给我们做代理服务的，我们委托中介公司帮我们找房子。

MD5的长度

32位

`char Ptr=0; Ptr='a'` 说明一下内存分配流程

golang内存的分配流程

```

1 func pointer_test() {
2     //空指针，输出为nil
3     var p *int
4     fmt.Printf("p: %v\n", p)
5     //指向局部变量，变量值初始为0
6     var i int
7     p = &i
8     fmt.Printf("p: %v,%v\n", p, *p)
9     //通过指针修改变量数值
10    *p = 8
11    fmt.Printf("p: %v,%v\n", p, *p)
12    //数组的初始化及输出
13    m := [3]int{3, 4, 5}
14    fmt.Printf("m:%v--%v,%v,%v\n", m, m[0], m[1], m[2])
15    //指针数组的初始化及输出
16    //j, k, l := 3, 4, 5
17    //x := [3]*int{&j, &k, &l}
18    x := [3]*int{&m[0], &m[1], &m[2]}
19    fmt.Printf("x:%v,%v,%v\n", x[0], x[1], x[2])
20    fmt.Printf("*x:%v,%v,%v\n", *x[0], *x[1], *x[2])
21    var n [3]*int
22    n = x
23    fmt.Printf("n:%v,%v,%v\n", n[0], n[1], n[2])
24    //指向数组的指针，也即二级指针的使用
25    y := []*[3]*int{&x}
26    fmt.Printf("y:%v,%v\n", y, y[0])
27    fmt.Printf("*y[0]:%v\n", *y[0])
28    fmt.Printf("*y[][0]:%v,%v,%v\n", *y[0][0], *y[0][1], *y[0][2])
29    /*
30        *X          ->    v
31        [3]X          ->    [v0][v1][v2]
32        []X           ->    [v0]...[vi]
33        [3]*X         ->    [p0][p1][p2]
34                        |   |   |
35                        j    k    l
36        []*[3]*X      ->    [n0]...[ni]
37                        | ... |
38                        [p0]->j
39                        [p1]->k
40                        [p2]->l
41    */
42    }
43
44    type Student struct {
45        name string
46        id   int
47        score uint
48    }
49
50    func memery_test() {
51        //new分配出来的数据是指针形式
52        p := new(Student)
53        p.name = "China"
54        p.id = 63333
55        p.score = 99
56        fmt.Println(*p)
57        //var定义的变量是数值形式

```

```

58     var st Student
59     st.name = "Chinese"
60     st.id = 666333
61     st.score = 100
62     fmt.Println(st)
63     //make分配slice、map和channel的空间，并且返回的不是指针
64     var ptr *[]Student
65     fmt.Println(ptr)      //ptr == nil
66     ptr = new([]Student) //指向一个空的slice
67     fmt.Println(ptr)
68     *ptr = make([]Student, 3, 100)
69     fmt.Println(ptr)
70     stu := []Student{{"China", 3333, 66}, {"Chinese", 4444, 77}, {"Chince",
5555, 88}}
71     fmt.Println(stu)
72 }
73
74

```

有几个相同的服务 A、B、C，挂了一个 A，REST API 怎么知道 A 挂了并调用 B

设计模式

单例模式

懒汉式：非线程安全的

在多线程下可能会创建多次对象

```

1 //使用结构体代替类
2 type Tool struct {
3     values int
4 }
5
6 //建立私有变量
7 var instance *Tool
8
9 //获取单例对象的方法，引用传递返回
10 func GetInstance() *Tool {
11     if instance == nil {
12         instance = new(Tool)
13     }
14     return instance
15 }

```

饿汉式

直接创建好对象，不需要判空，同时也是线程安全的，但是会在导入包的时候就会创建对象，并持续占有在内存中

```

1 type cfg struct {}
2 var cfg *config
3 func init() {
4     cfg = new(config)
5 }

```

```

6
7 // NewConfig 提供获取实例的方法
8 func NewConfig() *config {
9     return cfg
10 }
11 type config struct {}
12
13 //全局变量
14 var cfg *config = new(config)
15 // NewConfig 提供获取实例的方法
16 func NewConfig() *config {
17     return cfg
18 }

```

sync.Once()

确保创建对象的方法只执行一次

```

1 var once sync.Once
2
3 func GetInstance() *Tool {
4     once.Do(func() {
5         instance = new(Tool)
6
7     })
8     return instance
9 }

```

工厂模式

简单工厂模式

```

1 package simplefactory
2
3 import "fmt"
4
5 //API is interface
6 type API interface {
7     Say(name string) string
8 }
9
10 //NewAPI return Api instance by type
11 func NewAPI(t int) API {
12     if t == 1 {
13         return &hiAPI{}
14     } else if t == 2 {
15         return &helloAPI{}
16     }
17     return nil
18 }
19
20 //hiAPI is one of API implement
21 type hiAPI struct{}
22
23 //Say hi to name
24 func (*hiAPI) Say(name string) string {
25     return fmt.Sprintf("Hi, %s", name)
26 }

```

```

26 }
27
28 //HelloAPI is another API implement
29 type helloAPI struct{}
30
31 //Say hello to name
32 func (*helloAPI) Say(name string) string {
33     return fmt.Sprintf("Hello, %s", name)
34 }
35
36 package simplefactory
37
38 import "testing"
39
40 //TestType1 test get hiapi with factory
41 func TestType1(t *testing.T) {
42     api := NewAPI(1)
43     s := api.Say("Tom")
44     if s != "Hi, Tom" {
45         t.Fatal("Type1 test fail")
46     }
47 }
48
49 func TestType2(t *testing.T) {
50     api := NewAPI(2)
51     s := api.Say("Tom")
52     if s != "Hello, Tom" {
53         t.Fatal("Type2 test fail")
54     }
55 }

```

工厂方法模式

工厂方法模式使用子类的方式延迟生成对象到子类中实现。

Go中不存在继承 所以使用匿名组合来实现

```

1 package factorymethod
2
3 //Operator 是被封装的实际类接口
4 type Operator interface {
5     SetA(int)
6     SetB(int)
7     Result() int
8 }
9
10 //OperatorFactory 是工厂接口
11 type OperatorFactory interface {
12     Create() Operator
13 }
14
15 //OperatorBase 是Operator 接口实现的基类，封装公用方法
16 type OperatorBase struct {
17     a, b int
18 }
19
20 //SetA 设置 A
21 func (o *OperatorBase) SetA(a int) {

```

```
22     o.a = a
23 }
24
25 //SetB 设置 B
26 func (o *OperatorBase) SetB(b int) {
27     o.b = b
28 }
29
30 //PlusOperatorFactory 是 PlusOperator 的工厂类
31 type PlusOperatorFactory struct{}
32
33 func (PlusOperatorFactory) Create() Operator {
34     return &PlusOperator{
35         OperatorBase: &OperatorBase{},
36     }
37 }
38
39 //PlusOperator Operator 的实际加法实现
40 type PlusOperator struct {
41     *OperatorBase
42 }
43
44 //Result 获取结果
45 func (o PlusOperator) Result() int {
46     return o.a + o.b
47 }
48
49 //MinusOperatorFactory 是 MinusOperator 的工厂类
50 type MinusOperatorFactory struct{}
51
52 func (MinusOperatorFactory) Create() Operator {
53     return &MinusOperator{
54         OperatorBase: &OperatorBase{},
55     }
56 }
57
58 //MinusOperator Operator 的实际减法实现
59 type MinusOperator struct {
60     *OperatorBase
61 }
62
63 //Result 获取结果
64 func (o MinusOperator) Result() int {
65     return o.a - o.b
66 }
67
68 //测试函数
69 package factorymethod
70
71 import "testing"
72
73 func compute(factory OperatorFactory, a, b int) int {
74     op := factory.Create()
75     op.SetA(a)
76     op.SetB(b)
77     return op.Result()
78 }
79
```

```

80 func TestOperator(t *testing.T) {
81     var (
82         factory OperatorFactory
83     )
84
85     factory = PlusOperatorFactory{}
86     if compute(factory, 1, 2) != 3 {
87         t.Fatal("error with factory method pattern")
88     }
89
90     factory = MinusOperatorFactory{}
91     if compute(factory, 4, 2) != 2 {
92         t.Fatal("error with factory method pattern")
93     }
94 }
95

```

抽象工厂模式

抽象工厂模式用于生成产品族的工厂，所生成的对象是有关联的。

如果抽象工厂退化成生成的对象无关联则成为工厂函数模式。

比如本例子中使用RDB和XML存储订单信息，抽象工厂分别能生成相关的主订单信息和订单详情信息。如果业务逻辑中需要替换使用的时候只需要改动工厂函数相关的类就能替换使用不同的存储方式了。

```

1  package abstractfactory
2
3  import "fmt"
4
5  //OrderMainDAO 为订单主记录
6  type OrderMainDAO interface {
7      SaveOrderMain()
8  }
9
10 //OrderDetailDAO 为订单详情纪录
11 type OrderDetailDAO interface {
12     SaveOrderDetail()
13 }
14
15 //DAOFactory DAO 抽象模式工厂接口
16 type DAOFactory interface {
17     CreateOrderMainDAO() OrderMainDAO
18     CreateOrderDetailDAO() OrderDetailDAO
19 }
20
21 //RDBMainDAP 为关系型数据库的OrderMainDAO实现
22 type RDBMainDAO struct{}
23
24 //SaveOrderMain ...
25 func (*RDBMainDAO) SaveOrderMain() {
26     fmt.Print("rdb main save\n")
27 }
28
29 //RDBDetailDAO 为关系型数据库的OrderDetailDAO实现
30 type RDBDetailDAO struct{}
31
32 // SaveOrderDetail ...

```



```
33 func (*RDBDetailDAO) SaveOrderDetail() {
34     fmt.Print("rdb detail save\n")
35 }
36
37 //RBDDAOFactory 是RDB 抽象工厂实现
38 type RBDDAOFactory struct{}
39
40 func (*RBDDAOFactory) CreateOrderMainDAO() OrderMainDAO {
41     return &RDBMainDAO{}
42 }
43
44 func (*RBDDAOFactory) CreateOrderDetailDAO() OrderDetailDAO {
45     return &RDBDetailDAO{}
46 }
47
48 //XMLMainDAO XML存储
49 type XMLMainDAO struct{}
50
51 //SaveOrderMain ...
52 func (*XMLMainDAO) SaveOrderMain() {
53     fmt.Print("xml main save\n")
54 }
55
56 //XMLDetailDAO XML存储
57 type XMLDetailDAO struct{}
58
59 // SaveOrderDetail ...
60 func (*XMLDetailDAO) SaveOrderDetail() {
61     fmt.Print("xml detail save")
62 }
63
64 //XMLDAOFactory 是RDB 抽象工厂实现
65 type XMLDAOFactory struct{}
66
67 func (*XMLDAOFactory) CreateOrderMainDAO() OrderMainDAO {
68     return &XMLMainDAO{}
69 }
70
71 func (*XMLDAOFactory) CreateOrderDetailDAO() OrderDetailDAO {
72     return &XMLDetailDAO{}
73 }
74
75
76 //test
77 package abstractfactory
78
79 func getMainAndDetail(factory DAOFactory) {
80     factory.CreateOrderMainDAO().SaveOrderMain()
81     factory.CreateOrderDetailDAO().SaveOrderDetail()
82 }
83
84 func ExampleRdbFactory() {
85     var factory DAOFactory
86     factory = &RBDDAOFactory{}
87     getMainAndDetail(factory)
88     // Output:
89     // rdb main save
90     // rdb detail save
```

```

91 }
92
93 func ExampleXmlFactory() {
94     var factory DAOFactory
95     factory = &XMLDAOFactory{}
96     getMainAndDetail(factory)
97     // Output:
98     // xml main save
99     // xml detail save
100 }
101

```

生产者消费者模式

单向channel最典型的应用是“生产者消费者模型”

所谓“生产者消费者模型”: 某个模块（函数等）负责产生数据，这些数据由另一个模块来负责处理（此处的模块是广义的，可以是类、函数、协程、线程、进程等）。产生数据的模块，就形象地称为生产者；而处理数据的模块，就称为消费者。

单单抽象出生产者和消费者，还够不上是生产者 / 消费者模型。该模式还需要有一个缓冲区处于生产者和消费者之间，作为一个中介。生产者把数据放入缓冲区，而消费者从缓冲区取出数据。

```

1 package main
2
3 import "fmt"
4
5 //import "time"
6
7 //此通道只能写，不能读
8 func producer(out chan<- int){
9     for i:=0;i<10;i++){
10         out<-i*i
11     }
12     close(out)
13 }
14
15 //此通道只能读，不能写
16 func consumer(in <-chan int){
17     for num:=range in{
18         fmt.Println("num=",num)
19     }
20 }
21
22 func main(){
23     //创建双向，channel
24     ch:=make(chan int)
25
26     //生产者，生产数字，写入channel
27     //新开一个协程
28     go producer(ch) //channel传参，引用传递
29
30     //消费者，从channel读取内存，打印
31     consumer(ch)
32 }

```

代理模式

由于某些原因需要给某对象提供一个代理以控制对该对象的访问。这时，访问对象不适合或者不能直接引用目标对象，代理对象作为访问对象和目标对象之间的中介。

静态代理模式：

角色分析：

- 1、抽象角色：一般会使用接口或抽象类来解决
- 2、真实角色：被代理的角色
- 3、代理角色：代理真实角色，代理真实角色后我们会进行一些附属操作
- 4、访问角色：访问代理对象的人

```
1 //租房
2 public interface Rent {
3     void rent();
4 }
5 //房东
6 public class Master implements Rent{
7     @Override
8     public void rent() {
9         System.out.println("Master rent");
10    }
11 }
12 //中介
13 public class Proxy implements Rent{
14     private Master master;
15
16     public Proxy() {
17     }
18
19     public Proxy(Master master) {
20         this.master = master;
21     }
22
23     @Override
24     public void rent() {
25         see();
26         master.rent();
27         fare();
28     }
29 //看房
30 public void see(){
31     System.out.println("see");
32 }
33 //收费
34 public void fare(){
35     System.out.println("fare");
36 }
37 }
38 //测试类
39 public class Consumer {
40     public static void main(String[] args) {
41         Master master = new Master();
42         //进行代理
```

```

43         Proxy proxy = new Proxy(master);
44         //不需要通过对象，直接通过代理完成响应业务
45         proxy.rent();
46     }
47 }
48

```

可以从上述代码看出，我们通过创建中介这一代理完成了租房，并且还有看房、收费的附属操作。我们不需要使用房东对象，通过使用代理中介就可以完成操作。

代理模式优点：可以使真实角色的操作更加纯粹!不用去关注一些公共的业务，公共也就可以交给代理角色，实现了业务的分工，公共业务发生扩展的时候，方便集中管理!

代理模式缺点：一个真实角色就会产生一个代理角色;代码量会翻倍开发效率会变低，也许，这样无法理解到代理模式的好处。举个例子也许能更好理解，比如说我们想要在原有固定功能上新增业务，按照开闭原则我们是不能对原有代码进行修改的。但是我们可以通过代理模式，增加代理，在实现原有功能的情况下写入新的功能，创建对象时也就可以使用代理，完成操作。

动态代理模式：虽然静态代理模式可以很好的解决开闭原则，但是每有一个真实角色就会产生一个代理，代码量翻倍过于臃肿，开发效率较低。因此，我们就使用动态代理模式进行设计。

优点：①可以使真实角色的操作更加纯粹!不用去关注一些公共的业务。②公共也就可以交给代理角色!实现了业务的分工。③公共业务发生扩展的时候，方便集中管理。④一个动态代理类代理的是一个接口，一般就是对应的一类业务。⑤一个动态代理类可以代理多个类，只要是实现了同一个接口即可!

容器k8s

k8s相关

生命周期

pod对象从其创建开始到终止退出的时间范围称为生命周期

创建容器-初始化容器-启动后钩子-存活性探测-就绪性探测-success/delete

pod在生命周期中只会被调用一次，并且一旦被分配到某个节点，就会一直在该节点运行直到被终止阶段

pending：pod已经被k8s接收，但是有一个或多个容器尚未创建或运行

running：pod已经绑定到某个节点，并且pod中所有的容器都被创建，至少有一个容器在运行

succeed：pod中所有容器都已成功终止，不会再重启

failed：pod中所有容器都已终止，并且至少有一个容器是因为失败终止

unknown：因为某些原因无法取得pod的状态

pod调度过程

在做调度决定时需要考虑的因素包括：单独和整体的资源请求、硬件/软件/策略限制、亲和以及反亲和要求、数据局域性、负载间的干扰等等。

kube-scheduler给一个pod做调度选择时包含两个步骤

过滤

打分

然后将所有满足pod需求的node选出来

几种调度pod的方式：

与节点标签匹配的nodeSelector：k8s会为集群中节点添加一些标签，并让pod调度到特定的节点上，在运行上pod之间具有一些隔离性、安全性以及监管属性的节点上；nodeSelector字段添加到pod的规约中，并设置，目标节点中所需要的标签。k8s就会将pod调度到所指定的标签的节点上

亲和性和反亲和性：扩展了可以定义的约束类型，并且在语言表达能力和选择逻辑中更好表现

节点亲和性考虑权重，为每个亲和性类型的每个实例设置weight字段，调度器会遍历节点满足的所有偏好性规则并把对应表达式的weight相加；总分最高的优先级也最高

nodename：nodeName 是比亲和性或者 nodeSelector 更为直接的形式。nodeName 是 Pod 规约中的一个字段。如果 nodeName 字段不为空，调度器会忽略该 Pod，而指定节点上的 kubelet 会尝试将 Pod 放到该节点上。使用 nodeName 规则的优先级会高于使用 nodeSelector 或亲和性与非亲和性的规则。

结构组件

节点：将容器放在节点上执行的pod来执行工作负载，每个节点包括了运行pods所需要的服务

节点上的组件包括：kubelet、容器运行时以及kube-proxy

webbrowser：可视化操作

kubectI来接收Docker镜像进行部署

scheduler进行任务调度

Controller进行请求控制

API Server进行请求的统一网关处理

etcd用于存储集群中的网络及状态信息

Kubelet来接收控制器的处理任务

Container Registry用来存储镜像仓库

云计算的三种形式

本地部署：自己种菜，自己准备燃料，做饭过程中一切操作基本不依赖外界，自给自足。**对应到云计算中就是公司什么都自己准备好。**

IaaS：厨房是存储与服务器，燃料则是网络。你的公司运营中不需要操心服务器、存储、网络这些，因为他们就跟现代城市生活中的水电一样，**是云计算时代的基础设施。**

PaaS：这其中炊具是操作系统，餐具则是中间件。你的公司购买了平台服务，所以除了基础设施，服务商还会通过中间件**提供给你各种开发使用的解决方案，节省时间和资源。**

SaaS：普通消费者接触的基本就是这一层，最常见的就是各式web微应用。你甚至不需要自己准备数据（食材），**只需想好自己需要什么，然后想用就可以了。**

docker相关知识

<https://zhuanlan.zhihu.com/p/361449357>

Docker 是一个开源的应用容器引擎，让开发者可以打包他们的应用以及依赖包到一个可移植的镜像中，然后发布到任何流行的 Linux或Windows 机器上，也可以实现虚拟化。容器是完全使用沙箱机制，相互之间不会有任何接口。

2.什么是dockfile

Dockfile是一个用于编写docker镜像生成过程的文件，其有特定的语法。在一个文件夹中，如果一个名字为Dockfile的文件，其内容满足语法要求，在这个文件夹路径下执行命令:docker build --tag name:tag ., 就可以按照描述构建一个镜像了。name是镜像的名称，tag是镜像的版本或者是标签号，不写就是latest。注意后面有一个空格和点。

3.什么是docker-compose

假如，你有一个java镜像，一个mysql镜像，一个nginx镜像。如果没有docker-compose，那么每次启动的时候，你需要敲各个容器的启动参数，环境变量，容器命名，指定不同容器的链接参数等等一系列的操作，相当繁琐。而用了docker-composer之后，你就可以把这些命令一次性写在docker-composer.yml文件中，以后每次启动这一整个环境（含3个容器）的时候，你只要敲一个 `docker-composer up` 命令就ok了。

kafka的相关面试题

了解过消息队列吗

消息队列的使用场景：

解耦：发布订阅模式，使得总系统把需要的信息写在消息队列中，其他子系统通过订阅消息队列获取。总系统不用做更多改动

异步：非必要的程序使用异步执行，耗时变少

削峰：将用户提交的修改申请放在消息队列中，系统按照数据库最大能支持的并发数量从消息队列中慢慢拉取消息。高峰期积压变少，数据库连接异常不会出现

1 什么是kafka

Kafka是分布式发布-订阅消息系统，它最初是由LinkedIn公司开发的，之后成为Apache项目的一部分，Kafka是一个分布式，可划分的，冗余备份的持久性的日志服务，它主要用于处理流式数据。

kafka使用场景

- 日志收集：一个公司可以用Kafka可以收集各种服务的log，通过kafka以统一接口服务的方式开放给各种consumer，例如[hadoop](#)、Hbase、Solr等。
- 消息系统：解耦和生产者和消费者、缓存消息等。
- 用户活动跟踪：Kafka经常被用来记录web用户或者app用户的各种活动，如浏览网页、搜索、点击等活动，这些活动信息被各个服务器发布到kafka的topic中，然后订阅者通过订阅这些topic来做实时的监控分析，或者装载到hadoop、数据仓库中做离线分析和挖掘。
- 运营指标：Kafka也经常用来记录运营监控数据。包括收集各种分布式应用的数据，生产各种操作的集中反馈，比如报警和报告。
- 流式处理：比如[spark](#) streaming和storm
- 事件源

2 为什么要使用 kafka，为什么要使用消息队列

缓冲和削峰：上游数据时有突发流量，下游可能扛不住，或者下游没有足够多的机器来保证冗余，kafka在中间可以起到一个缓冲的作用，把消息暂存在kafka中，下游服务就可以按照自己的节奏进行慢慢处理。

解耦和扩展性：项目开始的时候，并不能确定具体需求。消息队列可以作为一个接口层，解耦重要的业务流程。只需要遵守约定，针对数据编程即可获得扩展能力。

冗余：可以采用一对多的方式，一个生产者发布消息，可以被多个订阅topic的服务消费到，供多个毫无关联的业务使用。

健壮性：消息队列可以堆积请求，所以消费端业务即使短时间死掉，也不会影响主要业务的正常进行。

异步通信：很多时候，用户不想也不需要立即处理消息。消息队列提供了异步处理机制，允许用户把一个消息放入队列，但并不立即处理它。想向队列中放入多少消息就放多少，然后在需要的时候再去处理它们。

kafka消费堆积问题排查

1.消费能力不足，导致消息堆积

可以在消费端开启多个线程（或线程池），对应topic的不同分区，处理多个消费请求。

2.上游数据激增，业务系统访问量激增（例如产生热点话题、热点新闻），导致大量数据涌入，有可能导致消息积压

解决方式：可以通过重置consumer的offset，不获取无效的历史数据，只读取实时数据

3.数据倾斜问题：producer 写入数据时候设置的key 发生数据倾斜，导致过度数据写入少量partition。

可以在producer端加盐，在消息数据前加随机数，使数据均匀写入partition，缓解倾斜问题。

3.Kafka中的ISR、AR又代表什么？ISR的伸缩又指什么

ISR:In-Sync Replicas 副本同步队列

AR:Assigned Replicas 所有副本

ISR是由leader维护，follower从leader同步数据有一些延迟（包括延迟时间replica.lag.time.max.ms和延迟条数replica.lag.max.messages两个维度，当前最新的版本0.10.x中只支持replica.lag.time.max.ms这个维度），任意一个超过阈值都会把follower剔除出ISR，存入OSR（Outof-Sync Replicas）列表，新加入的follower也会先存放在OSR中。AR=ISR+OSR。

4.kafka中的broker 是干什么的

broker 是消息的代理，Producers往Brokers里面的指定Topic中写消息，Consumers从Brokers里面拉取指定Topic的消息，然后进行业务处理，broker在中间起到一个代理保存消息的中转站。

5.kafka中的 zookeeper 起到什么作用，可以不用zookeeper么

zookeeper 是一个分布式的协调组件，早期版本的kafka用zk做meta信息存储，consumer的消费状态，group的管理以及 offset的值。考虑到zk本身的一些因素以及整个架构较大概率存在单点问题，新版本中逐渐弱化了zookeeper的作用。新的consumer使用了kafka内部的group coordination协议，也减少了对zookeeper的依赖，

但是broker依然依赖于ZK，zookeeper 在kafka中还用来选举controller 和 检测broker是否存活等等。

6.kafka follower如何与leader同步数据

Kafka的复制机制既不是完全的同步复制，也不是单纯的异步复制。完全同步复制要求All Alive Follower都复制完，这条消息才会被认为commit，这种复制方式极大的影响了吞吐率。而异步复制方式下，Follower异步的从Leader复制数据，数据只要被Leader写入log就被认为已经commit，这种情况下，如果leader挂掉，会丢失数据，kafka使用ISR的方式很好的均衡了确保数据不丢失以及吞吐率。Follower可以批量的从Leader复制数据，而且Leader充分利用磁盘顺序读以及send file(zero copy)机制，这样极大的提高复制性能，内部批量写磁盘，大幅减少了Follower与Leader的消息量差。

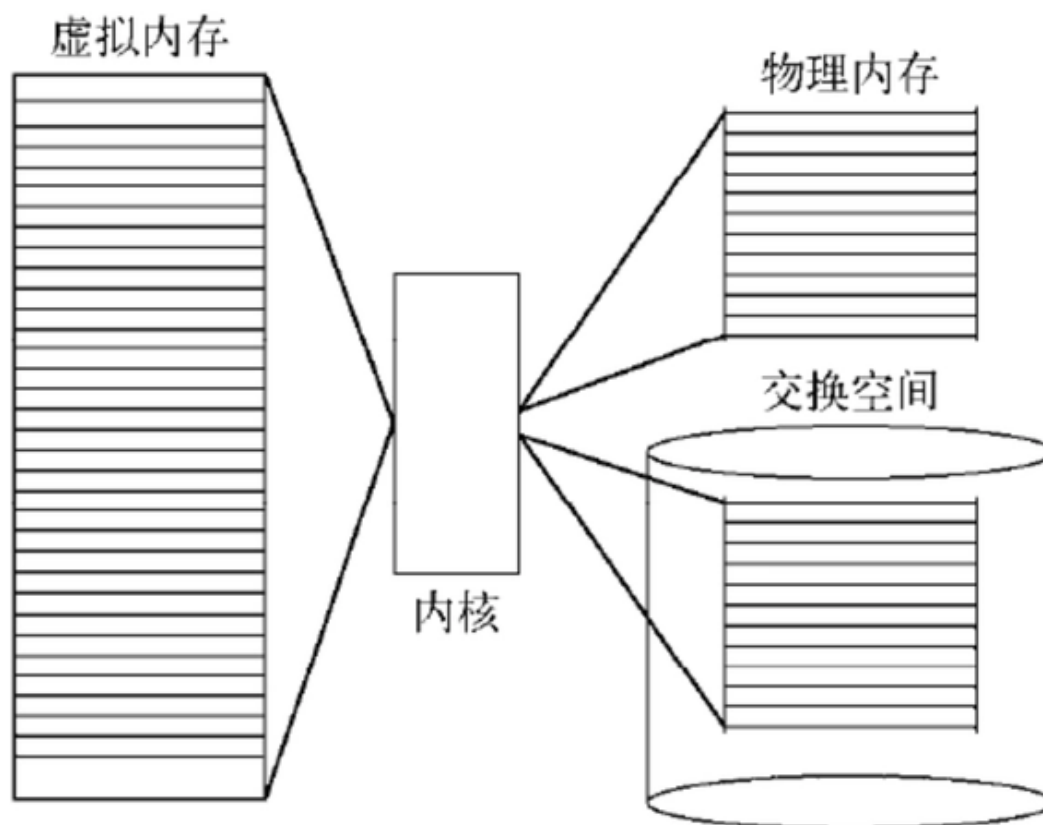
7.什么情况下一个 broker 会从 isr中踢出去

leader会维护一个与其基本保持同步的Replica列表，该列表称为ISR(in-sync Replica)，每个Partition都会有一个ISR，而且是由leader动态维护，如果一个follower比一个leader落后太多，或者超过一定时间未发起数据复制请求，则leader将其重ISR中移除。

8.kafka 为什么那么快

Cache Filesystem Cache PageCache缓存：所以 Kafka 的数据并不是实时的写入硬盘，它充分利用了现代操作系统分页存储来利用内存提高 I/O 效率。具体来说，就是把磁盘中的数据缓存到内存中，把对磁盘的访问变为对内存的访问。

MMAP：将磁盘文件映射到内存，用户通过修改内存就能修改磁盘文件。它的工作原理是直接利用操作系统的 Page 来实现磁盘文件到物理内存的直接映射。完成映射之后你对物理内存的操作会被同步到硬盘上（操作系统在适当的时候）。



通过 mmap，进程像读写硬盘一样读写内存（当然是虚拟机内存）。使用这种方式可以获取很大的 I/O 提升，省去了用户空间到内核空间复制的开销。

mmap 也有一个很明显的缺陷：不可靠，写到 mmap 中的数据并没有被真正的写到硬盘，操作系统会在程序主动调用 flush 的时候才把数据真正的写到硬盘。

顺序写 由于现代的操作系统提供了预读和写技术，磁盘的顺序写大多数情况下比随机写内存还要快。

Zero-copy 零拷技术减少拷贝次数

Batching of Data：批量化处理。合并小的请求，然后以流的方式进行交互，直顶网络上限。

9.kafka producer如何优化打入速度

增加线程

提高 batch.size

增加更多 producer 实例

增加 partition 数

设置 `acks=-1` 时，如果延迟增大：可以增大 `num.replica.fetchers` (follower 同步数据的线程数) 来调解；

跨数据中心的传输：增加 `socket` 缓冲区设置以及 `OS tcp` 缓冲区设置。

10.kafka producer 打数据，acks 为 0，1，-1 的时候代表啥，设置 -1 的时候，什么情况下，leader 会认为一条消息 commit 了

1 (默认) 数据发送到 Kafka 后，经过 leader 成功接收消息的确认，就算是发送成功了。在这种情况下，如果 leader 宕机了，则会丢失数据。

0 生产者将数据发送出去就不管了，不去等待任何返回。这种情况下数据传输效率最高，但是数据可靠性确是最低的。

-1 producer 需要等待 ISR 中的所有 follower 都确认接收到数据后才算一次发送完成，可靠性最高。当 ISR 中所有 Replica 都向 Leader 发送 ACK 时，leader 才 commit，这时候 producer 才能认为一个请求中的消息都 commit 了。

11.kafka unclean 配置代表啥，会对 spark streaming 消费有什么影响

`unclean.leader.election.enable` 为 true 的话，意味着非 ISR 集合的 broker 也可以参与选举，这样有可能就会丢数据，spark streaming 在消费过程中拿到的 end offset 会突然变小，导致 spark streaming job 挂掉。如果 `unclean.leader.election.enable` 参数设置为 true，就有可能发生数据丢失和数据不一致的情况，Kafka 的可靠性就会降低；而如果 `unclean.leader.election.enable` 参数设置为 false，Kafka 的可用性就会降低。

12.如果 leader crash 时，ISR 为空怎么办

kafka 在 Broker 端提供了一个配置参数：`unclean.leader.election`，这个参数有两个值：

true (默认)：允许不同步副本成为 leader，由于不同步副本的消息较为滞后，此时成为 leader，可能会出现消息不一致的情况。

false：不允许不同步副本成为 leader，此时如果发生 ISR 列表为空，会一直等待旧 leader 恢复，降低了可用性。

13.kafka 的 message 格式是什么样的

一个 Kafka 的 Message 由一个固定长度的 header 和一个变长的消息体 body 组成

header 部分由一个字节的 magic (文件格式) 和四个字节的 CRC32 (用于判断 body 消息体是否正常) 构成。

当 magic 的值为 1 的时候，会在 magic 和 crc32 之间多一个字节的 data：attributes (保存一些相关属性，

比如是否压缩、压缩格式等等)；如果 magic 的值为 0，那么不存在 attributes 属性

body 是由 N 个字节构成的一个消息体，包含了具体的 key/value 消息

14.kafka 中 consumer group 是什么概念

同样是逻辑上的概念，是 Kafka 实现单播和广播两种消息模型的手段。同一个 topic 的数据，会广播给不同的 group；同一个 group 中的 worker，只有一个 worker 能拿到这个数据。换句话说，对于同一个 topic，每个 group 都可以拿到同样的所有数据，但是数据进入 group 后只能被其中的一个 worker 消费。group 内的 worker 可以使用多线程或多进程来实现，也可以将进程分散在多台机器上，worker 的数量通常不超过 partition 的数量，且二者最好保持整数倍关系，因为 Kafka 在设计时假定了一个 partition 只能被一个 worker 消费 (同一 group 内)。

15.Kafka 中的消息是否会丢失和重复消费？

要确定 Kafka 的消息是否丢失或重复，从两个方面分析入手：消息发送和消息消费。

1、消息发送

Kafka消息发送有两种方式：同步（sync）和异步（async），默认是同步方式，可通过producer.type属性进行配置。Kafka通过配置request.required.acks属性来确认消息的生产：

0---表示不进行消息接收是否成功的确认；

1---表示当Leader接收成功时确认；

-1---表示Leader和Follower都接收成功时确认；

综上所述，有6种消息生产的情况（2种消息发送方式*3种acks值），下面分情况分析消息丢失的场景：

（1）acks=0，不和Kafka集群进行消息接收确认，则当网络异常、缓冲区满了等情况时，消息可能丢失；

（2）acks=1、同步模式下，只有Leader确认接收成功后但挂掉了，副本没有同步，数据可能丢失；

2、消息消费

Kafka消息消费有两个consumer接口，Low-level API和High-level API：

Low-level API：消费者自己维护offset等值，可以实现对Kafka的完全控制；

High-level API：封装了对partition和offset的管理，使用简单；

如果使用高级接口High-level API，可能存在一个问题就是当消息消费者从集群中把消息取出来、并提交了新的消息offset值后，还没来得及消费就挂掉了，那么下次再消费时之前没消费成功的消息就“诡异”的消失了；

解决办法：

针对消息丢失：同步模式下，确认机制设置为-1，即让消息写入Leader和Follower之后再确认消息发送成功；异步模式下，为防止缓冲区满，可以在配置文件设置不限制阻塞超时时间，当缓冲区满时让生产者一直处于阻塞状态；

针对消息重复：将消息的唯一标识保存到外部介质中，每次消费时判断是否处理过即可。

消息重复消费及解决参考：<https://www.javazhiyin.com/22910.html>

16.为什么Kafka不支持读写分离？

在Kafka中，生产者写入消息、消费者读取消息的操作都是与leader副本进行交互的，从而实现的是主写主读的生产消费模型。

Kafka并不支持主写从读，因为主写从读有2个很明显的缺点：

（1）数据一致性问题。数据从主节点转到从节点必然会有一个延时的时间窗口，这个时间窗口会导致主从节点之间的数据不一致。某一时刻，在主节点和从节点中A数据的值都为X，之后将主节点中A的值修改为Y，那么在这个变更通知到从节点之前，应用读取从节点中的A数据的值并不为最新的Y，由此便产生了数据不一致的问题。

（2）延时问题。类似Redis这种组件，数据从写入主节点到同步至从节点中的过程需要经历网络→主节点内存→网络→从节点内存这几个阶段，整个过程会耗费一定的时间。而在Kafka中，主从同步会比Redis更加耗时，它需要经历网络→主节点内存→主节点磁盘→网络→从节点内存→从节点磁盘这几个阶段。对延时敏感的应用而言，主写从读的功能并不太适用。

17.Kafka中是怎么体现消息顺序性的？

kafka每个partition中的消息在写入时都是有序的，消费时，**每个partition只能被每一个group中的一个消费者消费**，保证了消费时也是有序的。

整个topic不保证有序。**如果为了保证topic整个有序，那么将partition调整为1。**

首先kafka只保证partition内的消息是顺序的，并不保证topic维度是顺序的。Topic 是一个逻辑概念，是由多个分区文件组成的，因此想要做到 Topic 维度的数据顺序，代价是非常大的，所以 kafka 并不保证 Topic 维度的消息顺序。

- 生产者：保证将消息写入一个分区内。
- 服务端：创建只有一个分区的topic
- 消费端：保证一个线程消费一个分区。

生产者：

由于Kafka的一个Topic可以分为多个Partition，Partition中的数据是有序的，但是从多个Partition中读出来后的数据会变成无序的。因此要满足全局有序只能让一个Topic只有一个Partition。可以通过生产者在写消息的时候指定一个key，通过partitionKey将某类消息写入同一个Partition中。

服务端：

生产者发送数据到队列时，通过加锁保证有序。Kafka通过引入生产者的幂等性来解决重传机制引起的消息乱序。

每一个新的生产者在初始化的时候会生成一个Producer ID，即PID。对于每一个PID，该Producer发送消息的每个<Topic,Partition> 都对应一个单调递增的Sequence Number。broker端会缓存这个序号。每收到一条消息，判断该消息的序号是否比缓存的序号大1，是则接收，不是则丢弃。

消费者：

消费者端要保证顺序消费的话，只能使用单线程（效率低）或者保证消费顺序的线程模型。

解决方案：

消费者端创建多个内存队列，具有相同key的数据都保存到同一个队列中，然后让每个线程分别消费一个内存队列即可

18.消费者提交消费位移时提交的是当前消费到的最新消息的offset还是offset+1?

offset+1

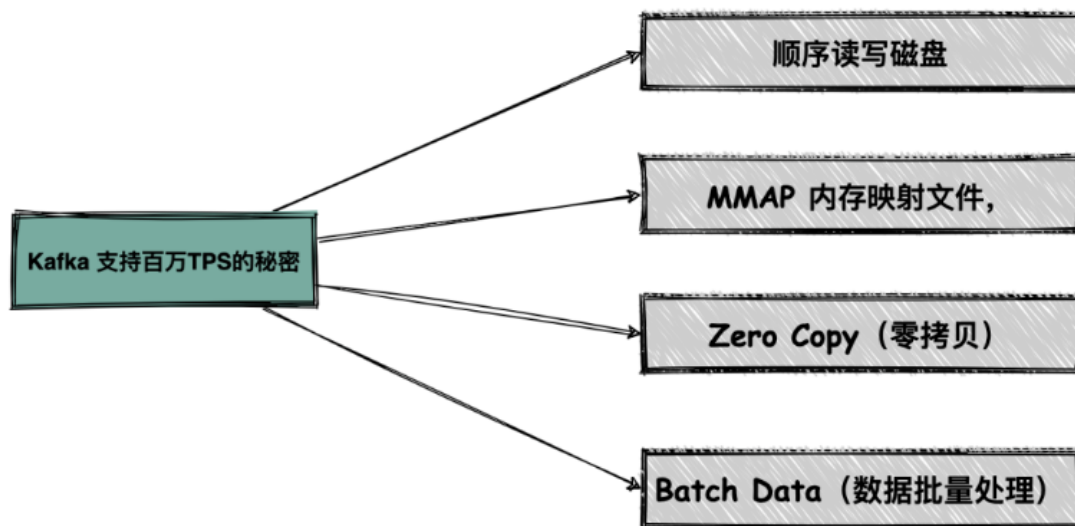
19.kafka如何实现延迟队列?

Kafka并没有使用JDK自带的Timer或者DelayQueue来实现延迟的功能，而是基于时间轮自定义了一个用于实现延迟功能的定时器（SystemTimer）。JDK的Timer和DelayQueue插入和删除操作的平均时间复杂度为 $O(n\log(n))$ ，并不能满足Kafka的高性能要求，而基于时间轮可以将插入和删除操作的时间复杂度都降为 $O(1)$ 。时间轮的应用并非Kafka独有，其应用场景还有很多，在Netty、Akka、Quartz、Zookeeper等组件中都存在时间轮的踪影。

底层使用数组实现，数组中的每个元素可以存放一个TimerTaskList对象。TimerTaskList是一个环形双向链表，在其中的链表项TimerTaskEntry中封装了真正的定时任务TimerTask。

Kafka中到底是怎么推进时间的呢？Kafka中的定时器借助了JDK中的DelayQueue来协助推进时间轮。具体做法是对于每个使用到的TimerTaskList都会加入到DelayQueue中。Kafka中的TimingWheel专门用来执行插入和删除TimerTaskEntry的操作，而DelayQueue专门负责时间推进的任务。再试想一下，DelayQueue中的第一个超时任务列表的expiration为200ms，第二个超时任务为840ms，这里获取DelayQueue的队头只需要 $O(1)$ 的时间复杂度。如果采用每秒定时推进，那么获取到第一个超时的任务列表时执行的200次推进中有199次属于“空推进”，而获取到第二个超时任务时有需要执行639次“空推进”，这样会无故空耗机器的性能资源，这里采用DelayQueue来辅助以少量空间换时间，从而做到了“精准推进”。Kafka中的定时器真可谓是“知人善用”，用TimingWheel做最擅长的任务添加和删除操作，而用DelayQueue做最擅长的时间推进工作，相辅相成。

20.kafka如何支持百万级TPS



Kafka 支持百万TPS的秘密

TPS和QPS

TPS (Transaction Per Second) 每秒处理的事务数。

站在宏观角度来说，一个事务是指客户端向服务端发起一个请求，并且等到请求返回之后的整个过程。从客户端发起请求开始计时，等到收到服务器端响应结果后结束计时，在计算这个时间段内总共完成的事务个数，我们称为TPS。

站在微观角度来说，一个数据库的事务操作，从开始事务到事务提交完成，表示一个完整事务，这个是数据库层面的TPS。

QPS (Queries Per Second) 每秒查询数，表示服务器端每秒能够响应的查询次数。这里的查询是指用户发出请求到服务器做出响应成功的次数，可以简单认为每秒钟的Request数量。

针对单个接口而言，TPS和QPS是相等的。如果从宏观层面来说，用户打开一个页面到页面渲染结束代表一个TPS，那这个页面中会调用服务器很多次，比如加载静态资源、查询服务器端的渲染数据等，就会产生两个QPS，因此，一个TPS中可能会包含多个QPS。

$QPS = \text{并发数} / \text{平均响应时间}$

顺序读写磁盘：按记录的逻辑顺序进行读写操作的存取方法，即按照信息在存储器中的实际位置决定的顺序使用信息；读写速率与随机读写相比快很多

zero copy (MMAP)：直接使用操作系统的page实现文件到物理内存的直接映射，完成映射后对物理内存的操作会被同步到硬盘上；通过MMAP技术可以像读写硬盘一样读写内存，可以获得很大的I/O性能提升，省去了用户空间到内核空间复制的开销

zero copy (send file)：数据直接在内核完成输入和输出，不需要拷贝到用户空间再写出去。kafka数据写入磁盘前，数据先写到进程的内存空间。减少数据传输次数

磁盘数据通过 DMA 拷贝到内核态 Buffer 后，直接通过 DMA 拷贝到 NIC Buffer(socket buffer)，无需 CPU 拷贝。这也是零拷贝这一说法的来源。除了减少数据拷贝外，因为整个读文件 - 网络发送由一个 sendfile 调用完成，整个过程只有两次上下文切换，因此大大提高了性能。

其实这项零拷贝技术，直接从内核空间 (DMA的) 到内核空间 (Socket的)、然后发送网卡。

应用的场景非常多，如Tomcat、Nginx、Apache等web服务器返回静态资源等，将数据用网络发送出去，都运用了sendfile。

简单理解 sendfile(in,out)就是，磁盘文件读取到操作系统内核缓冲区后、直接扔给网卡，发送网络数据。

batch data：读取数据时，kafka会直接将多条数据批量压缩成一个文件发送给消费者，消费者可通过offset进行消费，从而减少I/O消耗

mmap 和 sendfile总结

- 1、都是Linux内核提供、实现零拷贝的API;
- 2、sendfile 是将读到内核空间的数据，转到socket buffer，进行网络发送;
- 3、mmap将磁盘文件映射到内存，支持读和写，对内存的操作会反映在磁盘文件上。

RocketMQ 在消费消息时，使用了 mmap。kafka 使用了 sendFile。

什么是死锁 用代码实现

[死锁](#)常见于有多个协程阻塞，而当主协程中如果有channel发生阻塞，则必然死锁，死锁的直接原因在于channel的读与写没有成对出现，只有读，或者只有写都会造成死锁。

死锁的发生离不开阻塞，channel的阻塞分两种：

- 1.对于无缓冲channel，因为其容量为0，数据只读不写，或者只写不读都会造成堵塞
- 2.对于有缓冲channel，因为设置了容量，在channel容量满之前，仅往里面写数据而不读取是不会导致阻塞的，不会使得协程挂起，但如果channel满了还往里写入数据，就会导致阻塞

无缓冲区channel本身是用于不同协程间通信的，一般一个协程作为生产者，另一个作为消费者，遵循下面两个原则：

当上一次生产到channel的数据未被消费时，生产者继续生产将会阻塞

当生产者停止生产时，消费者继续消费将会阻塞

所以，当我们写的程序生产和消费的数量不对等时，便会发生阻塞造成死锁

举例子：

只有一个协程时，往一个channel写或者读数据

```
1 package main
2 import "fmt"
3 func main() {
4     ch1 := make(chan int)
5     fmt.Println(<-ch1)
6     ch := make(chan int, 3)
7     // 在无数据填入ch时，执行fmt.Println(<-ch)，会发生死锁
8     // fmt.Println(<-ch)
9     ch <- 1
10    ch <- 2
11    ch <- 3
12
13    for v := range ch {
14        fmt.Println(v)
15        //在读取完ch时，若不关闭ch，继续读取，也同样会发生死锁
16        if v==3{
17            close(ch)
18        }
19    }
20 }
21
```

```
22
23
24 //例子二，四个发送，三个接收，发送与接收数量不对等
25 package main
26
27 import (
28     "fmt"
29     "sync"
30 )
31 var wg sync.WaitGroup
32
33 func producer(ch chan<- int) {
34     defer wg.Done()
35     for i := 1; i <= 4; i++ {
36         ch <- i
37     }
38 }
39
40 func consumer(ch <-chan int){
41     defer wg.Done()
42     for i := 1; i <= 3; i++ {
43         fmt.Printf("result: %d\n", <-ch)
44     }
45 }
46
47 func main() {
48     ch := make(chan int)
49
50     wg.Add(2)
51     go producer(ch)
52     go consumer(ch)
53
54     wg.Wait()
55
56
57 }
```

nciwo